

Certified path tracking and applications

Suivi de racines certifié et applications

Thèse de doctorat de l'université Paris-Saclay

École doctorale n°574 : mathématiques Hadamard (EDMH)

Spécialité de doctorat : mathématiques aux interfaces

Graduate School : mathématiques

Référent : Université de Versailles Saint-Quentin-en-Yvelines

Thèse préparée dans l'unité de recherche **Inria Saclay-Île-de-France (Université Paris-Saclay, Inria)**, sous la direction de **Pierre LAIREZ**, chargé de recherche

Thèse soutenue à Paris-Saclay, le 9 septembre 2026, par

Alexandre GUILLEMOT

Composition du jury

Membres du jury avec voix délibérative

Mioara JOLDES

Directrice de recherche, LAAS-CNRS

Anton LEYKIN

Professeur, Georgia Tech, School of Mathematics

Mohab SAFEY EL DIN

Professeur des universités, LIP6, Sorbonne Université

Simon TELEN

Professeur, Max Planck Institute for Mathematics in the Sciences

Examinatrice

Rapporteur & Examineur

Président

Rapporteur & Examineur

Titre: Suivi de racines certifié et applications

Mots clés: Calcul formel, analyse numérique, algorithmes, suivi de racines

Résumé: Cette thèse est centrée sur le développement d'un algorithme de suivi de racines certifié, *Track*, pour le suivi numérique des zéros de systèmes polynomiaux paramétrés. La certification repose sur la méthode de Krawczyk pour isoler un zéro d'un système polynomial, rendue effective grâce à l'arithmétique d'intervalles. Nous prouvons la correction et la terminaison de *Track* dans un modèle de calcul réaliste, de sorte qu'il n'y ait pas de décalage entre la théorie et ce qui est implémenté. *Track* est implémenté en *Rust* dans le logiciel *Algpath*. Des résultats expérimentaux montrent une nette amélioration par rapport aux méthodes certifiées existantes.

Nous développons deux applications par des approches symboliques-numériques

basées sur le suivi de racines certifié. Premièrement, on s'intéresse aux calculs de monodromie. En utilisant *Algpath*, nous calculons, pour la première fois de façon rigoureuse, les triplets de monodromie des 1111 applications de Belyi de la base de données *LMFDB*. Deuxièmement, nous étudions le calcul de tresses à partir de données approchées. Cela s'applique aux tresses obtenues en suivant les racines de polynômes paramétrés par suivi de racines certifié. Les algorithmes sont implantés en *Rust* dans le logiciel *Braids*, qui dépend de *Algpath*.

Enfin, nous discutons l'implantation de logiciels mathématiques en *Rust*, à partir de notre expérience avec *Algpath*. Plus précisément, nous nous concentrons sur les bénéfices et les limites de la programmation générique en *Rust*.

Title: Certified path tracking and applications

Keywords: Computer algebra, numerical analysis, algorithms, homotopy continuation

Abstract: This thesis centers on the development of a certified homotopy continuation algorithm, *Track*, for the numerical continuation of zeros of parametrized polynomial systems. Certification is based on Krawczyk's method for isolating a zero of a polynomial system, made effective using interval arithmetic. We prove correctness and termination of *Track* in a realistic computational model, so that there is no discrepancy between theory and what is implemented. *Track* is implemented in the *Rust* software package *Algpath*. Experiments show a large improvement over existing certified methods.

We develop two applications through symbolic-numeric approaches based on certi-

fied homotopy continuation. First, we focus on monodromy computations. Using *Algpath*, we compute, for the first time rigorously, the monodromy triples of all 1111 Belyi maps in the *LMFDB* database. Second, we study braid computations from approximate data. This applies to braids obtained by tracking the roots of parametrized polynomials using certified homotopy continuation. The algorithms are implemented in the *Rust* software package *Braids*, which relies on *Algpath*.

Finally, we discuss implementing mathematical software in *Rust*, drawing from our experience with *Algpath*. Precisely, we focus on the benefits and the limitations of generics in *Rust*.

Résumé

Soit $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ une application polynomiale paramétrée, ce qui signifie que les n composantes de F sont des polynômes en $n + 1$ variables. Si $t \in \mathbb{C}$, on dénote par $F_t : \mathbb{C}^n \rightarrow \mathbb{C}^n$ la spécialisation de F par t en sa première variable. Soit $\zeta_0 \in \mathbb{C}^n$ un zéro régulier de F_0 . En supposant le problème bien posé, il va exister un unique chemin continu et régulier $\zeta : [0, 1] \rightarrow \mathbb{C}^n$ tel que $\zeta(0) = \zeta_0$ et pour tout $t \in [0, 1]$, $F_t(\zeta(t)) = 0$. On appelle ζ le *chemin de solution associé* à ζ_0 .

L’objectif d’un algorithme de suivi de racines est de calculer numériquement ζ , étant donné F et ζ_0 . Une des applications principales est la résolution numérique de systèmes polynomiaux ; dans ce cas, une approche heuristique couplée à une certification *a posteriori* des zéros du système peut suffire. Ces algorithmes peuvent également servir à calculer des données discrètes associées à F (permutations de monodromie, tresses) ; dans ce cas il est nécessaire de *certifier* le suivi de racines pour garantir un résultat correct. Dans cette thèse, nous développons un algorithme de suivi de racines certifié et nous l’appliquons pour résoudre des problèmes liés à l’étude de la topologie des variétés algébriques complexes, via des approches symbolique-numériques.

Suivi de racines certifié Nous commençons par décrire le modèle de calcul : on suppose une arithmétique d’intervalles à précision adaptative, qui bénéficie de bonnes propriétés de convergence lorsque la précision de calcul tend vers l’infini. Ces hypothèses, essentielles pour les preuves de terminaison, sont satisfaites par l’arithmétique d’intervalles implantée dans MPFI [76].

Dans ce modèle de calcul, nous développons un algorithme de suivi de racines certifié, **Track**, en nous basant sur des idées proposées par van der Hoeven [93]. La certification se base sur la méthode de Krawczyk, que l’arithmétique d’intervalles rend effective, ainsi que sur les modèles de Taylor, qui permettent de certifier des prédicteurs d’ordre supérieur.

Algpath est une implantation en Rust de **Track**, disponible à l’adresse

<https://gitlab.inria.fr/numag/algpath>.

Nous présentons des tests de performances qui montrent une nette amélioration par rapport aux méthodes certifiées existantes (**Macaulay2** avec **tracker** certifié [10]), ainsi qu’une réduction de l’écart de performances avec les méthodes non certifiées (**HomotopyContinuation.jl** [21]).

L’implantation en Rust est rendue difficile par la cohabitation entre du code bas niveau et du code haut niveau, ainsi que par la généricité des algorithmes vis-à-vis de l’arithmétique d’intervalles. Nous discutons de ces difficultés ainsi que des solutions possibles pour les surmonter.

Monodromie des applications de Belyi Considérons de nouveau une application polynomiale paramétrée F , et à la place de déplacer le paramètre de 0 à 1, nous le déplaçons le long d’une boucle de paramètre $\gamma : [0, 1] \rightarrow \mathbb{C}$. Notons $b = \gamma(0) = \gamma(1)$ le point base de la boucle, et X_b les solutions de F_b . En relevant γ en un chemin de solution partant de chaque $x \in X_b$ et en identifiant les points de départ et d’arrivée de l’ensemble des chemins de solution, on obtient une permutation de X_b , dite *permutation de monodromie*. Les méthodes de suivi de racines permettent de calculer de telles permutations, et la correction du résultat est assurée dès lors que le suivi de racines est certifié.

Nous utilisons **Algpath** pour calculer, pour la première fois rigoureusement, la monodromie des 1111 applications de Belyi présentes dans la L-functions and Modular Forms Data Base (LMFDB) [90]. Cela permet notamment de prouver la correction des groupes de monodromie de l’ensemble des entrées.

Calcul de tresses Lorsque n points se déplacent continûment dans le plan complexe sans entrer en collision, ils définissent une tresse géométrique. Artin [2] a démontré que les tresses peuvent être décomposées en tresses élémentaires, ce qui permet de donner une description combinatoire de la tresse géométrique. Nous nous intéressons au problème de calculer la tresse combinatoire associée à la tresse géométrique induite par le déplacement des racines d’un polynôme $g \in \mathbb{C}[t][z]$ lorsque l’on déplace t continûment.

La nature topologique des tresses amène naturellement à considérer une approche symbolique-numérique : on calcule une approximation numérique certifiée du déplacement des racines, qui nous permet de calculer de manière correcte la tresse combinatoire associée. Pour la première étape, nous utilisons un algorithme de suivi de racines certifié. Bien que la deuxième étape ait déjà été abordée dans [64], l’algorithme qui y est implanté s’adapte mal à la sortie de **Algpath**. Pour cette raison, nous développons un nouvel algorithme pour cette deuxième étape. Il se base sur la description de la tresse via un prédicat géométrique **Sep**, qui permet d’obtenir de l’information sur la position relative des brins de la tresse, mais qui ne requiert pas d’avoir leur position exacte pour être implanté. Nous présentons un algorithme **Braid** qui calcule la tresse combinatoire associée à une tresse géométrique représentée par une fonction **Sep**. **Braid** est implanté dans le projet **Braids**, disponible à l’adresse

<https://gitlab.inria.fr/aguillem/braids>.

Le logiciel permet le calcul de tresses associées au déplacement des racines d’un polynôme bivarié, en utilisant **Algpath** pour l’étape de suivi de racines certifié.

Írta

Remerciements

Premièrement, je remercie chaleureusement Pierre, pour m’avoir encadré pendant ces trois années de thèse. Tu as toujours été disponible pour m’aider à résoudre tout un tas de problèmes (scientifiques ou pas). J’adore la passion hautement contagieuse et la maîtrise impressionnante que tu as dans une variété incroyable de domaines ; ça m’a permis d’apprendre tellement de choses dans plein de sujets. Merci aussi pour la bonne humeur et les blagues, les soirées alcoolisées, les jeux, le rituel du café, le bon vin et les bières pour Guillaume.

I also warmly thank Anton and Simon for accepting to review this manuscript, and Mioara and Mohab for being part of the jury of my defense.

Je remercie Anna-Maria, Grégoire et Pierre-Guy pour avoir fait partie de mon CSI et pour l’attention que vous avez portée au bon déroulement de ma thèse.

Je remercie Frédéric, pour ton attention particulière pour tout ce qui touche à la vie administrative du doctorant, pour m’avoir aidé de nombreuses fois avec le cluster et pour les discussions—je dois dire parfois difficiles à suivre pour moi—aux pauses de midi.

Je remercie aussi chaleureusement Bahar et Katia, qui ont toujours été d’une grande aide.

Je remercie Éric, ami et grand frère dans le monde de la recherche. Tu m’as appris un paquet de choses très utiles et m’as énormément aidé pour m’intégrer pendant ma thèse. Je vais faire de mon mieux pour que mon code puisse calculer des tresses blazing fast et pour trouver du temps pour faire des parties de StS.

Je remercie Hadrien pour les sessions escalade, pour les memes, la bonne ambiance dans le bureau et les kouign-amann qui tassent le ventre. D’ailleurs, j’aimerais bien savoir si tu es finalement venu à bout de ton puzzle de 13000 pièces.

I thank Ricardo for helping me so many times whenever I had issues with the cluster, my computer or Sage. I was always impressed by your skills when it comes to actually using a computer.

I thank Rafael for all the good times at Mathexp and at conferences, all the strange German words that you tried to teach me, and for all the beers we had here and there (even the mango one).

Je remercie aussi Théo pour avoir partagé pendant une bonne période le bureau avec moi. On a des personnalités assez différentes et tu dois un peu penser que je suis un boomer, mais c’est toujours un plaisir de discuter avec toi.

Je remercie aussi toutes les personnes avec qui j’ai partagé les locaux de Mathexp : Alin, Guy, Philippe, Alexandre Goyer, Hadrien N., Linus, Claudia, Jaali et Julie.

Je remercie aussi les voisins de l’équipe Max. Merci à Yulia, Albin, Arnaud et Alixe pour les conversations quand je venais vous alpaguer dans votre bureau

après être allé chercher un café à la machine.

Je remercie toutes les personnes avec qui j'ai passé du temps en conférence : Louis, Rémi, Lucas, Alaa, Edern, Robin, Florent, Sriram, Weijia, Jérémy, Sara et les autres.

D'ailleurs, il y a deux personnes qui se reconnaîtront dans ce paragraphe (l'un au palais de dobermann et l'autre auto-stoppeur de légende) ; je ne suis pas complètement sûr de si je dois vous remercier. En tout cas, j'ai passé avec vous des soirées inoubliables—surtout pour leur côté désastreux—au CIRM.

Il faut aussi que je remercie Gabriel. Ton intelligence et ta culture scientifique rendent nos discussions aussi passionnantes qu'enrichissantes (et pardonne-moi si j'oublie tout d'une fois sur l'autre). Aussi je pense qu'avec tous les cours particuliers de Rust que tu m'as donnés, tu mériterais bien d'être cité comme collaborateur dans le développement de **Algpath**.

Je remercie aussi Ulysse, pour les discussions interminables (potentiellement à 2 h du matin en plein rush d'écriture de manuscrit) à propos d'optimisations barbares dans du code et autre sujet bien technique. Tu es une des personnes qui me comprennent le mieux, tant pour le niveau d'entêtement que j'ai pour des choses insignifiantes que pour la franche rigolade. Bon et n'oublie pas, la Casio Graph 35+ est venue à bout du projet Euler 14 (au prix d'un jeu de piles AAA), il faut venir à Lyon faire la vaisselle maintenant.

Je remercie Pierre, coloc aux cheveux bleus tout aussi bienveillant que déjanté. Tu m'as toujours encouragé pendant les moments les plus durs de l'écriture de mon manuscrit. Je sais que ça t'aurait bien fait marrer que je remercie des gens qui n'existent pas ; au final j'ai fait des remerciements assez normaux (mais sache que j'ai commenté un paragraphe qui remercie les corbeaux d'avoir cessé de taper frénétiquement sur la fenêtre de notre appart courant avril 2026).

Enfin, je remercie Marie de tout mon cœur. Les mots ne suffisent pas pour décrire l'importance de ta présence dans ma vie. À tes côtés, toutes les épreuves paraissent si faciles à surmonter, la vie est si drôle et passe si vite.

AI usage disclaimer

In the writing of this manuscript, GPT was used through Codex and ChatGPT to check language, find mathematical inaccuracies and correct typographical errors.

Contents

1	Introduction	1
1.1	Context	1
1.2	Homotopy continuation	6
1.3	Monodromy	14
1.4	Braids	17
2	Certified homotopy continuation	19
2.1	Computational model	19
2.2	Data structure for regular zeros of polynomial systems	29
2.3	Path tracking	43
2.4	Faster path tracking with predictors	49
2.5	Certified path tracking in projective space	55
2.6	Algpath	60
3	Monodromy and Belyi maps	69
3.1	Monodromy computations	69
3.2	Belyi maps	72
3.3	Monodromy verification of Belyi maps	74
4	Braid computations	79
4.1	Braids	80
4.2	Approximate data and arrangements	84
4.3	Computing braids from arrangements	92
4.4	Implementation	98
5	Writing mathematical code in Rust	115
5.1	Generics	116
5.2	Specialization pitfalls	125
5.3	Trait pitfalls	128
5.4	Towards computer algebra in Rust	134
6	Perspectives	143
6.1	Faster certified path tracking	143
6.2	Connectivity queries in real algebraic curves	145
6.3	Topology of algebraic varieties	146

Chapter 1

Introduction

1.1 Context

A *polynomial equation* is an equation of the form

$$a_0 + \cdots + a_d x^d = 0 \tag{1.1}$$

where $a_0, \dots, a_d$ are fixed coefficients and x is the indeterminate. Polynomial equations appear in many problems, whether from real-life applications, other sciences, or mathematics itself.

A natural and fundamental question to ask about polynomial equations is how to solve them. For equations of degree up to $d = 4$, there are explicit formulas for the solutions in terms of the coefficients of (1.1), involving additions, subtractions, multiplications, divisions, and k -th root extractions. In this case, we say that (1.1) can be solved *in radicals*. Unfortunately, no general expression in radicals for the solutions of (1.1) exists when $d \geq 5$, as proven by Abel in 1824. Note that there exist closed-form expressions for the solutions of (1.1) other than expressions in radicals; for instance, Sturmfels [87] constructs the solutions of (1.1) as hypergeometric series.

Computer algebra A general formula for the solutions of (1.1) yields an automatic way—an *algorithm*—to compute the roots of a polynomial with specified coefficients. More generally, *computer algebra* focuses on efficient algorithms for manipulating mathematical objects.

The data structures used to represent the objects under consideration play a crucial role. For instance, from the previous paragraph, we know that representing the solutions of (1.1) as expressions in radicals is too restrictive. On the other hand, we may be satisfied with *numerical approximations* of the solutions. If we are interested in the complex roots of a polynomial with rational coefficients, we

can obtain approximations of them in **SageMath**, a standard computer algebra system.

```
sage: f
-1/7*x^5 - 2*x^4 + x^3 + 11/2*x^2 - 3*x - 2
sage: f.roots(CC, multiplicities = False)
[-14.2944267232683,
 -1.66952699597698,
 -0.405239398453707,
 1.18459655884950 - 0.210609498501505*I,
 1.18459655884950 + 0.210609498501505*I]
```

Note that the polynomial f in the example above *cannot* be solved in radicals, as its Galois group is S_5 , the symmetric group of degree 5.

```
sage: NumberField(f, name = "nu").galois_group()
Galois group 5T5 (S5) with order 120 of
-1/7*x^5 - 2*x^4 + x^3 + 11/2*x^2 - 3*x - 2
```

Computer algebra emphasizes *exact* representations and computations: a numerical approximation is inexact, as opposed to the symbolic nature of expressions in radicals. With additional guarantees, such as an error bound, a numerical approximation can become a suitable data structure for the zeros of a polynomial equation. This leads to *symbolic-numeric* approaches, where computer algebra and numerical analysis meet. For instance, in **SageMath**,

```
sage: f.roots(ComplexBallField(30), multiplicities = False)
[[-14.294427 +/- 3.80e-7] + [+/- 1.16e-7]*I,
 [-1.66952700 +/- 4.93e-9] + [+/- 8.09e-10]*I,
 [-0.405239398 +/- 4.56e-10] + [+/- 1.98e-12]*I,
 [1.18459656 +/- 2.14e-9] + [0.21060950 +/- 2.60e-9]*I,
 [1.18459656 +/- 2.14e-9] + [-0.21060950 +/- 2.60e-9]*I]
```

yields rectangular boxes representing the complex roots of f , in the sense that each box contains exactly one root.

Computational algebraic geometry Generalizing the previous paragraph, a *polynomial system* is a system of n equations

$$\begin{cases} f_1 = 0, \\ \vdots \\ f_n = 0, \end{cases} \quad (1.2)$$

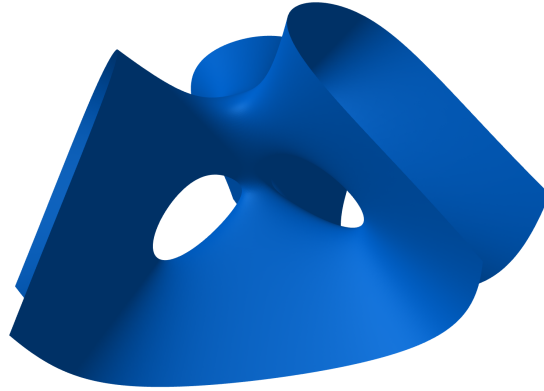


Figure 1.1: Real solution set of Clebsch's cubic (picture generated with SageMath).

where $f_1, \dots, f_n$ are multivariate polynomials in the variables $x_1, \dots, x_m$. We will now assume that the f_i have complex coefficients. We denote by $X = V(f_1, \dots, f_n) \subseteq \mathbb{C}^m$ the solution set of (1.2), that is

$$X = \{a \in \mathbb{C}^m : f_1(a) = \dots = f_n(a) = 0\}$$

The fundamental theorem of algebra states that a univariate polynomial equation of degree d with complex coefficients has d roots counted with multiplicity. With polynomial systems, the situation is much more diverse: X can be a finite set, the empty set, or a higher-dimensional object.

Symbolic approach At its core, *algebraic geometry* aims to study the geometric properties of the solution set X of a given polynomial system. From the computational point of view, symbolic tools, such as *Gröbner bases*, make it possible to address such questions algorithmically: is X empty? what is the dimension of X ? is X smooth? if X is a finite set, can we compute it numerically? They also allow us to compute equations for objects associated with X , such as its projection (precisely, its Zariski closure) onto a linear subspace of $\mathbb{C}^m$.

Example 1.1. Consider Clebsch's cubic, defined by a single equation in 3 variables

$$\begin{aligned} f = & 81(x^3 + y^3 + z^3) - 189(x^2(y + z) + y^2(z + x) + z^2(x + y)) + 54xyz \\ & + 126(xy + yz + zx) - 9(x^2 + y^2 + z^2) - 9(x + y + z) + 1. \end{aligned}$$

The solution set $V(f)$ is depicted in Figure 1.1. A point $a \in V(f)$ is singular if

$$\partial_x f(a) = \partial_y f(a) = \partial_z f(a) = 0.$$

Using SageMath, we can check that $V(f)$ has no singular points by verifying that $f = \partial_x f = \partial_y f = \partial_z f = 0$ has no solutions, using a symbolic computation:

```
sage: QQ[x, y, z].ideal([f] + f.gradient()).is_trivial()
True
```

◇

Numerical algebraic geometry Symbolic tools study X through algebraic computations with its defining equations, whereas *numerical algebraic geometry* relies on numerical techniques to study X and related objects. A central computational tool of the latter is *homotopy continuation*, also called *path tracking*.

Example 1.2. Consider the system $f = (f_1, f_2)$ given by 2 equations in 3 variables

$$\begin{cases} 18x^3 - 4x^2y + 18xy^2 - 4y^3 - 5x^2z - 5y^2z + 18xz^2 - 4yz^2 - 5z^3 - 18x + 4y + 5z \\ 7x^3 - 7x^2y + 7xy^2 - 7y^3 + x^2z + y^2z + 7xz^2 - 7yz^2 + z^3 - 7x + 7y - z \end{cases}$$

We can numerically compute the decomposition of $V(f)$ into irreducible components using, for instance, `HomotopyContinuation.jl` [21]:

```
julia> using HomotopyContinuation

julia> @var x, y, z
(x, y, z)

julia> f = System([...])
...

julia> numerical_irreducible_decomposition(f, show_progress = false)
Numerical irreducible decomposition with 2 components
=====
• 1 component(s) of dimension 2.
• 1 component(s) of dimension 1.
...
```

In fact, f was rigged from the start: it can be factored as

$$\begin{cases} (x^2 + y^2 + z^2 - 1)(18x - 4y - 5z) \\ (x^2 + y^2 + z^2 - 1)(7x - 7y + z), \end{cases}$$

and its solution set decomposes as the 2-sphere together with a line.

◇

Symbolic-numeric methods Without additional guarantees, the information about X obtained by numerical techniques is not guaranteed to be correct, which is unsatisfactory from a mathematical point of view. *Certified homotopy continuation* algorithms address this issue: they provide a form of certification for their results, ensuring that the computed information about X is correct, even though the core computations are numerical. Certified homotopy continuation can serve as the basis of symbolic-numeric approaches, such as the monodromy group computation in the following example.

Example 1.3. Consider the bivariate polynomial

$$f_t(x) = (11\mathbf{i} + 2)x^5t^2 + 8\mathbf{i}x^5t + 5xt + (-\mathbf{i} + 2)x - t - 2\mathbf{i}$$

For reference, this polynomial was taken from the LMFDB [90, 68] database (entry 5T3-5_4.1_4.1-a of the Belyi map section). As in the introductory example, we may consider f_t as a univariate polynomial, seen as an element of $\mathbb{C}(t)[x]$. A symbolic computation shows that f_t is square-free.

```
sage: gcd(f, f.derivative(x))
1
```

Thus, we can consider its Galois group over $\mathbb{C}(t)$, which provides information on the structure of its solutions. For such equations, this Galois group is isomorphic to the monodromy group induced by the projection onto the t variable of the set $V(f) = \{(t, x) \in \mathbb{C}^2 : f_t(x) = 0\}$, as proven by Harris [45]. This monodromy group can be computed using homotopy continuation, and its correctness is guaranteed with certification. Using **Algp**ath, the certified homotopy continuation software that will be presented in this document, we can compute this monodromy group.

The resulting group is identified to be the Frobenius group F_5 of order 20, and is solvable.

```
sage: G = PermutationGroup(["(1,3,4,5)", "(1,2,4,5,3)"])
sage: G.is_solvable()
True
```

Thus f_t can be solved in radicals over $\mathbb{C}(t)$. In fact, we can also consider the Galois group of f_t over $\mathbb{Q}(\mathbf{i})(t)$, and prove that it is also F_5 . Thus f_t is solvable in radicals over $\mathbb{Q}(\mathbf{i})(t)$, and we could try to describe its solutions with expressions in radicals using for instance the formulas provided by Dummit [36]. $\diamond$

Certified homotopy continuation and applications This thesis is centered on the development of a certified homotopy continuation algorithm—called **Track**—and software. First, we provide the tools needed to define **Track**: we review

Krawczyk’s operator, Moore’s criterion and develop a precise computational model, with an emphasis on practical considerations. We describe **Track**, and discuss further optimizations. We also present **Algpath**, a **Rust** implementation of **Track**, and compare it against other certified and non-certified implementations.

Then, we move on to applications. First, we focus on monodromy computations, for which certification is key. The performance improvement of **Algpath** over other certified algorithms makes it possible to tackle previously out-of-reach problems: for the first time, we rigorously compute the monodromy of all Belyi maps in the **LMFDB** database. We then turn to the computation of braids. Homotopy continuation can be used to obtain a numerical description of the geometric braid induced by the root motion of a bivariate polynomial. On the other hand, braids have a nice combinatorial description. We explain how to transform a geometric braid, obtained via certified homotopy continuation, into its combinatorial counterpart.

The last chapter is devoted to implementation. **Algpath** is implemented in **Rust**, a low-level compiled programming language. The implementation is complicated by the coexistence of low-level and high-level code. An important aspect is its extensive use of generic programming to avoid code duplication. Generic programming is challenging in **Rust** and presents many pitfalls. Drawing on our experience implementing **Algpath**, we explore different aspects of generics in **Rust** and discuss their benefits and limitations.

1.2 Homotopy continuation

Motivation Let $f = (f_1, \dots, f_n)$ be a square polynomial system (it has as many equations as variables) with complex coefficients, called the *target system*, and assume that its solution set is finite. The goal is to solve f . To this end, we can consider a system $g = (g_1, \dots, g_n)$, called the *start system*. The choice of g depends on f and should be a system that is easy to solve, for instance explicitly. One possible choice is a *roots of unity* start system, where $g_i = 1 - x_i^{d_i}$ with $d_i = \deg f_i$. The solution set X_0 of g consists of $d_1 d_2 \cdots d_n$ points:

$$X_0 = \left\{ \left(e^{\frac{2i\pi k_1}{d_1}}, \dots, e^{\frac{2i\pi k_n}{d_n}} \right) \in \mathbb{C}^n : \forall i \in [n], 0 \leq k_i < d_i \right\}.$$

Here, $[n]$ denotes the set $\{1, \dots, n\}$. Then, we build a continuous path from g to f , called the *homotopy*, in the space of $n \times n$ polynomial systems. For instance, we can consider the linear path

$$F_t(x) = (1 - t)g(x) + tf(x).$$

At $t = 0$, the solution set of $F_0 = g$ is just X_0 . Moving t from 0 to 1 induces a continuous displacement of the zeros of F_t . That is, given $\zeta_0 \in X_0$ and assuming

well-posedness, we can lift the displacement of t to a continuous path $\zeta : [0, 1] \rightarrow \mathbb{C}^n$ such that

1. $\zeta(0) = \zeta_0$;
2. $F_t(\zeta(t)) = 0$, for all $t \in [0, 1]$.

We call ζ a *solution path starting at ζ_0* . For a given $\zeta_0 \in X_0$, if we can compute an associated solution path ζ , then its endpoint $\zeta(1)$ is a solution of $F_1 = f$.

To find *all* solutions of f , the start system g must be well chosen. For instance, choosing g to be a roots of unity start system ensures that, for each solution ζ_1 of f , there exists a solution path that starts from a solution of g and ends at ζ_1 , provided that $[0, 1)$ does not intersect the finite set of complex parameter values for which F_t has a singular solution; this is a particular case of *parameter homotopy* (see Bates, Hauenstein, Sommese, and Wampler [7], for instance). Still, f may have fewer solutions than g , and some solution paths may collide at $t = 1$, or diverge to infinity. A smarter choice of homotopy may lead to fewer zeros to track; this is for instance the goal of *polyhedral homotopy* [49]. Although building a good homotopy is a central topic in homotopy continuation, it is not the focus of this thesis; for a survey on homotopy continuation and numerical algebraic geometry, see [85].

Homotopy continuation The problem of computing the solution path ζ can be tackled using a *homotopy continuation* algorithm.

We slightly generalize the setting of the previous paragraph. Let $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a parametrized polynomial map, meaning that its n components are polynomials in the variables t and $x_1, \dots, x_n$. For instance, F may be a straight-line homotopy, of the form $F(t, x) = (1 - t)g(x) + tf(x)$. For $t \in \mathbb{C}$, denote by $F_t : \mathbb{C}^n \rightarrow \mathbb{C}^n$ the map defined by $F_t(x) = F(t, x)$, for $x \in \mathbb{C}^n$. Let ζ_0 be a regular zero of F_0 , regular meaning that $\det(dF_0(\zeta_0)) \neq 0$. Let $\zeta : [0, 1] \rightarrow \mathbb{C}^n$ be a solution path starting at ζ_0 . We assume that ζ is regular over $[0, 1]$, that is, $\det(dF_t(\zeta(t))) \neq 0$ for all $t \in [0, 1]$ (which implies that ζ is unique).

Homotopy continuation algorithms aim at solving the following problem: given F and ζ_0 , compute $\zeta(1)$ by computing a numerical approximation of ζ . The standard approach is to follow a *predictor-corrector* strategy: starting from $x = \zeta_0$ at $t = 0$, alternate:

Prediction. compute $\tilde{\zeta}$, a continuous path (the *predictor*) over an interval $[t, t + h]$ that starts at $x \approx \zeta(t)$, and that should be a “good” approximation of ζ over $[t, t + h]$;

Correction. bring $\tilde{\zeta}(t + h)$ close to $\zeta(t + h)$ using Newton iterations for F_{t+h} , and replace x by the resulting point. Update t to $t + h$.

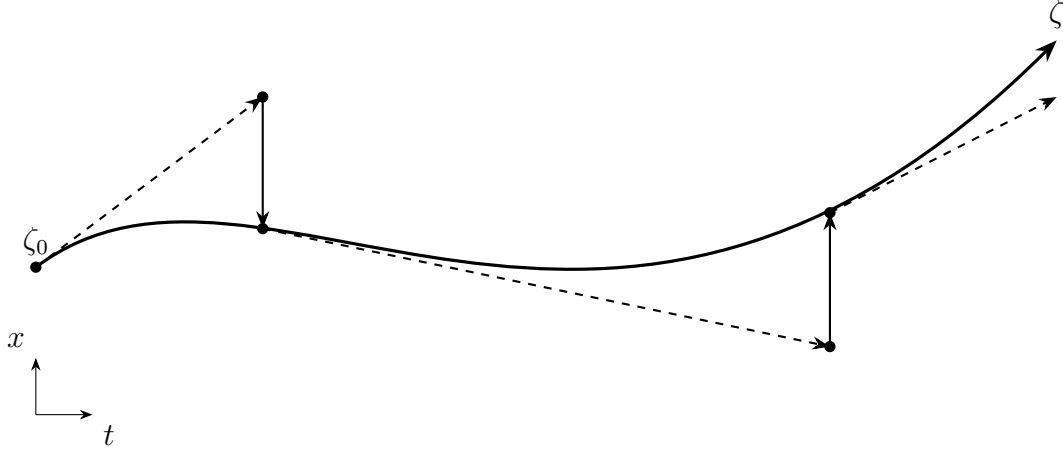


Figure 1.2: Predictor-corrector steps, using the tangent predictor. ζ is the tracked solution path. Dashed arrows correspond to prediction steps. Plain arrows correspond to correction steps.

In the algorithm above, the stepsize h is adaptively chosen at each iteration, for instance according to heuristics or, in the case of *certified* homotopy continuation, to ensure mathematical guarantees on $\tilde{\zeta}$ (for instance, see Dedieu, Malajovich, and Shub [31] and Kranich [57]). The algorithm terminates whenever $t = 1$, and returns x that is a numerical approximation of $\zeta(1)$.

Remark 1.1. In the context of polynomial system solving, the assumption that ζ is regular over $[0, 1]$ is too strong: the target system may have singular zeros, in which case ζ may be regular only over $[0, 1)$. In this case, one can stop tracking a path ending at a singular zero as t approaches 1 and then apply *endgame strategies* (for instance, see [7, Chapter 3]).

Certification Non-certified methods may halt before reaching the end of ζ because heuristics indicate a problem with the solution path (it diverges, it reaches a singularity, precision is lacking, ...). Although the algorithm does not succeed in computing $\zeta(1)$, the reason for stopping can be analyzed to understand the issue, and the result is not incorrect, strictly speaking.

Example 1.4. Consider the following univariate polynomial.

$$\begin{aligned} f = & x^{20} - 210x^{19} + 20615x^{18} - 1256850x^{17} + 53327946x^{16} - 1672280820x^{15} \\ & + 40171771630x^{14} - 756111184500x^{13} + 11310276995381x^{12} \\ & - 135585182899530x^{11} + 1307535010540395x^{10} - 10142299865511450x^9 \\ & + 63030812099294896x^8 - 311333643161390640x^7 + 1206647803780373360x^6 \\ & - 3599979517947607200x^5 + 8037811822645051776x^4 \\ & - 12870931245150988800x^3 + 13803759753640704000x^2 \\ & - 8752948036761600000x + 2432902008176640000. \end{aligned}$$

If we try to solve it using `HomotopyContinuation.jl`, we obtain:

```
julia> res = solve(f)
Result with 15 solutions
=====
• 20 paths tracked
• 15 non-singular solutions (15 real)
...
```

In fact, the number of solutions found varies between runs, with some solutions sometimes considered singular. The attentive reader may have noticed that f is Wilkinson’s polynomial: it has 20 simple roots, as it factors as

$$f = \prod_{i=1}^{20} (x - i).$$

Wilkinson’s polynomial is known for being poorly conditioned: its largest root, 20, is very sensitive to changes in the coefficients, which causes difficulties for homotopy continuation algorithms. $\diamond$

A more problematic issue is *path jumping*: if two solution paths come close to each other, it may happen that the algorithm jumps from one solution path to the other. In this case, the result does not correspond to $\zeta(1)$, although it is a zero of F_1 .

Example 1.5. Here, $F = F_t(x)$ is a univariate homotopy of degree 50 with respect to x that has clustered zeros at $t = \frac{1}{2}$. Because the example is large, we do not give it explicitly; it was built using the instructions provided by Telen, Van Barel, and Verschelde [89, Section 5.4], with $(n_e, CS, \alpha) = (10, 5, 10)$. When tracking the solution paths of F using `HomotopyContinuation.jl`, we obtain

```
julia> R = solve(F, ...)
Result with 50 solutions
```

```
=====
• 50 paths tracked
• 50 non-singular solutions (0 real)
...
```

Everything seems correct, as we find the expected number of solutions. But:

```
julia> certify(F, R, [1.0])
CertificationResult
=====
• 50 solution candidates given
• 50 certified solution intervals (0 real, 50 complex)
• 49 distinct certified solution intervals (0 real, 49 complex)
```

The `certify` method ensures that all 50 solutions correspond to solutions of F_1 , but only 49 of them are distinct, meaning that one solution is missing and another appears twice: path jumping occurred. Even if `certify` had isolated 50 distinct zeros, path jumping could have occurred: two tracking runs may swap solution paths, leading to a silent error.

Note that this example was specifically constructed to demonstrate that non-certified homotopy continuation results cannot be trusted from a mathematical point of view. `HomotopyContinuation.jl` is powerful software on which we frequently rely when certification is not needed. Moreover, effort has been made to increase the robustness of non-certified homotopy continuation, making it possible to tackle numerically difficult problems such as those presented above without paying the computational cost of certification. For instance, Telen, Van Barel, and Verschelde [89] use Padé approximants, not only to predict the next point, but also to deduce a stepsize that ensures that the approximant is reliable and stays far away from other solution paths. $\diamond$

Let $\tilde{\zeta}$ be the numerical path computed by a homotopy continuation algorithm on input F and ζ_0 , approximating the solution path ζ starting at ζ_0 . In what way does $\tilde{\zeta}$ represent ζ ? Again, a numerical approximation alone can only serve as a heuristic for ζ . The *certified* variant of homotopy continuation typically provides error bounds for $\tilde{\zeta}$, in the form of an isolating tubular neighborhood. That is, for every $t \in [0, 1]$, the output provides a ball $B_t \subseteq \mathbb{C}^n$ around $\tilde{\zeta}(t)$ that contains $\zeta(t)$ and no other zero of F_t (see Figure 1.3). This ensures that path jumping does not occur and can lead to rigorous mathematical results, although experiments show that certified homotopy continuation has a higher computational cost than its heuristic counterpart.

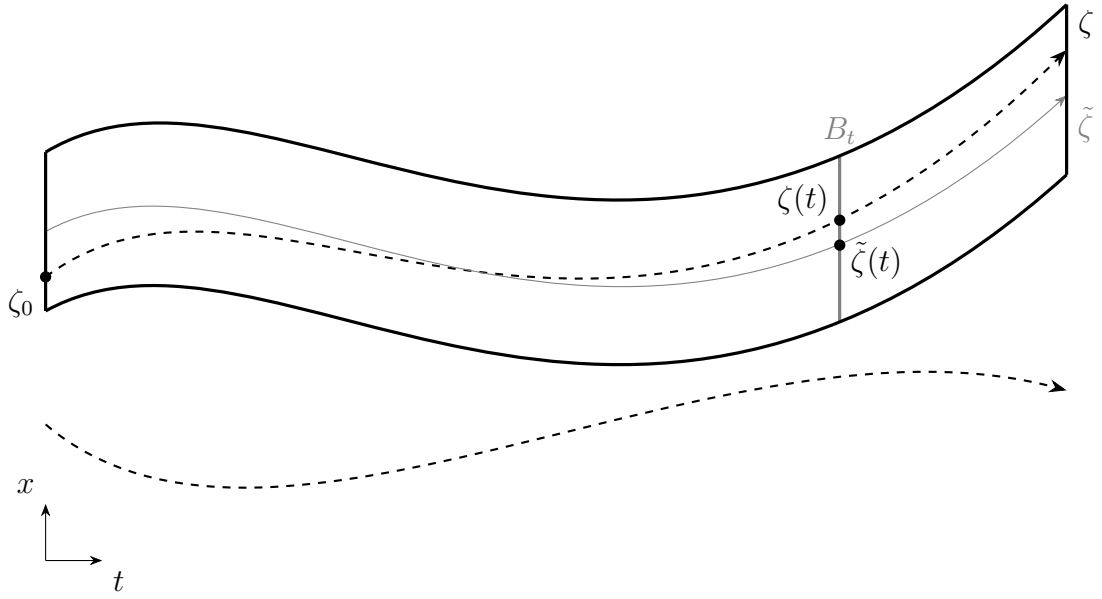


Figure 1.3: Tubular neighborhood enclosing the solution path ζ , as obtained using certified homotopy continuation. At time t , B_t isolates $\zeta(t)$ from other zeros of F_t . The other dashed arrow represents another solution path that does not enter the tubular neighborhood.

1.2.1 Previous work

Homotopy continuation The core idea of homotopy continuation was introduced independently by Garcia and Zangwill [39] and Drexler [32]. Since then, many theoretical works and implementations have been developed; standard references on homotopy continuation include [67, 86]. Implementations include the HOM4PS family [29], PHCpack [95], PHoM [44], POLSYS_GLP [88], Bertini [7], the `NumericalAlgebraicGeometry` package in `Macaulay2` [61] and `HomotopyContinuation.jl` [21]. Although some effort has been made to certify the endpoints [46, 20] and to improve robustness [89], these software packages do not implement certified homotopy continuation algorithms¹.

Smale’s α -theory Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a polynomial map, and $x \in \mathbb{C}^n$. Smale [84] introduces the quantity $\alpha(f, x)$. If

$$\alpha(f, x) \leq \frac{13 - 3\sqrt{17}}{4}, \quad (1.3)$$

then the Newton iteration starting from x for f will converge quadratically to a zero of f ; x is defined to be an *approximate solution of f* if (1.3) holds. Shub and Smale [82, 81] deduce an upper bound on the number of steps required to correctly track a solution path, where correctness is ensured by sufficiently small steps deduced from α -theory. Beltrán [9], Dedieu, Malajovich, and Shub [31], and Bürgisser and Cucker [24] derive an explicit bound on the stepsize using α -theory, and Beltrán and Leykin [10] provide an algorithm that is correct assuming exact computations over the reals (the Blum–Shub–Smale model, see [18, 17]). An implementation is available in `Macaulay2` as part of the `NumericalAlgebraicGeometry` package. Beltrán and Leykin [11] refine their algorithm to ensure certification in the Turing machine model, based on arithmetic over rationals, and provide a standalone implementation.

Also using α -theory, Hauenstein, Haywood, and Liddell [46] develop *a posteriori* certification of a solution path (for instance obtained via heuristic homotopy continuation) for Newton homotopies of the form

$$F_t(x) = f(x) + tv,$$

where $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ is a polynomial map and $v \in \mathbb{C}^n$. Note that this is not restrictive, as a general parametrized polynomial map $F_t(x)$ can be transformed into a Newton homotopy with variables $x_1, \dots, x_n, t$ and parameter s , defined by

$$\begin{bmatrix} F_t(x) \\ t \end{bmatrix} + s \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

¹Although the default tracker in `Macaulay2` is not certified, the `NumericalAlgebraicGeometry` package also provides a certified homotopy continuation implementation [10].

A certified homotopy continuation algorithm for Newton homotopies is provided by Hauenstein and Liddell [47].

Interval arithmetic Interval arithmetic-based certified homotopy continuation algorithms were developed in parallel to algorithms relying on α -theory. The first instance is provided by Kearfott and Xing [54]. This work was followed by van der Hoeven [93], which presents how Krawczyk’s operator can be used to certify solution paths, and explains how Taylor models can be used to improve efficiency. Recent work includes the algorithm presented by Duff and Lee [34], which was developed simultaneously and independently from the algorithm of this thesis.

Univariate case ($n = 1$) The approaches in the univariate case are more diverse. Schneps [80] presents a stepsize bound approach, although the bound is not explicit. The bound in Kranich [57], on which Bruin, Sijlsing, and Zotine [22] is based, is explicit, but computing it requires exact root extractions and exact computations over the reals.

Another strategy is to compute X_t , the zero set of F_t (this amounts to solving a univariate polynomial), over a discretization of $[0, 1]$, and connect the fibers X_t and $X_{t'}$ for consecutive parameter values $t < t'$. This leads to compute-and-connect-fibers strategies, as in Deconinck and van Hoeij [30] and Bartholdi, Buff, Graf Von Bothmer, and Kröker [4]. Certification of those methods in the Turing machine model would require additional work.

The method presented by Marco-Buzunariz and Rodríguez [64] relies on interval Newton iteration, with ideas similar to using interval arithmetic in the multivariate case, and is implemented in the software **SIROCCO**, available from **SageMath**.

Finally, Xu, Burr, and Yap [97] present an algorithm relying on a subdivision strategy and the root counting criterion of Becker, Sagraloff, Sharma, and Yap [8]. They also provide an implementation.

1.2.2 Contributions

Certified homotopy continuation based on interval arithmetic has received less attention than approaches relying on Smale’s α -theory. Before the work of this thesis, we lacked a precise algorithm in the Turing machine model, with a proof of correctness and termination.

The first contribution of this thesis, presented in [42, 40], addresses this point: building upon ideas from van der Hoeven [93], namely interval computation of Krawczyk’s operator and the use of Taylor models, we provide an explicit corrector-predictor loop, **Track**, in the Turing machine model.

We carefully define the computational model: we assume an interval arithmetic that depends on a precision parameter $\text{PREC} \in \mathbb{Z}_{>}$ and has good convergence properties as $\text{PREC} \rightarrow +\infty$. These assumptions are based on properties of arbitrary-precision interval arithmetic, as implemented in MPFI [76]. Although the correctness of **Track** is ensured by the basic enclosure property that any interval arithmetic should satisfy, termination requires careful management of PREC . From a practical point of view, increasing PREC reduces computational efficiency; for this reason, we also seek to keep PREC as low as possible while ensuring termination.

Algpath is a Rust implementation of **Track**, based on **Arb** [51]. The code is available at

<https://gitlab.inria.fr/numag/algpath>.

It allows, for the first time, a comparison of the two approaches—interval arithmetic and α -theory—and shows a substantial improvement over the implementation by Beltrán and Leykin [10]. We also compare **Algpath** against **HomotopyContinuation.jl**, a non-certified homotopy continuation implementation. Because one method is certified while the other is not, the comparison of the number of steps can only serve as an estimate of the cost of certification; a fair performance comparison would require a posteriori certification as in Hauenstein, Haywood, and Liddell [46], for instance. Still, the experimental results show that the gap between certified and non-certified homotopy continuation is significantly narrowed.

1.3 Monodromy

Again, consider a parametrized polynomial map $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$, and denote by F_t its specialization at $t \in \mathbb{C}$ in the first variable. Let $\gamma : [0, 1] \rightarrow \mathbb{C}$ be a loop in the parameter space, based at $b = \gamma(0) = \gamma(1)$. Let X_b be the zero set of F_b . If we lift γ to a solution path ζ (which is regular, assuming that γ dodges a certain critical set) starting from $x \in X_b$, the endpoint $\zeta(1)$ lies in X_b . Considering solution paths for each $x \in X_b$ and identifying the endpoints of the associated solution paths with the starting points yields a permutation of X_b , called the *monodromy permutation associated with γ* . This phenomenon is depicted in Figure 1.4.

1.3.1 Previous work

Homotopy-continuation-based approaches Homotopy continuation can be used to compute monodromy permutations: starting from each solution of F_b , we track the associated solution path along γ . Avoiding path jumping is crucial to ensure correctness of the permutation.

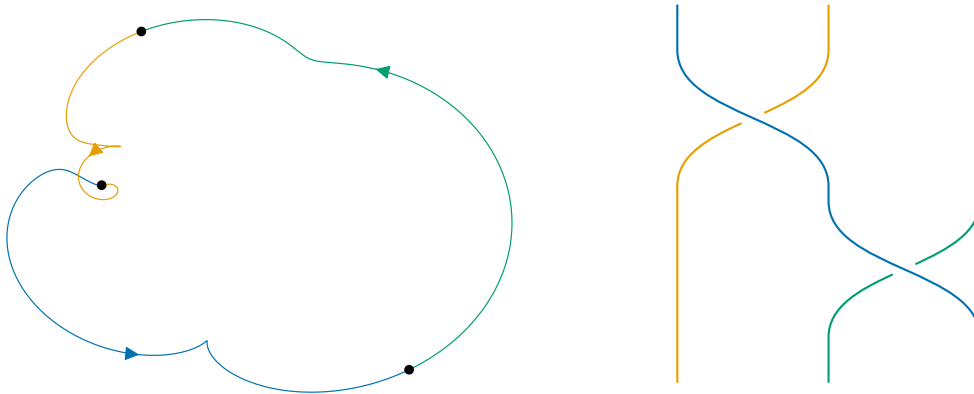


Figure 1.4: On the left, the displacement of the roots with respect to the x variable of $F_t(x) = (6\mathbf{i} - 1)x^3 - \mathbf{i}x^2t + (4\mathbf{i} - 4)xt^2 - t^3 - (\mathbf{i} + 1)x^2 - (\mathbf{i} + 13)xt + 3\mathbf{i}t^2 + (\mathbf{i} - 10)t + \mathbf{i}$, as t moves counterclockwise along the unit circle starting from 1. It induces a cyclic permutation of the roots of F_1 (the black dots), as well as a geometric braid on 3 strands. On the right, the corresponding combinatorial braid $\sigma_1\sigma_2$ is represented by a braid diagram.

Leykin and Sottile [62], Hauenstein, Rodriguez, and Sottile [48], and Duff, Korotynskiy, Pajdla, and Regan [33] tackle monodromy computations in enumerative geometry and practical applications using homotopy continuation. However, their practical results are not rigorous, as they are obtained using non-certified homotopy continuation.

One topic that motivated rigorous monodromy computations is Belyi maps. Given a smooth projective curve X , a *Belyi map* is a non-constant map $\phi : X \rightarrow \mathbb{P}_{\mathbb{C}}^1$, unramified away from $\{0, 1, \infty\}$. Such maps are uniquely determined by their monodromy and can always be defined over $\mathbb{Q}^{\text{al}}$, the algebraic closure of $\mathbb{Q}$. This has led to algorithms that compute equations for X and ϕ with prescribed monodromy. To our knowledge, all known methods for this task require checking that the computed equations have the correct monodromy. To this end, the methods of Schneps [80], Bruin, Sijsling, and Zotine [22], Deconinck and van Hoeij [30], and Bartholdi, Buff, Graf Von Bothmer, and Kröker [4] tackle the problem using homotopy continuation-like approaches. However, as explained in Subsection 1.2.1, implementing these methods to obtain a rigorous result is difficult.

Certified results First, Hauenstein, Haywood, and Liddell [46] provide an example of certified monodromy computation using a posteriori certification.

Next, certified homotopy continuation has been applied to compute line mon-

odromy on symmetric cubic surfaces: a smooth cubic surface in $\mathbb{CP}^3$ always contains 27 lines, and when considering a family of such surfaces, those lines induce a monodromy group. Brazelton and Raman [19] compute this group in the case of S_4 -symmetric cubic surfaces. Their result is rigorous, based on a mix of theoretical arguments and certified homotopy continuation.

More recently, Duff and Lee [35] provide examples of rigorous monodromy computations that comprise enumerative geometry, Belyi maps and applications.

Other rigorous approaches Elkies [37] and Barth and Wenz [3] compute monodromy groups for specific Belyi map instances using a mix of symbolic computations and theoretical arguments, although their approaches do not provide a general algorithm.

In the context of enumerative geometry, Billey and Vakil [16] provide a criterion that can be checked using symbolic tools to prove that the monodromy group of a Schubert problem is the full symmetric group.

Pichon-Pharabod and Telen [74] provide a rigorous method to compute the action of monodromy on the homology of families of smooth hypersurfaces, based on period computations. In particular, this allows to compute the action of monodromy on the lines of a family of smooth cubic surfaces in $\mathbb{P}^3$. The authors in particular compute the monodromy for all families invariant under the action of a subgroup of S_5 , thereby generalizing the result of Brazelton and Raman [19]. They also apply their method to compute the action of monodromy on conics lying on symmetric quartic surfaces.

1.3.2 Contributions

Using `Algpath`, we rigorously compute the monodromy triples of the 1111 Belyi maps in the LMFDB [68]. This work is presented in [43]. The results prove for the first time the correctness of the monodromy groups and, up to minor issues with permuting the branch points $\{0, 1, \infty\}$, of the monodromy triples attached to all entries in the database. We provide code to compute the monodromy of Belyi maps; it wraps `Algpath` and is available at

<https://gitlab.inria.fr/aguillem/belyi-monodromy>.

This demonstrates that recent advances make certified homotopy continuation a practical tool for computing the monodromy of finite algebraic maps.

1.4 Braids

When n points move continuously in the complex plane without colliding, they define a *geometric braid*. Up to homotopy, such braids can be described combinatorially by recording the winding of the points, leading to an algebraic structure first formalized by Artin [2]: the *braid group*, given by the following presentation

$$B_n = \langle \sigma_1, \dots, \sigma_{n-1} \mid \sigma_i \sigma_{i+1} \sigma_i = \sigma_{i+1} \sigma_i \sigma_{i+1}, |i - j| > 1 \Rightarrow \sigma_i \sigma_j = \sigma_j \sigma_i \rangle.$$

More details can be found in Section 4.1.

Consider a parametrized polynomial $f \in \mathbb{C}[t][x]$, and a continuous path $\gamma : [0, 1] \rightarrow \mathbb{C}$. Again, the displacement of t along γ induces a continuous motion of the roots of $f(t, -)$ in $\mathbb{C}$. Provided γ stays away from the discriminant locus of f , the roots do not collide, so they induce a geometric braid with $\deg_x f$ strands (see Figure 1.4). Given f and γ , the goal is to compute the associated combinatorial braid.

Braid computations find applications in the study of the topology of complex varieties: braid monodromy can be used to compute the fundamental group of the complement of a curve in $\mathbb{P}^2$ (Zariski [98], van Kampen [94], and Marco-Buzunariz and Rodríguez [64]) as well as to compute the action of the monodromy on the homology of a fibration (Pichon-Pharabod [73]).

The topological nature of braids leads to a symbolic-numeric approach: first numerically compute the root motion, then recover the combinatorial braid. The first step can be handled by certified homotopy continuation. They output a set of disjoint tubular neighborhoods around the root paths in $\mathbb{C} \times [0, 1]$ (see Figure 1.3), which is enough information to characterize the associated braid. The second step is to recover the braid from the tubular neighborhoods.

1.4.1 Previous work

Marco-Buzunariz and Rodríguez [64] solve the problem of computing braids given f and γ , and provide an implementation. Given a set of tubular neighborhoods obtained through their certified homotopy continuation software **SIROCCO**, they compute piecewise linear paths in each tube from which they can recover the combinatorial braid. The method presented in this thesis avoids this linearization step.

Another approach, by Rodriguez and Wang [78], revolves around solving a certain polynomial system whose solution set contains times at which the roots cross when projected onto the real axis, then determining the sign of the crossings using homotopy continuation. Still, further work is required to make their method certified and to handle geometric braids with non-generic crossings.

1.4.2 Contributions

We focus on the second step: given a set of tubular neighborhoods representing a geometric braid, how to compute the associated combinatorial braid? We answer this question in [41].

To avoid limiting the method to a specific representation of the tubular neighborhoods, whose exact nature differs from one path tracking algorithm to another (piecewise linear, Taylor models, ...), an abstract notion of *approximate data* representing a geometric braid is used. It reduces to what is essential to capture the braid, in the form of a geometric predicate **Sep**, which gives information on the relative positions of the points without retaining their precise locations, and that can be implemented on a specific representation of the tubular neighborhoods. More precisely, assume the point motion is given by n paths $\zeta_1, \dots, \zeta_n : [0, 1] \rightarrow \mathbb{C}$, each associated with the movement of one point. Given $i, j \in [n]$ distinct and $t \in [0, 1)$, $\text{sep}(i, j, t)$ returns a time $t' \in (t, 1]$ and an axis q , either real or imaginary, such that ζ_i and ζ_j do not cross when projected onto q in the time interval $[t, t']$. This data structure adapts well to situations when the point motion is computed approximately: even without access to the exact position of the points, error bounds can be used to determine if two points stay separated along the real or imaginary direction. Given a braid described by its **Sep** predicate, we present an algorithm to compute its associated combinatorial braid.

This method avoids computing piecewise linear approximations of the strands in the tubular neighborhoods. This linearization step is automatic when the path tracking backend outputs piecewise linear tubular neighborhoods [64, 34]; by contrast, linearization requires additional work when the tubular neighborhoods are given by Taylor models, which appear in **Algpath** and in [93]. In the latter case, each tubular neighborhood is given by higher-order pieces whose linearization may require multiple segments, whereas separation of the real or imaginary parts may be checked over the whole piece.

The algorithm is implemented as part of the **Braids** package, with **Algpath** as the certified homotopy continuation backend, and is available at

<https://gitlab.inria.fr/aguillem/braids>.

Chapter 2

Certified homotopy continuation

*This chapter is based on Guillemot and Lairez [42] and Guillemot [40]. We provide a more detailed version of the model of computations in Subsection 2.1.3. Two additional algorithms, **FindMb** and **Equals**, are presented in Subsection 2.2.3 and 2.2.5, respectively. In Section 2.5, we extend certified path tracking to the projective case. Finally, we give additional details on **Algpath** in Section 2.6.*

We begin by describing a certified homotopy continuation algorithm based on interval arithmetic. The method relies on isolating zeros of polynomial systems using Krawczyk’s operator. We introduce the computational model, inspired by adaptive precision interval arithmetic implementations. Next, we review Krawczyk’s method to isolate zeros of polynomial systems, that is made effective using interval arithmetic. This leads to the notion of Moore boxes, a data structure to represent regular zeros of polynomial systems. We describe a predictor-corrector loop **Track**, certified using Moore boxes, and prove correctness and termination in the adaptive precision interval arithmetic model. We explain how predictors and Taylor models can be used to improve computational time, as well as how the presented algorithm can be extended to the projective setting. The last section is dedicated to **Algpath**, an implementation of the presented algorithm.

2.1 Computational model

2.1.1 Interval arithmetic

An *interval arithmetic* over $\mathbb{R}$ consists of a set of real intervals $\square\mathbb{R}$, together with interval arithmetic operations over $\square\mathbb{R}$ extending associated real arithmetic operations.

For our purpose, it will suffice that the interval arithmetic provides binary operations $\boxplus$ and $\boxtimes$ defined over $\square\mathbb{R}$, respectively associated to $+$ and $\times$. These

interval binary operations should satisfy the *fundamental property of interval extensions*: for any $X, Y \in \square\mathbb{R}$ and any $x \in X$ and $y \in Y$, we have

$$x + y \in X \boxplus Y \text{ and } xy \in X \boxtimes Y.$$

We assume effectiveness: we have a data structure to represent elements of $\square\mathbb{R}$, and we have algorithms for $\boxplus$ and $\boxtimes$ in the Turing machine model. Given $X \in \square\mathbb{R}$, we assume that we can (exactly) compute its opposite $-X = \{-x : x \in X\}$. We naturally obtain $\boxminus$, an extension of real subtraction, by the formula $X \boxminus Y = X \boxplus (-Y)$, where $X, Y \in \square\mathbb{R}$.

Let us review some instances of interval arithmetic. We will talk about the differences between these models in Subsection 2.1.3.

Exact interval arithmetic We can choose $\square\mathbb{R}$ to be intervals $[a, b] \subset \mathbb{R}$ with $a, b \in \mathbb{Q}$, and define

$$\begin{aligned} [a, b] \boxplus [c, d] &= [a + c, b + d], \text{ and} \\ [a, b] \boxtimes [c, d] &= [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]. \end{aligned}$$

These are the usual formulas for interval arithmetic, but they are seldom used in this exact form because of the unbearable swell of the binary size of the interval endpoints they produce in any nontrivial computation.

64-bit floating-point interval arithmetic In practice, $\square\mathbb{R}$ is often the set of intervals with IEEE-754 64-bit floating-point numbers endpoints. In this case, the formula above is implemented with appropriate rounding of the interval endpoints to ensure that the fundamental property of interval extensions is satisfied, and is extended to unbounded intervals.

Ball arithmetic Ball arithmetic is a variant of the above, where the intervals are given by a midpoint x and a radius r , denoted by $[x \pm r]$, with corresponding interval $[x - r, x + r] \subseteq \mathbb{R}$. In the exact setting (that is, x and r are rationals), the operations are

$$\begin{aligned} [x \pm r] \boxplus [y \pm s] &= [(x + y) \pm (r + s)], \text{ and} \\ [x \pm r] \boxtimes [y \pm s] &= [(xy) \pm (|x|s + |y|r + rs)] \end{aligned}$$

While in exact interval arithmetic, $X \boxtimes Y$ is precisely the set $\{xy : x \in X, y \in Y\}$ (and it cannot be smaller due to the fundamental property of interval extensions), this is not always true for ball arithmetic. For instance, $[2 \pm 1] \boxtimes [2 \pm 2] = [4 \pm 8]$, while the set-theoretical product is $[6 \pm 6]$.

Arbitrary-precision interval arithmetic Another possibility is for the interval endpoints to be arbitrary-precision floating-point numbers $\{a2^b : a, b \in \mathbb{Z}\}$, also called dyadic numbers. The operations $\boxplus$ and $\boxtimes$ are defined similarly to the fixed-precision case, except that they depend on a precision parameter $\text{PREC} \in \mathbb{Z}_{>}$, in number of bits, and the roundings are done at PREC bits of precision. PREC may be changed in the course of the computation. Such an interval arithmetic is implemented in MPFI [76].

Ball arithmetic also has an adaptive precision counterpart, implemented in Arb [51]. For the latter, only the midpoint of a ball $[x \pm r]$ is maintained at PREC bits of precision, while the radius is a fixed-precision floating-point number with an arbitrary-precision exponent. For this reason, when PREC is large, Arb is expected to be twice as fast as arbitrary-precision interval arithmetic, which needs to maintain two numbers at PREC bits of precision.

Generalizations We define $\square\mathbb{C}$ as pairs of elements of $\square\mathbb{R}$, representing real and imaginary parts, endowed with the obvious extensions of $\boxplus$ and $\boxtimes$ deduced from the formulas of complex addition and multiplication. The elements of $\square\mathbb{C}$ are called *complex boxes*, naturally interpreted as rectangular subsets of $\mathbb{C}$.

We define $\square\mathbb{R}^n$ (resp. $\square\mathbb{C}^n$) as the set of vectors of real intervals (resp. complex boxes). Similarly to complex boxes, we naturally interpret vectors of intervals (resp. boxes) as subsets of $\mathbb{R}^n$ (resp. $\mathbb{C}^n$). Interval matrices are matrices with interval entries, and we extend matrix-vector arithmetic to intervals using the standard formulas.

Remark 2.1. For the sake of simplicity, we fix the definition of $\square\mathbb{C}$, $\square\mathbb{C}^n$, interval matrices and the associated operations. But interval arithmetic over $\mathbb{C}$ may take a different form than complex box arithmetic: for instance, one can choose elements of $\square\mathbb{C}$ to be midpoint radius complex balls for the complex modulus, that are “true” balls, in opposition to rectangular boxes associated to the definition of $\square\mathbb{C}$ we chose above. For details about complex ball arithmetic, see [92].

Moreover, interval operations may be implemented differently; for instance, Arb’s interval matrix multiplication implements a more advanced algorithm than naive multiplication.

2.1.2 Arithmetic circuits and interval extensions

Arithmetic circuits We represent a polynomial function $f : \mathbb{C}^n \rightarrow \mathbb{C}^m$ (the m components of f are complex polynomial functions in n variables) using an *arithmetic circuit*, also known as a *straight-line program*. An arithmetic circuit with input space $\mathbb{C}^n$ is a directed acyclic graph, multiple edges allowed, and an ordered set of m nodes, called *output nodes*. Nodes can be of the following type:

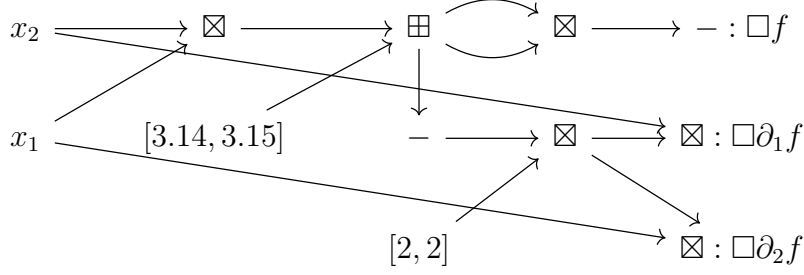


Figure 2.2: Circuit that computes an interval extension of $-(x_1x_2 + \pi)^2$.

2.1.3 Model and hypotheses

Proofs of correctness of the presented algorithms mostly depend on the fundamental property of interval extensions. As for termination, we need additional hypotheses. Exact interval arithmetic enjoys nice theoretical properties: interval operations are continuous (with respect to the Hausdorff metric), monotonous and map singletons to singletons. Although this eases termination proofs, we rule it out because of its impracticability. It is also clear that finitely many representable numbers, as provided by 64-bits floating-point interval arithmetic, will not be enough to express termination arguments based on topology and convergence. For instance, such intervals are not sufficient to isolate two zeros of a polynomial whose distance from one another is smaller than the minimum distance between two 64-bit floating-point numbers.

So we introduce the adaptive precision interval arithmetic model, inspired from arbitrary-precision interval and ball arithmetic implementations.

Ingredients

We assume an abstract interval arithmetic with interval set $\square\mathbb{R}$. The basic operations $\boxplus$ and $\boxtimes$ in this arithmetic depend on a global parameter $\text{PREC} \in \mathbb{Z}_{>}$.

We fix once and for all complex box arithmetic and matrix-vector arithmetic, as explained in Subsection 2.1.1.

To be able to define metrics and avoid technical complications, we will assume that elements of $\square\mathbb{R}$ are non-empty and bounded.

PREC can be thought of as the precision of floating-point computations in number of bits, which can be raised or lowered along computations. The *unit round-off* u_{PREC} is defined as $u_{\text{PREC}} = 2^{-\text{PREC}}$.

The interval data is independent of PREC : whenever PREC is changed along an algorithm, all current interval variables stay unchanged. Precision of the data is

indirectly changed by performing interval operations on it. For instance, assuming $\text{PREC} = 128$, adding two intervals whose endpoints are given with 256 bits of precision should result in an interval with 128-bit-precision endpoints. The latter is motivated by the fact that we do not want precision changes to alter the mathematical objects that some interval data represents. This is the model that **Arb** follows, as well as **MPFI** for which computations are done at the precision of the result operand.

Metrics To formulate assumptions on the interval arithmetic, we need some metrics. Let $\| - \|$ be the real ∞ -norm on $\mathbb{R}^m$, that is

$$\|(x_1, \dots, x_m)\| = \max_{1 \leq i \leq m} |x_i|.$$

Let X, Y be nonempty compact subsets of $\mathbb{R}^m$. The *width* (or *diameter*) of X is defined as

$$\text{width}(X) = \sup_{x, y \in X} \|x - y\|.$$

The *magnitude* of X is defined as

$$\text{mag}(X) = \sup_{x \in X} \|x\|.$$

We endow non-empty compact subsets of $\mathbb{R}^m$ with the Hausdorff distance, denoted by dist . That is,

$$\text{dist}(X, Y) = \max \left\{ \sup_{x \in X} \inf_{y \in Y} \|x - y\|, \sup_{y \in Y} \inf_{x \in X} \|x - y\| \right\}.$$

Lastly, $\text{mid}(X)$ denotes the *midpoint* of X , which is some element of X . We transport those metrics to elements of $\square\mathbb{R}^m$, seen as rectangular subsets of $\mathbb{R}^m$. We endow $\mathbb{C}^n$ and $\square\mathbb{C}^n$ with those metrics, via the identification $\mathbb{C}^n \simeq \mathbb{R}^{2n}$.

We assume that width, mag and mid can be computed effectively over $\square\mathbb{R}^m$.

The attentive reader may have noticed that this assumption does not hold for **MPFI** and **Arb**. We will come back to this when talking about the limitations of the model.

Let X, Y be non-empty compact subsets of $\mathbb{R}^m$ (or $\mathbb{C}^n$), let $x \in X$ and let $\lambda \in \mathbb{R}$. Let $\lambda X = \{\lambda x : x \in X\}$. We will use extensively the following trivial properties of width, mag and dist:

1. $\text{mag}(\lambda X) = |\lambda| \text{mag}(X)$;
2. $\text{width}(\lambda X) = |\lambda| \text{width}(X)$;
3. $\text{mag}(X) \leq \|x\| + \text{width}(X)$;
4. $\text{width}(X) \leq 2 \text{mag}(X)$;
5. $\text{dist}(X, \{0\}) = \text{mag}(X)$.

Hypotheses

We assume that

1. $\square\mathbb{R}$ contains $\mathbb{Z}$, that is, $\{k\} \in \square\mathbb{R}$ for all $k \in \mathbb{Z}$.

Moreover, let X be either a real interval vector or a complex box vector. We assume that:

2. multiplication of X by a power of 2 is an exact operation;
3. if 0_n denotes the $n \times n$ zero matrix, then $0_n \cdot X = 0$.

In fact, we will only need constants 0, 1, 7, 40 and a few powers of 2 to write all the algorithms in this chapter, but it is less obscure to ask 1 and 2, while this is compliant with practical implementations. Similarly, we assume Item 2 as it simplifies algorithm analysis. Notice that 1 and 2 imply that $\square\mathbb{R}$ contains all dyadic numbers.

We assume that all considered circuits have representable complex constants: if $c \in \mathbb{C}$ is a constant in a circuit f , then $\{c\} \in \square\mathbb{C}$.

Given a circuit $f : \mathbb{C}^n \rightarrow \mathbb{C}^m$, the *natural interval extension* of f , denoted $\square f$, is the extension obtained by transforming each constant c into the singleton $\{c\}$.

Remark 2.2. We could consider circuits with more general constants, as long as we can effectively compute an interval around them. For instance, it is convenient to also consider circuits with rational, or even complex algebraic constants. While correctness of upcoming algorithms also holds in this case, we need extra assumptions to ensure termination (see Remark 2.3).

For termination arguments, we rely on convergence properties as $\text{PREC} \rightarrow +\infty$. Arbitrary-precision interval arithmetic, as implemented in **MPFI**, satisfies the following property: for any $M \geq 1$ and any $[a, b], [c, d] \in \square\mathbb{R}$ included in $[-M, M]$,

$$[a, b] \boxplus [c, d] \subseteq [a + c - Mu_{\text{PREC}}, b + d + Mu_{\text{PREC}}], \text{ and}$$

$$[a, b] \boxtimes [c, d] \subseteq [\min(ac, ad, bc, bd) - M^2 u_{\text{PREC}}, \max(ac, ad, bc, bd) + M^2 u_{\text{PREC}}].$$

That is, over a compact K , the roundoff error (compared to the exact setting) is uniformly bounded by a term proportional to u_{PREC} . Based on this observation, we formulate the next assumption. It will serve as the basis of all termination arguments in this chapter.

Assumption 2.1. Let $b \in \{\boxplus, \boxtimes\}$, seen as a function from $\square\mathbb{R}^2$ to $\square\mathbb{R}$. For any compact $K \subseteq \mathbb{R}^2$, there is some $L \geq 0$, independent of the unit roundoff, such that for any $X, Y \in \square\mathbb{R}^2$ included in K :

1. $\text{width}(b(X)) \leq L(\text{width}(X) + u_{\text{PREC}})$;
2. $\text{dist}(b(X), b(Y)) \leq L(\text{dist}(X, Y) + u_{\text{PREC}})$.

Immediate consequences Because Assumption 2.1 is stable under composition of interval binary operations, it also holds for complex box and complex box matrix-vector arithmetic. More generally, it holds for the natural interval extension of a circuit f .

Proposition 2.1. Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^m$ be a circuit and let $\square f$ be its natural interval extension in the adaptive precision model. For any compact $K \subseteq \mathbb{C}^n$, there is some $L \geq 0$, independent of the unit roundoff, such that for any $X, Y \in \square\mathbb{C}^n$ included in K :

1. $\text{width}(\square f(X)) \leq L(\text{width}(X) + u_{\text{PREC}})$;
2. $\text{dist}(\square f(X), \square f(Y)) \leq L(\text{dist}(X, Y) + u_{\text{PREC}})$.

This result is merely an adaptation, to our model of computations, of a classical result about interval extensions; for instance, see Neumaier [69, Theorem 2.1.1].

Proof. We first prove that Proposition 2.1 is stable under composition: let $f : \mathbb{C}^n \rightarrow \mathbb{C}^m$ and $g : \mathbb{C}^m \rightarrow \mathbb{C}^l$ be two circuits, and assume they satisfy conditions 1 and 2. Let K be a compact and L be the constant associated to K , applying Proposition 2.1 to f . First, notice that conditions 1 and 2 imply that there exists a compact K' such that for all $X \subseteq K$, $\square f(X) \subseteq K'$. Let L' be the constant associated to K' , applying Proposition 2.1 to g . Let $X, Y \subseteq K$. As $\square f(X)$ and $\square f(Y)$ are subsets of K' , we get

$$\begin{aligned} \text{width}(\square g(\square f(X))) &\leq L'(\text{width}(\square f(X)) + u_{\text{PREC}}) \leq L''(\text{width}(X) + u_{\text{PREC}}), \text{ and} \\ \text{dist}(\square g(\square f(X)), \square g(\square f(Y))) &\leq L''(\text{dist}(X, Y) + u_{\text{PREC}}) \end{aligned}$$

where $L'' = L'(L + 1)$. Because $X \mapsto \square g(\square f(X))$ is the natural extension of $g \circ f$, 1 and 2 are stable under composition. As Proposition 2.1 holds for $\boxplus$ and $\boxtimes$, it holds in general, by induction on the number of nodes of f . $\square$

Remark 2.3. Without assuming that the constants in all considered circuits are singleton intervals, we cannot initialize the induction of the proof above. But in fact, the proof also holds if the circuit f has more general constants, and the intervals computed around the constants when extending to the intervals are bounded by a quantity proportional to u_{PREC} .

Lemma 2.2. *Fix $X \in \square\mathbb{C}^n$. Let M be a $n \times n$ interval matrix. Then MX , computed using interval arithmetic, tends to 0 as $M \rightarrow 0_n$ and $\text{PREC} \rightarrow +\infty$.*

Proof. Because M tends to 0_n and X is fixed, the pairs (M, X) stay in a compact K . By Proposition 2.1–2, there exists a constant L such that

$$\text{dist}(MX, 0_n X) \leq L \underbrace{(\text{dist}(M, 0_n))}_{\rightarrow 0} + \underbrace{u_{\text{PREC}}}_{\rightarrow 0}$$

where MX is computed using interval arithmetic. By assumption on the model, we have $0_n X = 0$, so $\text{dist}(MX, 0_n X) = \text{dist}(MX, 0)$, which concludes the proof. $\square$

Note that in the proof above, the constant L depends on the circuit chosen for matrix-vector product, circuit which we assume fixed throughout the remainder of this chapter, as stated earlier.

Limitations We assume that width and mag can be effectively computed, as it simplifies algorithm analysis. This assumption may not hold in practice. For instance, when considering MPFI intervals $[a, b] \subseteq \mathbb{R}$, although mag can be computed exactly by the formula $\max\{|a|, |b|\}$, we cannot compute the width $b - a$ at PREC bits of precision in general. For instance, $\text{width}([-2^{-\text{PREC}-1}, 1]) = 1 + 2^{-\text{PREC}-1}$, which cannot be represented with PREC bits of precision. Similarly, when considering Arb balls $[x \pm r]$, we can get the width $2r$ exactly, but not the magnitude $|x| + r$. In practice, given $X \in \square\mathbb{R}$, we compute intervals $\text{mag}_{\square}(X)$ and $\text{width}_{\square}(X)$ enclosing $\text{mag}(X)$ and $\text{width}(X)$ respectively. The proofs of this chapter still hold, as long as the relative approximation error of $\text{mag}_{\square}(X)$ and $\text{width}_{\square}(X)$ is bounded by a quantity proportional to u_{PREC} . That is, there exists a constant C such that for all $X \in \square\mathbb{R}$,

$$\begin{aligned} \text{width}(\text{mag}_{\square}(X)) &\leq \text{mag}(X)Cu_{\text{PREC}}, \text{ and} \\ \text{width}(\text{width}_{\square}(X)) &\leq \text{width}(X)Cu_{\text{PREC}}. \end{aligned}$$

For instance, in Arb we can compute $\text{mag}_{\square}$ of some ball $[x \pm r]$ by computing $|x| + r$ using ball arithmetic. The radius of the result will be an ulp of $|x| + r$ at precision PREC , so is bounded by $2(|x| + r)u_{\text{PREC}}$.

While MPFI satisfies Assumption 2.1, Arb does not. For instance, let $X = [0 \pm 1]$, $Y = [0 \pm 0]$ and for $k \in \mathbb{Z}_{\geq}$, $Y_k = [0 \pm 2^{-k}]$. Then

$$X \boxplus Y = [0 \pm 1], \quad X \boxplus Y_k = [0 \pm (1 + \epsilon)]$$

where ϵ is bounded from below by 2^{-30} . While $\text{dist}((X, Y), (X, Y_k)) = 2^{-k}$, we have $\text{dist}(X \boxplus Y, X \boxplus Y_k) \geq 2^{-30}$, thus Assumption 2.1–2 cannot hold.

Unfortunately, Assumption 2.1–2 is key to the termination of the path tracking algorithm we present. Still, correctness only depends on the fundamental property of interval extensions, which **Arb** satisfies. This means that if an **Arb**-based implementation returns a result, it must be correct. Moreover, by implementing an algorithm written in the adaptive precision model using **Arb**, we rule out stalling that could be avoided simply by increasing **PREC**. Such situations are not handled in the exact model, but appear on numerically challenging inputs. Finally, because Assumption 2.1–2 fails for **Arb** on corner cases, we do not expect stalling on a practical input; at least we never witnessed this behavior with the implementation that will be presented in Section 2.6.

Writing conventions Whenever we describe an algorithm, we assume that all considered values are represented by intervals. For instance, when “ $x \in \mathbb{R}$ ” appears in the input of an algorithm, this means that $\{x\} \in \square\mathbb{R}$ and we use $\{x\}$ (described using the data structure associated to $\square\mathbb{R}$) to represent x . All constants appearing in algorithms can be understood as singleton intervals.

We assume that all computations in algorithms are performed using interval arithmetic and interval extensions, unless stated otherwise. We may omit specific notation for interval operations to improve readability. For instance, if $x + y$ appears in an algorithm, it should be understood as $x \boxplus y$. To avoid confusion, we will leave some hints in the formulas, to recall that they are performed using interval arithmetic. For instance, the reader will encounter “ $A \cdot \square f(x)$ ”, where “ $\square$ ” is purposely left to recall that this computation is performed using interval arithmetic.

Interval arithmetic computations are precisely performed according to the usual operator precedence, as the order of operations is of utmost importance. For instance, if $x \in \square\mathbb{R}$, we do not want to expand $(1-1)x$, as in interval arithmetic $x - x$ is not 0. Another way to think about it is to see such computations as evaluations of arithmetic circuits defined by operator precedence, although in practice we do not build an arithmetic circuit and evaluate it each time we compute something using interval arithmetic.

Some computations in algorithms may be commented with “unchecked arithmetic”. These computations only involve scalar input (represented by singleton intervals), and are not required to be performed using interval arithmetic. This usually happens when we do not require an explicit error bound on the output. Still, the approximation error should be bounded by a quantity proportional to u_{PREC} on a compact, as in Proposition 2.1–1.

In proofs, we explicitly inform the reader if the value under consideration is computed using interval arithmetic, either through specific notation or by stating it explicitly. For instance, we write “consider $\delta = A \cdot \square f(x)$, computed in interval

arithmetic”, as opposed to “consider $\delta = Af(x)$ ”, where the latter should be understood to be the theoretical value.

On a final note, we may compare real values using $<, \leq, >, \geq$. We naturally extend these operators to intervals: for instance, given $X, Y \in \square\mathbb{R}$, $X \leq Y$ if and only if for all $x \in X$ and for all $y \in Y$, $x \leq y$. Note that correctly implementing comparisons for balls (as in **Arb**) is not as easy as for intervals. Most of the comparisons will happen between singleton intervals, so they can be thought of without hesitation as comparisons between real numbers.

Remark 2.4. We assume that all variables are represented using intervals to have a model that depends only on $\square\mathbb{R}$. In a practical implementation it may be sound to introduce additional data structures. For instance, an arbitrary-precision floating point number suffices to represent a scalar value, instead of a singleton interval.

This may also cause complications: if $x, y \in \mathbb{R}$ are representable numbers, we can obtain a tight upper bound on $x + y$ by computing $\text{mag}(x \boxplus y)$. In practice we would simply compute $x + y$ using floating-point arithmetic at precision **PREC**, rounding towards $+\infty$, but such operations are not described by the model.

2.2 Data structure for regular zeros of polynomial systems

Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a polynomial map. We introduce the notion of *Moore box*, a data structure to represent regular zeros of f , whose effectiveness relies upon checking Moore’s criterion using interval arithmetic. We give a few basic routines on our data structure:

1. construction of a Moore box from a candidate zero;
2. refinement of Moore boxes;
3. equality test of the zeros represented by two Moore boxes.

2.2.1 Moore’s criterion

Let V be a finite dimensional linear space over $\mathbb{R}$. For this subsection, we let $\|\cdot\|$ be any norm on V . We denote by $\|\cdot\|_{\text{op}}$ the associated operator norm on $\text{End}(V)$. Let B denote the closed unit ball for $\|\cdot\|$. Let $f : V \rightarrow V$ be a continuously differentiable map.

Theorem 2.3 (Moore’s criterion). *Let $\rho \in (0, 1)$. Let $x \in V$, $r > 0$, and let $A : V \rightarrow V$ be a linear map. Assume that for any $u, v \in rB$,*

$$-Af(x) + [\text{id} - A \circ \text{d}f(x + u)](v) \in \rho rB. \quad (2.1)$$

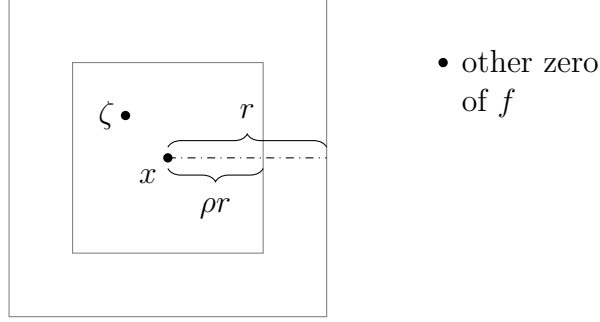


Figure 2.3: Illustration of Moore's criterion, where $V = \mathbb{R}^2$ and $\|\cdot\|$ is the ∞ -norm, for a map $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$. The inner rectangle is the boundary of $x + \rho r B$, the outer rectangle is the boundary of $x + r B$. ζ is the isolated zero of f .

Then there exists $\zeta \in x + \rho r B$ such that $f(\zeta) = 0$ and is the unique zero of f in $x + r B$.

Note that we do not need to assume that A is non-singular; it follows automatically from Equation 2.1.

The operator mapping a compact convex set E containing 0 in its interior to $K(E) = -Af(x) + [\text{id}_V - A \circ \text{d}f(x + E)](E)$, has been introduced by Krawczyk [58] to refine isolating boxes. Moore [65] showed that the inclusion $K(E) \subseteq E$ implies the existence of a zero of f in $x + E$, by a reduction to Brouwer's fixed-point theorem. Then Rump [79, Theorem 7.4] proved uniqueness of the root if $K(E) \subseteq \overset{\circ}{E}$ (which implies $K(E) \subseteq \rho E$ for some $\rho \in (0, 1)$ by compactness of E).¹ In the variant above, we assume that $K(E) \subseteq \rho E$ for some $\rho \in (0, 1)$, and in addition we assume that E is a centered ball with respect to some norm (which is always the case if $E = -E$, in addition to E being compact, convex and a neighborhood of 0). In exchange for this extra assumption we can prove the theorem with Banach's fixed-point theorem applied to the map $y \mapsto y - Af(y)$, which is more elementary than Brouwer's, and work with the operator norm induced by the norm instead of the spectral radius. Because this map will be central in what follows, let us make its definition clear.

Definition 2.1 (Quasi-Newton iteration). Let $A : V \rightarrow V$ be a linear map. The *quasi-Newton iteration associated to A for f* is the map

$$\begin{array}{ccc} V & \rightarrow & V \\ y & \mapsto & y - Af(y) \end{array}$$

¹In fact Rump merely requires the strict inclusion $K(E) \subset E$ but this is not enough to obtain uniqueness, as shown by a simple linear projection $f(x, y) = (x, 0)$. Alefeld and Mayer [1, Theorem 11] provide a correct version with a slightly weaker premise than $K(E) \subseteq \overset{\circ}{E}$ but stronger than $K(E) \subset E$.

We will simply talk about the *quasi-Newton iteration* if A and f are clear from the context. Moore's criterion is an easy consequence of the following proposition, that also provides inequalities that we will use when analyzing algorithms in later development.

Proposition 2.4. *Assume conditions of Moore's criterion. Let g be the quasi-Newton iteration map associated to A for f . For any $y \in x + rB$, we have*

1. $\|dg(y)\|_{\text{op}} \leq \rho$;
2. A and $df(y)$ are invertible;
3. $\|df(y)^{-1}A^{-1}\|_{\text{op}} \leq (1 - \rho)^{-1}$;
4. $\|A\|_{\text{op}} \leq (1 + \rho)\|df(y)^{-1}\|_{\text{op}}$.

Moreover, g has a fixed point ζ in $x + \rho rB$, unique in $x + rB$, and

5. $(1 - \rho)\|y - \zeta\| \leq \|Af(y)\| \leq (1 + \rho)\|y - \zeta\|$.

Proof of Proposition 2.4. Let $y \in x + rB$ and $v \in rB$. Equation (2.1) rewrites as

$$\| -Af(x) + dg(y)(v) \| \leq \rho r. \quad (2.2)$$

As $-v$ belongs to rB , we also have

$$\| -Af(x) + dg(y)(-v) \| = \| Af(x) + dg(y)(v) \| \leq \rho r. \quad (2.3)$$

The triangle inequality, (2.2) and (2.3) then imply

$$2\|dg(y)(v)\| \leq \|dg(y)(v) - Af(x)\| + \|dg(y)(v) + Af(x)\| \leq 2\rho r,$$

which shows that $\|dg(y)\|_{\text{op}} \leq \rho$. Now $\|dg(y)\|_{\text{op}} \leq \rho < 1$ implies the invertibility of the operator $\text{id} - dg(y) = A \circ df(y)$ [60, Thm. 18.2.1] and

$$\|(\text{id} - dg(y))^{-1}\|_{\text{op}} = \|df(y)^{-1}A^{-1}\|_{\text{op}} \leq (1 - \rho)^{-1},$$

which proves Item 3. This implies that both A and $df(y)$ are invertible (Item 2). Inequality 4 is also an easy consequence of inequality 1.

We prove that g has a unique fixed point $\zeta \in x + \rho rB$. First, we prove that $g(x + rB) \subseteq x + \rho rB$. Let $u \in rB$. By integrating the derivative, we compute

$$\begin{aligned} \|g(x + u) - x\| &= \|g(x + u) - g(x) + g(x) - x\| \\ &= \left\| \int_0^1 dg(x + tu)(u) dt - Af(x) \right\| \\ &\leq \int_0^1 \underbrace{\| -Af(x) + dg(x + tu)(u) \|}_{\leq \rho r, \text{ by (2.2)}} dt \leq \rho r. \end{aligned}$$

This shows that $g(x + rB) \subseteq x + \rho rB \subset x + rB$. Now, Item 1 ensures that g is ρ -Lipschitz continuous. As $\rho < 1$, Banach's fixed-point theorem ensures that g has a unique fixed point $\zeta \in x + rB$, and in fact, $\zeta \in g(x + rB) \subseteq x + \rho rB$. Item 5 is an easy consequence of the ρ -Lipschitz continuity of the quasi-Newton iteration. $\square$

Proof of Moore's criterion (Theorem 2.3). Proposition 2.4 ensures that g has a fixed point ζ in $x + \rho rB$, unique in $x + rB$. As $g(\zeta) = \zeta - Af(\zeta) = \zeta$, we have $Af(\zeta) = 0$. Proposition 2.4-2 ensures that A is invertible, thus $f(\zeta) = 0$. $\square$

2.2.2 Moore boxes

We now set the vector space V of the previous subsection to $\mathbb{C}^n \simeq \mathbb{R}^{2n}$, and the norm $\| - \|$ to be the (real) ∞ -norm. Let $B \subseteq \mathbb{C}^n$ be the closed unit ball for $\| - \|$. Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a polynomial map and $\rho \in (0, 1)$.

Definition Let $x \in \mathbb{C}^n$, $r \in \mathbb{R}_{>}$ and $A \in \mathbb{C}^{n \times n}$. We say that (x, r, A) is a ρ -Moore box for f if for all $u, v \in rB$, (2.1) is satisfied, that is

$$-Af(x) + (I_n - A \cdot df(x + u))v \in \rho rB.$$

By Moore's criterion, there exists $\zeta \in \mathbb{C}^n$ a zero of f in $x + \rho rB$, unique in $x + rB$. We call ζ the *zero associated to m* . We say that ρ is the *contraction factor*. A *Moore box for f* is a ρ -Moore box for f , for some $\rho \in (0, 1)$.

We assume that the radius of all Moore boxes considered in an algorithm is a power of 2. With this assumption, we can invert r and multiply by r and r^{-1} exactly. We will keep this property along all algorithms.

Moore box certification using interval arithmetic Let $x \in \mathbb{C}^n$, $r \in \mathbb{R}_{>}$ and $A \in \mathbb{C}^{n \times n}$. If we compute Krawczyk's operator

$$K = -r^{-1}A \cdot \square f(x) + (I_n - A \cdot \square df(x + rB)) \cdot B \quad (2.4)$$

using interval arithmetic, and $\text{mag}(K) \leq \rho$ holds, then (x, r, A) is a ρ -Moore box. Indeed, according to the fundamental property of interval extensions, (2.1) must hold. This leads to Algorithm 1.

Input. $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ a polynomial map; $x \in \mathbb{C}^n$; $r \in \mathbb{R}_>$; $A \in \mathbb{C}^{n \times n}$; $\rho \in (0, 1)$.

Output. A boolean.

```

1 def IsMb( $f, x, r, A, \rho$ ):
2      $K \leftarrow -r^{-1}A \cdot \Box f(x) + (I_n - A \cdot \Box df(x + rB)) \cdot B$ 
3     return  $\text{mag}(K) \leq \rho$ 

```

Algorithm 1: Interval certification of a Moore box.

In later algorithms, we will sometimes introduce a variable m to denote the triple (x, r, A) , and call $\text{IsMb}(f, m, \rho)$ interpreted in the obvious way, to lighten notations.

Lemma 2.5 (Correctness of IsMb). *Let $x \in \mathbb{C}^n$, $A \in \mathbb{C}^{n \times n}$, $r \in \mathbb{R}_>$ and $\rho \in (0, 1)$. If $\text{IsMb}(f, x, r, A, \rho)$ returns **True**, then (x, r, A) is a ρ -Moore box for f .*

Remark 2.5. While $\text{IsMb}(f, x, r, A, \rho)$ returning **true** implies that (x, r, A) is a ρ -Moore box, the converse is not true, because of interval arithmetic overestimation.

Remark 2.6. It is required to compute $-r^{-1}Af(x) + (I_n - A \cdot \Box df(x + rB))B$ in the order suggested by operator precedence (see 2.1.3, paragraph on writing conventions). For instance, we may be tempted to expand $(I_n - A \cdot \Box df(x + rB))B$ to first compute $\Box df(x + rB)B$ then multiply by A on the left, to avoid matrix multiplications; in this case, IsMb will never return true because of the term $I_n \cdot B$.

2.2.3 Moore box initialization

Given $x \in \mathbb{C}^n$ a candidate zero of f , we can try to build a Moore box centered around x for f . To satisfy (2.1), we choose A to be $df^{-1}(x)$, up to roundoff errors and we try out different radiuses. While we could also perform Newton iterations to get x closer to a zero of f before certifying a Moore box, we chose to keep the input x unchanged. This guarantees that the Moore box and the candidate zero represent the same zero, and if the method fails, that the candidate was a bad candidate to begin with.

In practice, **PREC** should be set to the precision of the input x beforehand. The starting radius (by default, set to 1) can be specified if some information on the expected size of the Moore box is known, for instance by an approximate computation of the distance between the zeros of f . Moreover, **FindMb** raises an error after 100 radius halvings, number of iterations that can be increased by specifying a smaller $r_{\min}$.

FindMb does *something* until $\text{IsMb}(f, x, r, A, \frac{7}{8})$ holds and returns (x, r, A) , or raises an error, so it is correct. Still, a single run of **FindMb** may be insufficient to

Input. $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$; $x \in \mathbb{C}^n$.

Output. A $\frac{7}{8}$ -Moore box for f with center x , if no error is raised.

```

1 def FindMb( $f, x$ ;  $r = 1, r_{\min} = 2^{-100}$ ):
2    $A \leftarrow df(x)^{-1}$  //unchecked arithmetic
3   while  $r \geq r_{\min}$ :
4     if IsMb( $f, x, r, A, \frac{7}{8}$ ):
5       return ( $x, r, A$ )
6      $r \leftarrow \frac{r}{2}$ 
7   raise error

```

Algorithm 2: Moore box constructor from a candidate zero.

obtain a Moore box with center x . Nevertheless, it will succeed if x is close enough to a regular zero ζ of f , $r_{\min}$ is taken sufficiently small and $\text{PREC} \rightarrow +\infty$.

Proposition 2.6. *Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a circuit. Consider sequences $x = (x_i) \in (\mathbb{C}^n)^{\mathbb{N}}$ and $r = (r_i) \in \mathbb{R}_{>}^{\mathbb{N}}$. If x converges to a regular zero ζ of f , r converges to 0 and $\text{PREC} \rightarrow +\infty$, then there exists $i \in \mathbb{N}$ such that $\text{FindMb}(f, x_i; r_{\min} = r_i)$ succeeds without raising an error.*

The computation of $A = A_i$ at Line 2 need not be performed in interval arithmetic (this is the practical appeal of Krawczyk’s operator). Proposition 2.6 holds as long as the distance from A_i to $df(x_i)^{-1}$ tends to zero as PREC and i tend to $+\infty$, but the result does not depend on the accuracy of this computation.

Proof. We check that for some $i \in \mathbb{N}$, there exists $s \in [r_i, 1]$ a power of 2 such that $\text{IsMb}(f, x_i, s, A_i, \frac{7}{8})$ returns **true**, where $A_i = df^{-1}(x_i)$ up to roundoff errors. For $s \in (0, 1]$, consider

$$K_{i,s} = -s^{-1} \underbrace{A_i \cdot \square f(x_i)}_{e_i} + \underbrace{(I_n - A_i \cdot \square df(x_i + sB))}_{\Delta_{i,s}} B,$$

computed using interval arithmetic. Consider all values x_i, A_i for $i \in \mathbb{N}$, and $s \in (0, 1]$. These values belong to a compact, as $x_i \rightarrow \zeta$ and as df^{-1} is continuous in a neighborhood of ζ . We can apply Proposition 2.1–1 for $\Delta_{i,s}$: there exists a constant L_1 , independent of i and s , such that $\text{width}(\Delta_{i,s}) \leq L_1(s + u_{\text{PREC}})$. Moreover, $1 - A_i df(x_i) \in \Delta_{i,s}$, so we have

$$\text{dist}(\Delta_{i,s}, 0) = \text{mag}(\Delta_{i,s}) \leq \underbrace{\|1 - A_i df(x_i)\|}_{\xrightarrow{i, \text{PREC} \rightarrow +\infty} 0} + \text{width}(\Delta_{i,s}).$$

Now consider $\Delta_{i,s} \cdot B$. Proposition 2.1–2 ensures that there exists a constant L_2 , independent of i and s , such that

$$\text{mag}(\Delta_{i,s} \cdot B) = \text{dist}(\Delta_{i,s} \cdot B, 0) = \text{dist}(\Delta_{i,s} \cdot B, 0 \cdot B) \leq L_2(\text{dist}(\Delta_{i,s}, 0) + u_{\text{PREC}}).$$

Now restrict the values of s , so that $\text{mag}(e_i) \leq s$. This means that $s^{-1}e_i$ lives in a compact, so we can apply Proposition 2.1–1: there exists a constant L_3 , independent of i and s , such that

$$\begin{aligned} \text{width}(K_{i,s}) &\leq L_3(\max\{\text{width}(-s^{-1}e_i), \text{width}(\Delta_{i,s} \cdot B)\} + u_{\text{PREC}}) \\ &\leq L_3(s^{-1} \underbrace{\text{width}(e_i)}_{\xrightarrow{i, \text{PREC} \rightarrow +\infty} 0} + \underbrace{\text{width}(\Delta_{i,s} \cdot B)}_{\leq 2 \text{mag}(\Delta_{i,s} \cdot B)} + u_{\text{PREC}}). \end{aligned}$$

Finally, as $A_i f(x_i) \in e_i$, we have

$$\text{mag}(K_{i,s}) \leq s^{-1} \underbrace{\|A_i f(x_i)\|}_{\xrightarrow{i \rightarrow +\infty} 0} + \text{width}(K_{i,s}).$$

Putting everything together, there exists a constant L , independent of i and s , such that

$$\text{mag}(K_{i,s}) \leq (1 + s^{-1})\epsilon(i, \text{PREC}) + Ls, \quad (2.5)$$

where ϵ is a function that tends to 0 when i and PREC tend to $+\infty$. Fix s to a small enough power of 2 so that $Ls \leq \frac{1}{2}$. We can find $N \in \mathbb{N}$ such that, for all $i \geq N$, $\text{mag}(e_i) \leq s$ (as $\text{mag}(e_i) \rightarrow 0$) for (2.5) to apply, and $s \geq r_i$, so that **FindMb** always tries s when $i \geq N$. As ϵ tends to 0 when i and PREC tend to $+\infty$, eventually we will have $\text{mag}(K_{i,s}) \leq \frac{7}{8}$, and the test at Line 4 will succeed. $\square$

2.2.4 Refinement of Moore boxes

Following the original idea of Krawczyk [58], who introduced his operator to refine isolating boxes, we can refine a Moore box $m = (x, r, A)$ for a polynomial map $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ by computing the intersection

$$(x + rB) \cap (x - A \cdot \Box f(x) + (I_n - A \cdot \Box df(x + rB))rB).$$

Unfortunately, this cannot always work, because the overestimation due to interval arithmetic may be too severe. It is possible that m is a Moore box, but the intersection above is just $x + rB$, providing no useful refinement. In other words, it is possible that m is a ρ -Moore box but **IsMb**(f, m, ρ) returns **False**.

Algorithm We are given (a circuit representing) a polynomial map $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$, a ρ -Moore box (x, r, A) with an associated zero ζ of f , and some $\tau \in (0, 1)$. We want to compute another Moore box with same associated zero and contraction factor τ .

This is what **Refine** (Algorithm 3) does. The main loop (Line 3) maintains a triple (y, s, U) and stops when this triple is checked to be a τ -Moore box in interval arithmetic, using **IsMb**. After the first iteration, the matrix U is always $\mathrm{d}f(y)^{-1}$, give or take some roundoff errors. As for **FindMb**, it is sufficient to assume that the distance from U to $\mathrm{d}f(y)^{-1}$ goes to zero as **PREC** increases. After the main loop, we know that (y, s, U) is a τ -Moore box. Before returning it, we check if, by chance, $(y, 2s, U)$ is also a τ -Moore box. We double the radius until it is not the case (or the radius exceeds 1) and then return the Moore box.

When Moore's criterion cannot be checked, we try to improve the triple (y, s, U) , either by using a quasi-Newton iteration $y \leftarrow y - Af(y)$, or by shrinking the box with $s \leftarrow \frac{1}{2}s$. The choice depends on $\|Af(y)\|$, the size of the quasi-Newton step, compared to τs , with the goal of balancing the two terms of Krawczyk's operator (2.4).

Because increasing precision costs computational time in practice, we want to avoid unnecessary precision increases. Still, **Refine** may warn that **PREC** is not large enough. Two checks (Lines 8 and 12) may trigger a precision warning. First, after shrinking the box, when $\frac{s}{\tau r}$ drops below some threshold. This is because we expect $r\tau\rho^{-1}$ to be the radius of the τ -Moore box that we are looking for. For an informal analysis, assume that $f(x) = 0$ and $A = \mathrm{d}f^{-1}(x)$. Then (x, r, A) being a ρ -Moore box means that for all $u \in rB$,

$$\| \underbrace{I_n - \mathrm{d}f^{-1}(x)\mathrm{d}f(x+u)}_{=\Delta} \|_{\mathrm{op}} \leq \rho.$$

If we expand $\mathrm{d}f(x+u)$ to order 1, we get

$$\Delta = -\mathrm{d}^2 f(x)(u) + O(\|u\|^2).$$

By linearity of $\mathrm{d}^2 f(x)$, we expect that $\|\Delta\|_{\mathrm{op}} \leq \tau$ by scaling u by $\tau\rho^{-1}$. So when $\frac{s}{\tau r}$ is too small we may suspect that Moore's criterion failed because of roundoff errors. In this case, we want to increase **PREC** in such a way that u_{PREC} goes to 0 faster than s . Second, when performing a quasi-Newton iteration, we compute $\delta = A \cdot \square f(y)$ and $g = y - \delta$ using interval arithmetic. If the roundoff errors $\mathrm{width}(g)$ are comparable to the iteration size $\mathrm{mag}(\delta)$, it means that the iteration is overtaken by roundoff errors, so we increase **PREC**. This ensures the convergence of y (Lemma 2.8).

Constants choice **Refine** features some arbitrary constants for which we picked explicit values. Let us name them: $\rho = \frac{7}{8}$, the contraction factor of the input;

$\alpha = \frac{1}{64}$, used to compare $\text{mag}(\delta)$; $\lambda = \frac{1}{2}$, used to shrink s ; $\beta = 40$, used in the precision check.

Naturally, there is some flexibility in the choice of these constants. To keep **Refine** correct, we need to have $\lambda < 1$ obviously. Values closer to 1 will produce bigger boxes with more iterations. The smaller α , the closer y is to the exact root. The quasi-Newton iteration converges rapidly, so there is little performance penalty in lowering α , but it may cause the precision check to fail earlier. For correctness, we need $\alpha + \rho < 1$. (cf. proofs of Lemma 2.9 and Lemma 2.10). The parameter ρ can be close to 1; it improves performance to allow for Moore boxes with a larger contraction factor. We do not have to worry about the speed of convergence of the quasi-Newton iteration, because interval arithmetic is typically pessimistic. So if we can check a Moore box with contraction factor $\frac{7}{8}$, then it probably has a much lower actual contraction factor. However, other constants degrade if ρ is too close to 1. For correctness, we need $\frac{\beta\rho+1}{\beta-1} < 1$. This number is the geometric ratio in Lemma 2.8.

Remark 2.7. Checking Moore's criterion is costly. From the practical point of view, with the path tracking algorithm in mind, it is beneficial for performance to perform first one or two quasi-Newton iterations (with the appropriate precision check) before entering the main loop.

Remark 2.8. The returned radius s is first set to r , then eventually halved in the first while loop, and eventually doubled in the second while loop, without exceeding 1. Thus, $s \leq \max\{r, 1\}$.

Analysis

Theorem 2.7 (Correctness and termination of **Refine**). *Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a circuit, let (x, r, A) be a $\frac{7}{8}$ -Moore box for f and let $\tau \in (0, 1)$. On input f, x, r, A and τ , **Refine** (Algorithm 3) terminates and outputs a τ -Moore box for f with same associated zero as the input box.*

Proof. **Refine** does *something* until $\text{IsMb}(f, y, s, U, \tau)$ holds. Then a second loop does *something* that preserves this property. So it is obvious, by design, that the algorithm outputs a τ -Moore box. It remains to check that the associated zero of the input and output are the same, and that **Refine** terminates.

We first study the sequence $y_0 = x, y_1, \dots$ where y_k is the value of y after the k th quasi-Newton iteration (Line 10). In principle, the quasi-Newton iteration converges, but roundoff errors could take over. The precision check (Line 9) ensures that it does not happen.

Lemma 2.8. *For any $k \geq 0$, $\|y_k - x\| \leq r$ and $\|y_k - \zeta\| \leq \left(\frac{12}{13}\right)^k r$.*

Input. $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$; (x, r, A) , a $\frac{7}{8}$ -Moore box; $\tau \in (0, 1)$

Output. A τ -Moore box with same associated zero as (x, r, A)

```

1  def Refine( $f, x, r, A, \tau$ ):
2       $y \leftarrow x$ ;    $s \leftarrow r$ ;    $U \leftarrow A$ ;
3      while not IsMb( $f, y, s, U, \tau$ ):
4           $\delta \leftarrow A \cdot \square f(y)$ 
5          if mag( $\delta$ )  $\leq \frac{1}{64}\tau s$ :
6               $s \leftarrow \frac{1}{2}s$ 
7              if  $s < \frac{1}{16}\tau r$ :
8                  PREC  $\leftarrow$  PREC + 2    //increase precision enough so that  $u_{\text{PREC}} = o(s)$ 
9              elif 40 width( $y - \delta$ )  $\leq$  mag( $\delta$ ):
10                  $y \leftarrow \text{mid}(y - \delta)$ 
11             else:
12                 PREC  $\leftarrow$  PREC + 1
13              $U \leftarrow df(y)^{-1}$     //unchecked arithmetic
14
15     while  $2s \leq 1$  and IsMb( $f, y, 2s, U, \tau$ ):
16          $s \leftarrow 2s$ 
17     return  $y, s, U$ 

```

Algorithm 3: Refinement of a Moore box.

Proof. We prove the claim by induction. First, $y_0 = x$, so we have $\|y_0 - x\| = 0$ and $\|y_0 - \zeta\| \leq r$, by Moore's criterion (Theorem 2.3). Let $g : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be the quasi-Newton iteration associated with matrix A and f , that is $g(y) = y - Af(y)$. Assume we are computing y_{k+1} at Line 10 and consider $\delta_k = A \cdot \square f(y_k)$ and $g_k = y_k - \delta_k$, respectively computed (in interval arithmetic) at Line 4 and Line 10 of the same iteration. Then, by definition, $y_{k+1} = \text{mid}(g_k)$. Define $\epsilon_k = y_{k+1} - g(y_k) \in \mathbb{C}^n$. Because y_{k+1} and $g(y_k)$ belong to the interval quasi-Newton iteration g_k , we must have

$$\|\epsilon_k\| = \|y_{k+1} - g(y_k)\| \leq \text{width}(g_k) \quad (2.6)$$

Recall that g is a $\frac{7}{8}$ -Lipschitz continuous function on $x + rB$ such that $g(\zeta) = \zeta$

and $g(x + rB) \subseteq x + \frac{7}{8}B$. As $y_k \in x + rB$ by induction hypothesis, we have

$$\|y_{k+1} - \zeta\| = \|\epsilon_k + g(y_k) - g(\zeta)\| \leq \frac{7}{8}\|y_k - \zeta\| + \|\epsilon_k\|, \quad \text{and} \quad (2.7)$$

$$\|y_{k+1} - x\| = \|\epsilon_k + g(y_k) - x\| \leq \frac{7}{8}r + \|\epsilon_k\|. \quad (2.8)$$

Whenever y_{k+1} is computed, it means that the check at Line 9 must have passed, so

$$40 \text{ width}(g_k) \leq \text{mag}(\delta_k).$$

In combination with (2.6), we get

$$\|\epsilon_k\| \leq \text{width}(g_k) \leq \frac{1}{40} \text{mag}(\delta_k). \quad (2.9)$$

Moreover,

$$\text{mag}(\delta_k) \leq \|Af(y_k)\| + \text{width}(\delta_k) \leq \|Af(y_k)\| + \frac{1}{40} \text{mag}(\delta_k),$$

which implies that $\frac{39}{40} \text{mag}(\delta_k) \leq \|Af(y_k)\|$. By Proposition 2.4–5 it follows

$$\text{mag}(\delta_k) \leq \frac{40}{39} \|Af(y_k)\| \leq \frac{40}{39} (1 + \frac{7}{8}) \|y_k - \zeta\| = \frac{25}{13} \|y_k - \zeta\|. \quad (2.10)$$

In combination with (2.9), we obtain

$$\|\epsilon_k\| \leq \frac{5}{104} \|y_k - \zeta\|. \quad (2.11)$$

It follows from (2.7) and (2.11) that

$$\|y_{k+1} - \zeta\| \leq \frac{7}{8} \|y_k - \zeta\| + \frac{5}{104} \|y_k - \zeta\| = \frac{12}{13} \|y_k - \zeta\| \leq \left(\frac{12}{13}\right)^{k+1} r \quad (2.12)$$

using the induction hypothesis. Moreover, we have $\|y_k - \zeta\| \leq r$ again using the induction hypothesis. From (2.8) and (2.11), we get

$$\|y_{k+1} - x\| \leq \left(\frac{7}{8} + \frac{5}{104}\right) r \leq r,$$

which proves the induction step. $\square$

Lemma 2.9. *At any point of **Refine**, $\|y - \zeta\| \leq s$.*

Proof. The inequality obviously holds at the start of **Refine**. It remains to check that it still holds when y is moved by a quasi-Newton iteration or when s is halved. The norm $\|y - \zeta\|$ decreases when a quasi-Newton iteration is performed, by (2.12). So the inequality is preserved in this case. In the second case, we note that Line 6 is only reached when $\text{mag}(\delta) \leq \frac{1}{64} \tau s$. Since $\|Af(y)\| \leq \text{mag}(\delta)$, this implies, together with Proposition 2.4–5, that

$$\|y - \zeta\| \leq \frac{1}{1 - \frac{7}{8}} \|Af(y)\| \leq 8 \text{mag}(\delta) \leq \frac{1}{8} \tau s < \frac{1}{8} s.$$

So after $s \leftarrow \frac{s}{2}$, we have $\|y - \zeta\| \leq \frac{1}{4} s$, and the inequality holds. $\square$

Since $\zeta \in y + sB$, it is clear that the Moore box returned by **Refine** has associated zero ζ , by uniqueness of the associated root. It only remains to settle termination.

Lemma 2.10. *Considering the values of y , s , U and **PREC** in an infinite run of **Refine**, we have:*

1. $s \rightarrow 0$;
2. $u_{\text{PREC}} = o(s)$;
3. $y \rightarrow \zeta$;
4. $U \rightarrow df(\zeta)^{-1}$;
5. $\|Uf(y)\| \leq \frac{1}{2}\tau s$ eventually.

Proof. First, we show that $s \rightarrow 0$. Assume it does not, implying that after a certain point, s is never halved, so we always have

$$\text{mag}(\delta) > \frac{1}{64}\tau s. \quad (2.13)$$

Since s is not halved, every iteration of the main loop fails the “if” condition, so it reaches the “elif” condition: the precision check. It may fail it (and the working precision is raised) or pass it (and a quasi-Newton iteration is performed). The precision check cannot fail forever. Indeed, every fail causes the working precision to increase. Thus, $40 \text{ width}(y - \delta) \rightarrow 0$, by Proposition 2.1–1. By (2.13), $\text{mag}(\delta)$ is bounded below, so after a certain point we must have $40 \text{ width}(y - \delta) \leq \text{mag}(\delta)$ and the precision check passes. This implies that infinitely many quasi-Newton iterations are performed. By (2.10), this implies that $\text{mag}(\delta) \rightarrow 0$, in contradiction with (2.13). Therefore, $s \rightarrow 0$, proving 1. When s is halved, after a certain point we always reach Line 8 where **PREC** is incremented by 2, meaning that $u_{\text{PREC}} = 2^{-\text{PREC}}$ is divided by 4. This ensures that $u_{\text{PREC}} = o(s)$, thereby checking 2. In particular, $\text{PREC} \rightarrow +\infty$.

The radius s is only halved when $\text{mag}(\delta) \leq \frac{1}{64}\tau s$. Since s is repeatedly halved, this implies that $\text{mag}(\delta) \rightarrow 0$, therefore $Af(y) \rightarrow 0$ and $y \rightarrow \zeta$ by Proposition 2.4–5, proving 3. Since $\text{PREC} \rightarrow +\infty$ and $U = df(y)^{-1}$, up to roundoff errors, we also have $U \rightarrow df(\zeta)^{-1}$, proving 4. So it only remains to check 5. We decompose

$$Uf(y) = \underbrace{(U - df(y)^{-1})}_{=O(u_{\text{PREC}})} \cdot \underbrace{f(y)}_{\rightarrow 0} + \underbrace{(df(y)^{-1}A^{-1})}_{\|-\|_{\text{op}} \leq 8} \cdot \underbrace{Af(y)}_{\|-\| \leq \frac{1}{64}\tau s}, \quad (2.14)$$

using Proposition 2.4–3. This proves that $\|Uf(y)\| \leq \frac{1}{8}\tau s + o(s)$. So eventually $\|Uf(y)\| \leq \frac{1}{2}\tau s$. $\square$

To prove termination of **Refine**, consider an infinite run. By Items 1, 2 and 3 of Lemma 2.10, we have $s \rightarrow 0$, $u_{\text{PREC}} = o(s)$ (in particular $u_{\text{PREC}} \rightarrow 0$) and $y \rightarrow \zeta$. This

implies that $y + sB \rightarrow \{\zeta\}$ in the Hausdorff metric. Let $\Delta = I_n - U \cdot \square df(y + sB)$, computed in interval arithmetic. Proposition 2.1–1, yields $\text{width}(\Delta) \rightarrow 0$. Because Δ contains $I_n - Udf(y)$, which tends to $I_n - df(\zeta)^{-1}df(\zeta) = 0$, we get that $\Delta \rightarrow 0$ in the Hausdorff metric. By Lemma 2.2, $\Delta \cdot B$ also tends to 0. Moreover, let $e = U \cdot \square f(y)$, computed in interval arithmetic. Using Proposition 2.1–1, we have $\text{width}(e) = O(u_{\text{PREC}})$. After multiplication by s^{-1} (which is an exact operation, as s is assumed to be a power of 2), we get $\text{width}(s^{-1}e) \rightarrow 0$ as $u_{\text{PREC}} = o(s)$. Let $K = -s^{-1}e + \Delta \cdot B$ computed using interval arithmetic. Because $-s^{-1}Uf(y) \in K$, we have

$$\text{mag}(K) \leq \| -s^{-1}Uf(y) \| + \text{width}(K).$$

By Assumption 1 on $\boxplus$, we get that $\text{width}(K) \rightarrow 0$, as the widths of e and Δ tend to 0. Moreover, $\|s^{-1}Uf(y)\| \leq \frac{1}{2}\tau$ according to Lemma 2.10–5. So $\text{mag}(K) \leq \frac{1}{2}\tau + o(1)$, which is eventually less than τ . This means that $\text{IsMb}(f, y, s, U, \tau)$ returns true, and the main loop terminates. Due to the condition $s \leq \frac{1}{2}$, the second loop, that tries growing s , also terminates. $\square$

Remark 2.9. For Lemma 2.8 and 2.9 to hold, it suffices to have $y_0 \in x + \frac{1}{8}B$ (rather than $y_0 = x$). Under this assumption, we have

$$\|x - y_0\| \leq \frac{1}{8}r \leq r \text{ and } \|y_0 - \zeta\| \leq \|y_0 - x\| + \|x - \zeta\| \leq \frac{1}{8}r + \frac{7}{8}r = r$$

which ensures the initialization of the induction in the proofs of both lemmas. Thus in this case, **Refine** is also correct and terminates.

2.2.5 Equality test

We conclude this section on Moore boxes with the equality test for the zeros represented by Moore boxes.

Algorithm First, we refine m and m' to a smaller contraction factor $\frac{1}{4}$ (Line 2). This ensures that the associated zeros are different if and only if the isolating regions are disjoint, and we return the truth value of the latter (Lines 4 and 6).

Analysis

Proposition 2.11 (Correctness and termination of **Equals**). *Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a polynomial map. Let m, m' be two $\frac{7}{8}$ -Moore boxes for f , with respective associated zeros ζ and ζ' . Then **Equals**(f, m, m') returns **true** if and only if $\zeta = \zeta'$.*

Equals features arbitrary constants: $\rho = \frac{1}{4}$, chosen to be a power of two to ensure exact scaling, and $c = 3$ a small integer. More generally, Proposition 2.11 holds as long as $\rho < \frac{1}{3}$ and $c \in [\frac{2}{1-\rho}, \frac{1}{\rho})$.

Input. Two $\frac{7}{8}$ -Moore boxes m and m' for $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ a polynomial map.

Output. A boolean.

```

1 def Equals( $f, m, m'$ ):
2   ( $x, r, *$ )  $\leftarrow$  Refine( $f, m, \frac{1}{4}$ );   ( $x', r', *$ )  $\leftarrow$  Refine( $f, m', \frac{1}{4}$ )
3   loop:
4     if  $3 \text{mag}(x - x') - r - r' < 0$ :
5       return true
6     if  $3 \text{mag}(x - x') - r - r' > 0$ :
7       return false
8   PREC  $\leftarrow$  PREC + 1

```

Algorithm 4: Equality test of associated zeros.

Proof. Consider $(x, r, *)$ and $(x', r', *)$ the $\frac{1}{4}$ -Moore boxes computed at Line 2. Correctness of **Refine** ensures that they represent ζ and ζ' , respectively. Possibly after inverting the roles of the $\frac{1}{4}$ -Moore boxes, we can assume that $r \geq r'$. If $\zeta = \zeta'$, Theorem 2.3 ensures that $x + \frac{r}{4}B$ and $x' + \frac{r'}{4}B$ intersect, so

$$3\|x - x'\| \leq 3\left(\frac{r}{4} + \frac{r'}{4}\right) < r + r'.$$

In this case, we cannot have $3 \text{mag}(x - x') - r - r' > 0$ in interval arithmetic, and eventually after enough precision increases, Proposition 2.1–1 ensures that $3 \text{mag}(x - x') - r - r' < 0$. So **Equals** terminates and returns **true**. Conversely, assume that $\zeta \neq \zeta'$. In this case, $\zeta' \notin x + rB$, so

$$\begin{aligned}
3\|x - x'\| &= 3(\|x - x'\| + \|x' - \zeta'\| - \|x' - \zeta'\|) \\
&\geq 3\|x - \zeta'\| - \frac{3r'}{4} \\
&> 3r - r' \geq r + r'.
\end{aligned}$$

For the same reason as previously, precision increases ensure that **Equals** terminates and returns **false**. $\square$

Theoretically, $\zeta = \zeta'$ if and only if $3\|x - x'\| \leq r + r'$. Because we cannot decide if $3\|x - x'\| = r + r'$ using interval arithmetic only, we use a $\frac{1}{4}$ contraction factor at Line 2 to rule out this possibility. If instead we compute $\frac{1}{3}$ -Moore boxes, we need other means to decide if $3\|x - x'\| \leq r + r'$. A solution is to compute $3\|x - x'\| - r - r'$ with correct rounding towards 0 using floating-point numbers with

arbitrary precision exponents, which is enough to deduce its sign. This is more or less the strategy used in the `arb_overlaps` function of `Arb` to decide whether two real balls intersect.

Question 2.1. Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a polynomial map. Let (x, r, A) and (x', r', A') be two Moore boxes for f , with respective represented zeros ζ and ζ' . Is it true that $\zeta = \zeta'$ if and only if $x + rB$ and $x' + r'B$ intersect?

Clearly, the direct implication is true, $\zeta = \zeta'$ being in the intersection. If the reciprocal is true, we could skip refining the input Moore boxes at Line 2 in `Equals`. Other than a proof over $\mathbb{R}$ in dimension 1, the answer to this question remains unknown. We experimentally found intersecting regions around two distinct zeros, each being contracted by a different quasi-Newton iteration. Those regions are not balls, thus they do not give a negative answer to the question. But this also hints that proving the result would require to use fine properties of Krawczyk's operator over balls. On a final note, it seems that the sufficient refinement contraction factor $\frac{1}{3}$ naively deduced from Moore's criterion can be improved to a factor $\frac{1}{2}$.

2.3 Path tracking

2.3.1 Setting

Let $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ be represented by an arithmetic circuit with $n + 1$ input nodes. For $t \in \mathbb{C}$, let $F_t : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be the specialization of F in the first variable at t . Let $(T, X) \in \square\mathbb{C} \times \square\mathbb{C}^n$. We denote by $\square F_T(X)$ the interval evaluation of the extension of F at (T, X) . Automatic differentiation of the circuit F provides a circuit for the map $(t, x) \mapsto dF_t(x) = \partial_x F(t, x)$, whose interval evaluation at (T, X) is denoted $\square dF_T(X)$.

Notations For $T \in \square\mathbb{C}$, F_T is formally the pair (F, T) . It is not itself a circuit, but represents the partial specialization of F at T . The interval map $\square F_T$ defined by $X \mapsto \square F_T(X)$ (resp. the map $\square dF_T$ defined by $X \mapsto \square dF_T(X)$) is an interval extension of F_t (resp. dF_t), for all $t \in T$.

We overload algorithms `IsMb` and `Refine` to accept F_T as their input function. For instance, `IsMb`(F_T, m, ρ) computes Krawczyk's operator (2.4) using $\square F_T$ and $\square dF_T$ in place of interval extensions of the input function and its differential.

Statement of the problem Let $\zeta_0 \in \mathbb{C}^n$ be a regular zero of F_0 , that is $F_0(\zeta_0) = 0$ and assume that the $n \times n$ matrix $dF_0(\zeta_0)$ is invertible. There is a unique open interval $I \subseteq \mathbb{R}$ containing 0 and a unique continuously differentiable function $\zeta : I \rightarrow \mathbb{C}^n$ such that:

1. $\zeta(0) = \zeta_0$;
2. $F_t(\zeta(t)) = 0$ and $dF_t(\zeta(t))$ is invertible, for any $t \in I$.

This follows from the study of the differential equation

$$\frac{d}{dt}\zeta(t) = -dF_t(\zeta(t))^{-1} \cdot \dot{F}_t(\zeta(t)) \quad (2.15)$$

obtained from the equation $F_t(\zeta(t)) = 0$ by differentiation, where $\dot{F}_t(x)$ denotes the derivative of F along the parameter, evaluated at $(t, x) \in \mathbb{C} \times \mathbb{C}^n$. The existence and uniqueness of a local solution is given by the Picard-Lindelöf theorem [60, Theorem 19.1.1]. There is a unique maximal solution interval, namely I .

In what follows, we assume that $\zeta_0 = \zeta(0)$ is given by a $\frac{7}{8}$ -Moore box, and we aim at computing a Moore box for $\zeta(1)$, as a zero of F_1 , assuming that $1 \in I$.

2.3.2 Algorithm

The path tracking operation is performed by **Track** (Algorithm 5), using **IsMb** (Algorithm 1) and **Refine** (Algorithm 3). The main ingredient is to use **IsMb** to check that a triple (x, r, A) is a Moore box for F_t , for a range of values of t .

Proposition 2.12. *Let $\rho \in (0, 1)$, let $m \in \mathbb{C}^n \times \mathbb{R}_> \times \mathbb{C}^{n \times n}$ and let $T \in \square\mathbb{R}$. Assume **IsMb** (F_T, m, ρ) returns **True**. Then m is a ρ -Moore box for F_t , for all $t \in T$.*

Proof. The result directly follows from the fact that for $t \in T$, $\square F_T$ is an interval extension of F_t , $\square dF_T$ is an interval extension of dF_t , and correctness of **IsMb** (Lemma 2.5). $\square$

Given a $t \in [0, 1]$ and a $\frac{7}{8}$ -Moore box for F_t , we refine it into a $\frac{1}{8}$ -Moore box m (Line 4, corrector step), then we search for a step size h such that m is a $\frac{7}{8}$ -Moore box for F_t over $[t, t + h]$ (Line 5, predictor step). This step size is returned by the subroutine **Extend**, which checks whether **IsMb** $(F_{[t, t+h]}, m, \frac{7}{8})$ holds (Line 12). If successful, we know that m is a $\frac{7}{8}$ -Moore box for F_{t+h} and we can repeat the process. Else, h is divided by 2. Note that at Line 9, **Extend** doubles h : although this is not required for termination, it allows the step size to increase so that the algorithm can take longer steps when possible. **Track** falls under the general predictor-corrector—or to be precise, *corrector-predictor*—paradigm for homotopy continuation algorithms.

Remark 2.10. At Line 11, the next time s is computed using the formula $\text{mag}(t+h)$. This is because $t + h$ cannot be computed exactly in general, so is computed using

Input. A circuit $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$; m a $\frac{7}{8}$ -Moore box for F_0 .

Output. A Moore box for F_1 .

```

1  def Track( $F, m$ ):
2       $t \leftarrow 0$ ;    $h \leftarrow 1$ 
3      while  $t < 1$ :
4           $m \leftarrow \text{Refine}(F_t, m, \frac{1}{8})$     //corrector
5           $t, h \leftarrow \text{Extend}(F, m, t, h)$     //predictor
6      return  $m$ 
7
8  def Extend( $F, m, t, h$ ):
9       $h \leftarrow 2h$ 
10     loop:
11          $s \leftarrow \text{mag}(t \boxplus h)$ ;    $T \leftarrow [t, s]$ ;
12         if IsMb( $F_T, m, \frac{7}{8}$ ):
13             return  $s, h$ 
14          $h \leftarrow \frac{1}{2}h$ 
15         if  $u_{\text{PREC}} > h$ :
16             PREC  $\leftarrow$  PREC + 1

```

Algorithm 5: Path tracking.

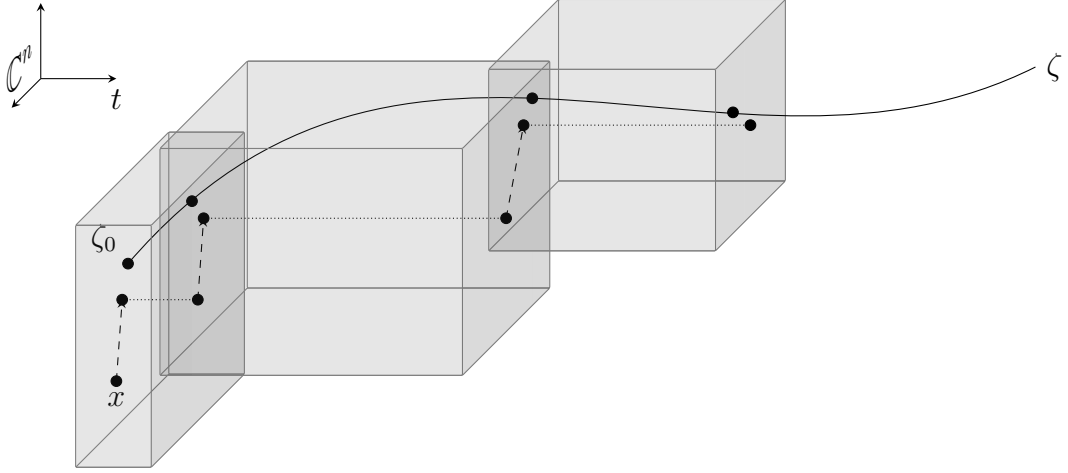


Figure 2.4: Three iterations of **Track** (Algorithm 5). ζ is the solution path. The input Moore box is omitted for readability. Dashed arrows correspond to **Refine** steps (Line 4). Dotted horizontal lines correspond to Moore box certification over an interval (**Extend** subroutine).

interval arithmetic. We then take its mag (instead of its mid) so that $h \leq s - t$, which prevents stalling.

Also at Line 11, we consider the interval $T = [t, s]$. Without further assumptions on the interval arithmetic, we do not know if this interval belongs to $\square\mathbb{R}$. For termination, it is sufficient to compute a tight interval enclosing T , as long as $T \rightarrow t$ in the Hausdorff metric, when $h \rightarrow 0$ and $\text{PREC} \rightarrow +\infty$.

2.3.3 Analysis

The correctness of **Track** is glaring, using Proposition 2.12, but termination is not. Can we find a positive h at each step? Does the process eventually reach $t = 1$? Or may it converge to a lower value of t ?

Theorem 2.13 (Correctness and termination of **Track**). *Let $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a circuit. Let $m = (x, r, A)$ be a $\frac{7}{8}$ -Moore box for F_0 with associated zero ζ_0 . Let $I \subseteq \mathbb{R}$ and $\zeta : I \rightarrow \mathbb{C}^n$ be the solution path defined as in Section 2.3.1. On input F and m , **Track** (Algorithm 5) terminates if and only if $1 \in I$. In this case, it outputs a $\frac{7}{8}$ -Moore box for F_1 with associated zero $\zeta(1)$.*

Proof. We first prove the termination of the loop in **Extend** (Line 10). Assume it does not terminate. Let $K_T \in \square\mathbb{C}^n$ denote the box vector computed in $\text{IsMb}(F_T, m, \frac{7}{8})$ and let K_t denote the one that would be computed in $\text{IsMb}(F_t, m, \frac{1}{8})$.

Because the loop does not terminate, we always have $\text{mag}(K_T) > \frac{7}{8}$. However, the input m of **Extend** comes from **Refine**, with contraction factor $\frac{1}{8}$. This procedure checks $\text{IsMb}(F_t, m, \frac{1}{8})$, so it is guaranteed that $\text{mag}(K_t) \leq \frac{1}{8}$. Again, because the loop does not terminate, we have $h \rightarrow 0$ and $\text{PREC} \rightarrow +\infty$. So s computed at Line 11 tends to t , and so T tends to t in the Hausdorff metric, while m is fixed. In particular, $K_T \rightarrow K_t$, by Proposition 2.1–2, which makes the two inequalities above contradict each other.

We now consider the iterations of the main loop in **Track** (Line 3). Let $m_k = (x_k, r_k, A_k)$, t_k , and T_k be the value of the respective variables at the end of the k th iteration. Moreover, let $T_0 = \{0\} \in \square\mathbb{R}$ and let $m_0 = (x_0, r_0, A_0)$ denote the input Moore box. Let N be the total number of iterations, perhaps infinite. Recall that I is the maximal interval of definition of ζ .

Lemma 2.14. *For any $0 \leq k \leq N$, $T_k \subseteq I$ and for any $t \in T_k$, m_k is a $\frac{7}{8}$ -Moore box for F_t with associated zero $\zeta(t)$.*

Proof. We proceed by induction on k . The base case $k = 0$ is simply the input assumption. Assume the statement holds for $k - 1$. Let $s = t_{k-1}$, that belongs to $T_{k-1} \cap T_k$. By induction hypothesis, m_{k-1} is a $\frac{7}{8}$ -Moore box for F_s with associated zero $\zeta(s)$. By definition,

$$m_k = \text{Refine}(F_s, m_{k-1}, \frac{1}{8}).$$

(We now drop the index k everywhere.) The correctness of **Refine** (Theorem 2.7) implies that m is a $\frac{1}{8}$ -Moore box for F_s with same associated zero. By construction, $\text{IsMb}(F_T, m, \frac{7}{8})$ holds. By Proposition 2.12, this implies that m is a $\frac{7}{8}$ -Moore box for F_t for any $t \in T$.

It remains to prove that $T \subseteq I$ and that for any $t \in T$, the zero of F_t associated to $m = (x, r, A)$ is $\zeta(t)$. Let $J = \{t \in T \cap I : \|\zeta(t) - x\| \leq r\}$. This is, by definition, a closed set in $T \cap I$. It is not empty: $s \in J$. Moreover, for any $t \in J$, $\zeta(t)$ is the zero of F_t associated to m , because there is a unique zero of F_t in the ball $x + rB$. Therefore, by Moore's criterion (Theorem 2.3), we also have $\|\zeta(t) - x\| \leq \frac{7}{8}r$. It follows that J is also open in $T \cap I$. Since $T \cap I$ is an interval, we have $J = T \cap I$.

It only remains to prove that $T \subseteq I$. Recall that $s \in T \cap I$ and let $b = \sup(T \cap I)$. As $b \in T$, there exists $\epsilon > 0$ and a solution $\tilde{\zeta}$ of (2.15) on $S = [b - \epsilon, b + \epsilon]$ starting at ζ_b , the zero of F_b associated to m . As $F_b(\zeta_b) = 0$, we must also have $F_t(\tilde{\zeta}(t)) = 0$ on S . Uniqueness of the zero associated to (x, r, A) ensures that $\tilde{\zeta} = \zeta$ over $[b - \epsilon, b)$. The path defined by ζ until $b - \epsilon$ and $\tilde{\zeta}$ over S is solution of the Cauchy problem (2.15), thus we must have $b + \epsilon \in I$, and so $T \subseteq I$. This concludes the proof of Lemma 2.14. $\square$

Let us resume the proof of Theorem 2.13. Lemma 2.14 proves part of it: if Algorithm 5 terminates, then $1 \in I$ (because $1 \in T_N \subseteq I$) and $\zeta(1)$ is the associated zero of the output. It remains to prove that if $1 \in I$, then **Track** terminates. For contradiction, assume it does not. The value of the variable t converges to some $s \in [0, 1]$. The step size h goes to 0, as it is bounded from above by the difference between two consecutive values of t (see Line 11); thus $\text{PREC} \rightarrow +\infty$ and $T \rightarrow \{s\}$ in the Hausdorff metric. Lastly, Lemma 2.14 and Theorem 2.3 imply that x stays in a bounded set. Indeed, we always have $\|x - \zeta(t)\| \leq r$ for $t \in [0, 1]$, and r is bounded by construction of **Refine** (see Remark 2.8). Since ζ is continuous on $[0, 1]$, it is bounded. Similarly Proposition 2.4–4 implies that A stays in a bounded set, because $t \mapsto \text{d}F_t(\zeta(t))^{-1}$ is continuous for $t \in [0, 1]$.

By construction of **Refine**, we always have

$$\text{mag} \left(-r^{-1}A \cdot \square F_t(x) + \underbrace{(I_n - A \cdot \square \text{d}F_t(x + rB)) \cdot B}_{\Delta} \right) \leq \frac{1}{8}, \quad (2.16)$$

after Line 4 of each iteration.

We first consider the case where r stays away from 0: there is some $r_0 > 0$ such that $r \geq r_0$ at every iteration. Since $h \rightarrow 0$, it is divided infinitely many times, and at least one **IsMb**(...) check fails when this happens. So infinitely often, we have

$$\text{mag} \left(-r^{-1}A \cdot \square F_T(x) + (I_n - A \cdot \square \text{d}F_T(x + rB)) \cdot B \right) > \frac{1}{2}. \quad (2.17)$$

As established above, A and r are bounded, so we may assume that they converge (and r converges to a positive value). Since t and T have the same limit, in the Hausdorff metric, and $\text{PREC} \rightarrow +\infty$, the two left-hand sides in (2.16) and (2.17) also have the same limit (Proposition 2.1–2), which contradicts the inequalities.

So we assume that r does not stay away from zero, and considering a subset of the iterations, we may assume that $r \rightarrow 0$. In this case, $x \rightarrow \zeta(s)$. By construction of **Refine**, we also have $A \rightarrow \text{d}F_s(\zeta(s))^{-1}$. Moreover, when some r is returned by **Refine**, this means that the box could not be grown, that is

$$\text{mag} \left(-\frac{1}{2}r^{-1} \underbrace{A \cdot \square F_t(x)}_e + \underbrace{(I_n - A \cdot \square \text{d}F_t(x + 2rB)) \cdot B}_{\Delta'} \right) > \frac{1}{8}. \quad (2.18)$$

Let $K = -r^{-1}e + \Delta B$, $K' = -r^{-1}e + 2\Delta' B$. From (2.16), we know that $\text{mag}(-r^{-1}e) \leq \frac{1}{8}$, thus $-r^{-1}e$ stays in a compact. According to Proposition 2.1–2 and Lemma 2.2, $\text{dist}(K, K')$ tends to 0, as Δ and Δ' both tend to 0, so

$$\text{mag}(K') = \text{dist}(K', 0) \leq \underbrace{\text{dist}(K', K)}_{\rightarrow 0} + \underbrace{\text{dist}(K, 0)}_{=\text{mag}(K) \leq \frac{1}{8}}.$$

This contradicts (2.18), which implies, after multiplying by 2, that $\text{mag}(K') > \frac{1}{4}$. This concludes the proof of termination. $\square$

Remark 2.11. The final loop in **Refine** simplifies the termination argument above—as (2.16) and (2.17) feature the same triple (x, r, A) —but a more technical analysis shows that this loop is not necessary to ensure termination. However, from the efficiency point of view, it is important to give Moore boxes the opportunity to grow after they have shrunk in the neighborhood of a stiff point. For example, we can insert $s \leftarrow 2s$ at the beginning of **Refine**.

2.4 Faster path tracking with predictors

2.4.1 Rationale

The performance of Algorithm 5 can be greatly improved by incorporating Taylor models. Consider an iteration of the main loop of **Track**. After the **Refine** step, we have t , x , r , and A such that

$$\text{mag}(-r^{-1}A \cdot F_t(x) + (I_n - A \cdot dF_t(x + rB)) \cdot B) \leq \frac{1}{8},$$

and we want a $\delta > 0$ as large as possible such that for all $\eta \in [0, \delta]$,

$$\text{mag}\left(\underbrace{-r^{-1}A \cdot F_{t+\eta}(x)}_V + \underbrace{(I_n - A \cdot dF_{t+\eta}(x + rB)) \cdot B}_\Delta\right) \leq \frac{7}{8}.$$

For an informal analysis, we may consider r and δ as infinitesimally small and compute with first-order expansions. We obtain that

$$\begin{aligned} \text{mag}(\Delta \cdot B) &= O(r + \eta), \text{ and} \\ \text{mag}(V) &= r^{-1}\|v\|\eta + O(r^{-1}\eta^2) = O(r^{-1}\eta), \end{aligned}$$

where $v = -A \cdot \dot{F}_t(x)$ is the *speed vector* related to the variation of the zero that we track as the parameter changes (see Equation 2.15). So the V term is likely to be the main obstruction in raising δ and this suggests that we may expect $\delta \simeq r$.

In the special case where the speed vector v vanishes, we may expect the much better $\delta \simeq \sqrt{r}$. In principle, it is easy to perform the first-order correction and reduce to the stationary case by introducing the auxiliary system $G_\eta(x) = F_{t+\eta}(x + \eta v)$ which is made to satisfy $\dot{G}_0(x) = 0$. If we can rigorously track a zero of G_η as η moves from 0, we can certainly transfer this information to the original system $F_{t+\eta}$. Higher-order corrections are also possible: we can enforce the cancellation of more terms in V , which will increase the step size until the Δ term takes over. However, this idea does not combine nicely with interval arithmetic

which never cancels anything. This phenomenon, well known as the *dependency problem*, rules out all approaches based on cancellations among dominant terms. Taylor models are a classical way to circumvent it.

Example 2.1. Consider the one-equation system $F_t(x) = t - x^2$ and its derivative $dF_t(x) = -2x$ over $\mathbb{R}$. At $t = 1$, we have the $\frac{1}{8}$ -Moore box $(1, \frac{1}{8}, -\frac{1}{2})$, which we can check in exact interval arithmetic:

$$8 \cdot \underbrace{\frac{1}{2} \square F_1(1)}_{=0} + \underbrace{\left[1 + \frac{1}{2} \square dF_1\left(1 + \frac{1}{8}B\right)\right]}_{=\frac{1}{8}B} B = \frac{1}{8}B.$$

We further check that $(1, \frac{1}{8}, -\frac{1}{2})$ is a $\frac{7}{8}$ -Moore box for $F_{1+\eta}$ for all $\eta \in [0, \frac{3}{16}]$. Indeed, we have

$$8 \cdot \underbrace{\frac{1}{2} \square F_{1+\eta}(1)}_{=\eta} + \underbrace{\left[1 + \frac{1}{2} \square dF_{1+\eta}\left(1 + \frac{1}{8}B\right)\right]}_{=\frac{1}{8}B} B = 4\eta + \frac{1}{8}B \subseteq \frac{7}{8}B, \quad (2.19)$$

an inclusion which we check in interval arithmetic by substituting $[0, \frac{3}{16}]$ for η .

We now introduce $G_\eta = F_{1+\eta}(x + \frac{1}{2}\eta)$. Notice that $\frac{1}{2}$ is the speed vector defined earlier. The triple $(1, \frac{1}{8}, -\frac{1}{2})$ is a $\frac{1}{8}$ -Moore box for G_0 (which is just F_1). For $\eta \geq 0$, we have

$$\underbrace{8 \cdot \frac{1}{2} \square G_\eta(1)}_{=-\eta^2} + \underbrace{\left[1 + \frac{1}{2} \square dG_\eta\left(1 + \frac{1}{8}B\right)\right]}_{=-\frac{1}{8}B - \frac{1}{2}\eta} B = \frac{1}{8}B + \frac{1}{2}B\eta - \eta^2 \quad (2.20)$$

from which we deduce easily that $(t, \frac{1}{8}, -\frac{1}{2})$ is a $\frac{7}{8}$ -Moore box for $F_{1+\eta}$ for any $\eta \in [0, \frac{\sqrt{14}-1}{2}]$, that contains for instance $[0, \frac{10}{16}]$ and is a significant improvement over what we obtained for $F_{1+\eta}$. But substituting $\eta = [0, \frac{10}{16}]$ in the left-hand side of the above equation without expanding with respect to η yields $[\frac{-373}{64}, \frac{-33}{16}]$, which does not fit in $\frac{7}{8}B$. In fact, the maximal step size is even smaller than $\frac{3}{16}$ obtained with the naive approach (2.19). This is because interval arithmetic loses track of the fact that each instance of η represents the same value, so the expected cancellations do not occur. To make interval arithmetic work, we first need to expand symbolically with respect to η , as in (2.20) (compare with (2.19), the first-order term is much improved), and then substitute $\eta = [0, \frac{10}{16}]$. $\diamond$

2.4.2 Taylor models

Let $I \in \square\mathbb{R}$ be an interval containing 0. A *Taylor model of order ν on I* is a polynomial $P(\eta) = A_0 + A_1\eta + \dots + A_{\nu+1}\eta^{\nu+1}$ of degree at most $\nu+1$ with coefficients in $\square\mathbb{C}$. A Taylor model P on I encloses a function $f : \mathbb{R} \rightarrow \mathbb{C}$ if for any $t \in I$,

there are $a_0, \dots, a_{\nu+1} \in \mathbb{C}$ such that $a_i \in A_i$ and $f(t) = a_0 + a_1 t + \dots + a_{\nu+1} t^{\nu+1}$. If $J \in \square\mathbb{R}$ is a subinterval of I then the interval computation of $P(J)$ contains $f(J)$ for any function f enclosed by P .

Since the variable η is bound to I , we can squeeze a Taylor model of order ν into one of order $\nu - 1$ by replacing the last two terms $A_\nu \eta^\nu + A_{\nu+1} \eta^{\nu+1}$ by the single term $(A_\nu \boxplus (A_{\nu+1} \boxtimes I)) \eta^\nu$. If P encloses a function f , then the reduced model still does. Order- ν Taylor model addition is defined in the same way as polynomial addition, but using $\boxplus$. We also define multiplication by the usual polynomial multiplication formula, but using $\boxplus$ and $\boxtimes$, which leads to a Taylor model of order $2\nu + 1$. Then, we repeatedly squeeze to reduce it to order ν . These operations are naturally compatible with the enclosure of functions.

Given a circuit $F : \mathbb{C}^n \rightarrow \mathbb{C}^m$, we can naturally evaluate F at a vector of Taylor models, by replacing each operation node by its Taylor model operation counterpart, in the same way as we extend circuits to intervals. The compatibility with enclosure of Taylor model arithmetic ensures that if a function $\mathcal{X} : I \rightarrow \mathbb{C}^n$ is enclosed by the input P (a vector of Taylor models over I , each enclosing the components of $\mathcal{X}$), then the Taylor model obtained by evaluating F at P using Taylor model arithmetic will be a vector of Taylor models enclosing $F \circ \mathcal{X}$.

In general, the first coefficients $A_0, \dots, A_\nu$ of a Taylor model of order ν are narrow intervals enclosing the Taylor expansion of an enclosed function; their widths reflect only roundoff errors. The last term $A_{\nu+1} \eta^{\nu+1}$ reflects the $O(\eta^{\nu+1})$ term of an order- ν Taylor expansion. For more details on Taylor models, we refer to Moore, Kearfott, and Cloud [66], Neumaier [70], Joldes [52], and Berz and Hoffstätter [15].

2.4.3 Algorithm

Checking Moore's criterion along a predictor We consider the same setting as in Section 2.3. Suppose that for some $t \in [0, 1]$, we have a $\frac{1}{8}$ -Moore box (x, r, A) for F_t . Suppose also that we have a vector $\mathcal{X}(\eta)$ of polynomials such that $\mathcal{X}(0) = x$. Naturally, we will choose $\mathcal{X}(\eta)$ to approximate the zero $\zeta(t + \eta)$ of $F_{t+\eta}$ the best we can, but we assume nothing. $\mathcal{X}$ is called the *predictor*. Let $h > 0$ be what we think is a good step size.

Using the arithmetic of order- ν Taylor models on the domain $[0, h]$ (we will typically choose $\nu = 3$), we compute

$$\mathcal{K} = -r^{-1} A \cdot F_{t+\eta}(\mathcal{X}) + [I_n - A \cdot dF_{t+\eta}(\mathcal{X} + rB)] \cdot B, \quad (2.21)$$

which is a vector of Taylor models (we interpret intervals as constant Taylor models). Then we compute $\mathcal{K}([0, h])$ and check if it is included in $\frac{7}{8}B$. If it is, then $(\mathcal{X}(s), r, A)$ is a $\frac{7}{8}$ -Moore box for F_{t+s} for any $s \in [0, h]$. This follows from

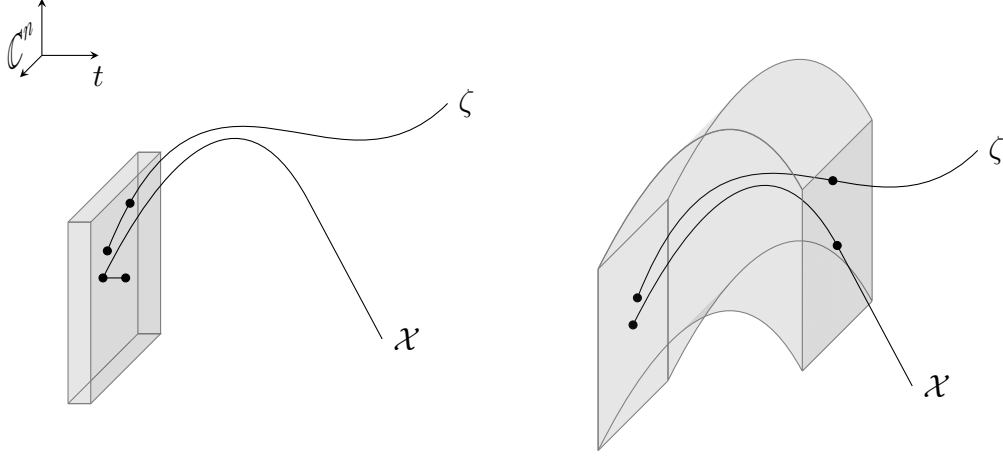


Figure 2.5: Moore box certification on an interval. On the left, the certification does not follow the predictor $\mathcal{X}$, as in Algorithm 5. On the right, certification is performed along $\mathcal{X}$, which allows for a longer step size.

the compatibility of the Taylor model arithmetic with enclosures. We can then proceed to the next path tracking iteration, setting t to $t + h$ and the Moore box to $(\mathcal{X}(h), r, A)$.

If $\mathcal{K}([0, h])$ is not included in $\frac{7}{8}B$, we can try with $\mathcal{K}([0, \frac{h}{2}])$, or maybe $\mathcal{K}([0, \frac{h}{4}])$; we do not need to restart the computation of $\mathcal{K}$ from scratch with a lower step size. In principle, we may assume that $\mathcal{K}(0) \subseteq \frac{1}{8}B$, so there should be some $j \in (0, h]$ such that $\mathcal{K}([0, j]) \subseteq \frac{7}{8}B$, but if we need j to be very small, it makes more sense instead to recompute $\mathcal{K}$ over a smaller domain.

Path tracking with tangent predictor By Equation (2.15), we have

$$\zeta(t + \eta) = \zeta(t) - dF_t(\zeta(t))^{-1} \cdot \dot{F}_t(\zeta(t))\eta + O(\eta^2).$$

Since x approximates $\zeta(t)$ and A approximates $dF_t(\zeta(t))^{-1}$, we may choose our predictor to be $\mathcal{X}_{\text{tangent}} = x - A \cdot \dot{F}_t(x)\eta$. We compute $\dot{F}_t(x)$ by automatic differentiation, similarly to $dF_t(x)$. This is the *tangent predictor* and it leads to Algorithm 6, a revised version of **Track**.

For a given step size h , $t + h$, as well as $\mathcal{X}(h)$ cannot be computed exactly in general. Instead, to advance by h , we first compute a target time $s = \text{mag}(t + h)$ in interval arithmetic (Line 13), which is a (tight) upper bound for $t + h$ (thus for t , preventing stalling). Next, we compute the interval $I = [0, s - t]$ (Line 15) and compute $\mathcal{K}$ using Taylor model arithmetic over I . Again, at Line 15 and Line 20, the theoretical interval I may not belong to $\square\mathbb{R}$, but we are satisfied with a tight interval containing it (see Remark 2.10). Assume $\text{mag}(\mathcal{K}(I)) \leq \frac{7}{8}$.

Input. A circuit $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$; (x, r, A) , a $\frac{7}{8}$ -Moore box for F_0 .

Output. A Moore box for F_1 .

```

1  def Track( $F, x, r, A$ ):
2       $t \leftarrow 0$ ;    $h \leftarrow \frac{1}{2}$ 
3      while  $t < 1$ :
4           $x, r, A \leftarrow \text{Refine}(F_t, x, r, A, \frac{1}{8})$ 
5           $x, t, h \leftarrow \text{Extend}(F, x, r, A, t, h)$ 
6      return  $\text{Refine}(F_1, x, r, A, \frac{7}{8})$ 
7
8  def Extend( $F, x, r, A, t, h$ ):
9       $h \leftarrow 2h$ 
10      $\mathcal{X} \leftarrow x - A \cdot \dot{F}_t(x)\eta$     //Taylor model in the variable  $\eta$ 
11     loop:
12         // 1. Compute target times
13          $s_1, s_2, s_3 \leftarrow \text{mag}(t + h), \text{mag}(t + \frac{h}{2}), \text{mag}(t + \frac{h}{4})$ 
14         // 2. Compute  $\mathcal{K}$  (2.21) using order-2 Taylor model arithmetic over  $I$ 
15          $I \leftarrow [0, s_1 - t]$ 
16          $\mathcal{K} \leftarrow -r^{-1}AF_{t+\eta}(\mathcal{X}) + [I_n - \text{Ad}F_{t+\eta}(\mathcal{X} + rB)]B$ 
17         // 3. Try certification using  $\mathcal{K}$ 
18         for  $s$  in  $s_1, s_2, s_3$ :
19              $s \leftarrow \min\{s, 1\}$ 
20             if  $\text{mag}(\mathcal{K}([0, s - t])) \leq \frac{7}{8}$ :
21                 loop:
22                      $y \leftarrow \square \mathcal{X}(s - t)$ 
23                     if  $\text{width}(y) \leq \frac{1}{16}r$ 
24                         return  $\text{mid}(y), s, h$ 
25                     else:
26                          $\text{PREC} \leftarrow \text{PREC} + 1$ 
27         // 4. If the above fails, decrease  $h$  and go back to step 1
28          $h \leftarrow \frac{h}{2}$ 
29     if  $h < u_{\text{PREC}}$ :
30          $\text{PREC} \leftarrow \text{PREC} + 1$ 

```

Algorithm 6: Path tracking with the tangent predictor.

Because $s - t \in I$, $(\mathcal{X}(s - t), r, A)$ is a $\frac{7}{8}$ -Moore box for F_s . Still, we cannot compute $x = \mathcal{X}(s - t)$ exactly, but as explained in Remark 2.9, we are satisfied with a point y close to x before calling **Refine**, in the sense that y should belong to $x + \frac{1}{8}rB$. For this reason, we compute $\square\mathcal{X}(s - t)$ in interval arithmetic, and check that its width is not bigger than $\frac{1}{16}r$ (loop at Line 21). If the latter fails, we increase **PREC** and repeat. By Proposition 2.1–1, this will eventually succeed. This process is detailed in **Extend** (Algorithm 6).

For correctness, the interval I computed at Line 15 must contain the theoretical values $s_i - t$ for $i \in \{1, 2, 3\}$. While it is reasonable to assume that $s_1 \geq s_2 \geq s_3$, the model of Section 2.1 does not ensure it. For safety measures, one can reorder s_1, s_2 and s_3 so that they are decreasing.

At Line 19, we ensure that s cannot exceed 1. This is because the Moore box returned by **Track** is valid at t returned by **Extend**, and we do not want the last t to be greater than 1. This was not an issue for the previous version of **Track** (Algorithm 5) as the center of the box was unchanged when certifying it over an interval.

Finally, as previously explained, the value x returned by **Extend** does not make (x, r, A) a Moore box, so the result of **Track** is not a Moore box. To remedy this, we insert a last **Refine** call (Line 6) at the end of **Track** so that the returned result is indeed a Moore box.

Theorem 2.15. *Algorithm 6 is correct and terminates, in the sense of Theorem 2.13.*

Mutatis mutandis, the proof is the same as that of Theorem 2.13. The quality of the predictor does not matter much, as long as it stays bounded.

Predictor choice What predictor can we choose? Ideally, we would choose $\mathcal{X}$ to be a truncated Taylor expansion of $\zeta(t + \eta)$ around $\eta = 0$. But only the first term is easy to get, which leads to the tangent predictor. If we record the previous values x' and v' of x and of the tangent vector v , respectively, and the previous step size h' , we can compute the *Hermite predictor*

$$\mathcal{X}_{\text{Hermite}} = x + v\eta + (2h'(v + v') - 3(x - x'))\frac{\eta^2}{h'^2} + (h'(v + v') - 2(x - x'))\frac{\eta^3}{h'^3}.$$

It is the unique polynomial with $\mathcal{X}(0) = x$, $\mathcal{X}'(0) = v$, $\mathcal{X}(-h') = x'$ and $\mathcal{X}'(-h') = v'$. This predictor, with order-3 Taylor models, gave us very good results, on which we report in Section 2.6. Compared to uncertified methods, the complexity of the predictor has a larger impact on the computation time: the predictor is not only used as a guess, we need to validate it. So the balance between the complexity of the predictor and the number of iterations it saves does not favor high-order methods.

By design of Algorithm 6, from one Moore box (x, r, A) , at a given t , to the next point x' at time t' , it is necessary that $(I_n - A \cdot dF_{t'}(x')) \cdot B \subseteq \frac{7}{8}B$. This provides a theoretical maximum step length allowed by Moore's criterion, independent of the quality of the predictor, or overestimation issues in interval arithmetic. In experimentation, we observed that the Hermite predictor brings us very close to this theoretical limit. So we do not expect higher-order predictors to improve performance.

2.5 Certified path tracking in projective space

When tracking a zero of some polynomial map F parametrized by t , it may happen that the solution path ζ diverges to infinity before t reaches 1. Moreover, the practical performance of an implementation may degrade when the solution path is far from 0, even with a predictor. Although the reason is not completely clear to us, one explanation is that the constant L of Proposition 2.1 increases when K gets larger, thus the roundoff errors tend to have a larger impact. To circumvent these issues, we adapt **Track** (Algorithm 5) to the projective setting.

We denote by $\mathbb{P}_{\mathbb{C}}^n = \mathbb{P}^n$ the n -dimensional projective space. If $(x_0, \dots, x_n) \in \mathbb{C}^{n+1} \setminus \{0\}$, we denote by $[x_0 : \dots : x_n]$ its equivalence class in $\mathbb{P}^n$. We let $U_i \subseteq \mathbb{P}^n$ be the (Zariski) open subset defined by $\{[x_0 : \dots : x_n] \in \mathbb{P}^n : x_i \neq 0\}$, that is the complement of $V(x_i)$. We denote by $\varphi_i : U_i \rightarrow \mathbb{C}^n$ the standard chart isomorphism defined by

$$\varphi_i([x_0 : \dots : x_n]) = \left(\frac{x_0}{x_i}, \dots, \frac{x_{i-1}}{x_i}, \frac{x_{i+1}}{x_i}, \dots, \frac{x_n}{x_i} \right).$$

Its inverse maps $y = (y_1, \dots, y_n) \in \mathbb{C}^n$ to the class of y with a 1 added at the i -th position.

We enlarge our interval arithmetic with an interval extension of inversion, defined over intervals that do not contain 0. This way, the map $\varphi_j \circ \varphi_i^{-1} = \varphi_j \varphi_i^{-1}$ can be extended to intervals, for $i, j \in [0, n]$. We make Assumption 2.1 for interval inversion, provided that the compact $K \subseteq \mathbb{R}$ does not contain 0. Because inversion of an interval X is not defined if $0 \in X$, we encapsulate computations that involve inversions in a “try” block.

To adapt **Track** to the projective setting, we rely on transferring a Moore box m from chart i to chart j . The main difficulty lies in how to use m to deduce a Moore box in chart j : surely, we can change of chart the center of m , but what should be the radius of the new box? And how can we ensure that the new box represents the correct zero?

2.5.1 Setting

Consider a map $F : \mathbb{C} \times \mathbb{C}^{n+1} \rightarrow \mathbb{C}^n$, given by n polynomials in $t, x_0, \dots, x_n$ that are homogeneous in the x variables. For instance, F can be obtained after homogenizing a parametrized polynomial map $G_t(x)$ with respect to the x variables. Let $\zeta_0 \in \mathbb{P}^n$ be a regular zero of F_0 . We assume there exists a solution path $\zeta : [0, 1] \rightarrow \mathbb{P}^n$ such that $\zeta(t)$ is a regular zero of F_t , for all $t \in [0, 1]$. The goal is to track ζ .

2.5.2 Certified change of chart

For $i \in [0, n]$, let $F^i : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ be the map obtained from F by evaluating the x_i variable to 1. Up to renaming variables, we can assume that $\zeta_0 \in U_0$. In this case, $\varphi_0(\zeta_0)$ is a zero of F_0^0 , and we can start path tracking using F^0 : we track ζ in the 0-th chart. If ζ diverges in the 0-th chart (that is, ζ exits U_0), we need to transfer the working Moore box to a more appropriate chart i and continue tracking with F^i . Change of chart is handled by **ToChart** (Algorithm 7).

Setting Let f be a circuit with $n + 1$ input nodes and n components. We call this circuit *homogeneous* if its components are homogeneous polynomial functions. Let $i \in [0, n]$ be a chart index. Formally, f^i is the pair (f, i) , which we can evaluate at $x \in \mathbb{C}^n$ by inserting a 1 at the i -th position of x , and evaluating f at this enhanced point. f^i represents the dehomogenization of f with respect to the i -th variable. We extend f^i and its differential to intervals in the obvious way.

Algorithm Let $m = (x, r, A)$ be a Moore box for f^i . To pass m from chart i to chart j , we first change x to chart j (Line 6). We then try a variety of radiuses using **FindMb**, with a lower bound on s to ensure correctness. Those operations may fail, as changing charts may involve a division by an interval containing 0, or **FindMb** may fail after reaching $r_{\min}$. In this case, we refine (x, r, A) to a smaller contraction factor and repeat. Termination is ensured as y converges to $\varphi_j(\zeta)$, while for each new y we try a range of radiuses to which we add progressively smaller radiuses during the run. An early version of **Refine** was based on this idea.

Analysis

Theorem 2.16 (Correctness and termination of **ToChart**). *On valid input f, x, r, A and chart indices i, j , **ToChart** (Algorithm 7) terminates and is correct.*

Proof. Assume **ToChart** terminates. The center of the returned Moore box is y computed at Line 6, and the radius s is bigger than $r_{\min}$ computed at Line 7.

Input. A homogeneous circuit $f : \mathbb{C}^{n+1} \rightarrow \mathbb{C}^n$; a Moore box (x, r, A) for f^i with associated zero $\zeta \in U_i \cap U_j \subset \mathbb{P}^n$.

Output. A Moore box for f^j with associated zero ζ .

```

1  def ToChart( $f, x, r, A, i, j$ ):
2       $\tau \leftarrow \frac{1}{8}$ ;
3      loop:
4           $x, r, A \leftarrow \text{Refine}(f^i, x, r, A, \tau)$ 
5          try:
6               $y \leftarrow \text{mid}(\square \varphi_j \varphi_i^{-1}(x))$ 
7               $r_{\min} \leftarrow \text{mag}(\square \varphi_j \varphi_i^{-1}(x + \tau r B) - y)$ 
8              return FindMb( $f^j, y; r_{\min} = r_{\min}$ )
9          except:
10              $\tau \leftarrow \frac{1}{2} \tau$ 
11              $\text{PREC} \leftarrow \text{PREC} + 1$ 

```

Algorithm 7: Moore box chart switch.

Let $X = \square \varphi_j \varphi_i^{-1}(x + \tau r B) - y$, computed using interval arithmetic at Line 7. As $\varphi_i(\zeta) \in x + \tau r B$, we have $\varphi_j(\zeta) - y \in X$, so

$$\|\varphi_j(\zeta) - y\| \leq \text{mag}(X) = r_{\min} \leq s.$$

This rewrites as $\varphi_j(\zeta) \in y + sB$, and the uniqueness part of Moore's criterion (Theorem 2.3) yields the result.

To settle termination, assume an infinite run, by contradiction. In this case, **ToChart** always fails the “try” block and reaches the “except” block. Thus $\tau \rightarrow 0$ and $\text{PREC} \rightarrow +\infty$. By correctness of **Refine**, we have $\varphi_i(\zeta) \in x + \tau r B$ and r is bounded by construction, so $x \rightarrow \varphi_i(\zeta)$. Because $\zeta \in U_j$, eventually we have $\varphi_i^{-1}(x + \tau r B) \subseteq U_j$ after enough iterations, as $x + \tau r B \rightarrow \{\varphi_i(\zeta)\}$ in the Hausdorff metric. So Line 6 and 7 cannot fail forever, and y tends to $\varphi_j(\zeta)$ while $r_{\min}$ tends to 0. Proposition 2.6 ensures that **FindMb** will eventually succeed. $\square$

2.5.3 Projective path tracking

Algorithm Using **ToChart** (Algorithm 7), we can naturally extend **Track** to the projective setting. At each iteration of **Track**, we compute the norm of the center of the box x ; if it becomes large, we change to a more appropriate chart. The

Input. A homogeneous circuit $F : \mathbb{C} \times \mathbb{C}^{n+1} \rightarrow \mathbb{C}^n$; (x, r, A) , a $\frac{7}{8}$ -Moore box for F_0 in chart 0.

Output. A chart index $i \in [0, n]$ and a Moore box for F_1 in chart i .

```

1 def Track( $F, x, r, A$ ):
2    $t \leftarrow 0$ ;    $h \leftarrow \frac{1}{2}$ ;
3    $i \leftarrow 0$    //chart index
4   while  $t < 1$ :
5     if  $\text{mag}(x) \geq 4$ :
6        $j \leftarrow$  index of the component of  $\varphi_i^{-1}(x)$  of largest modulus
7        $x, r, A \leftarrow \text{ToChart}(F_t, x, r, A, i, j)$ 
8        $i \leftarrow j$ 
9       do a predictor-corrector step in chart  $i$ , using  $F^i$ 
10  return  $i, (x, r, A)$ 

```

Algorithm 8: Path tracking in projective space.

chosen threshold on the norm of x at Line 5 ensures that the algorithm does not frantically change charts, thereby ensuring termination.

Analysis

Theorem 2.17. *On a valid input, Algorithm 8 is correct and terminates.*

Proof. Correctness is deduced from the correctness of **Track** (Theorem 2.13) and **ToChart** (Theorem 2.16).

For termination, assume an infinite run of Algorithm 8, that stalls at $s \in (0, 1)$. If **Track** never changes chart, then $\text{mag}(x) < 4$, so it must terminate as x is bounded (so is ζ in the 0-th chart), using the same argument as Theorem 2.13.

Assume it changes chart at least once, so that $r \leq 1$ by construction of **ToChart** and **Refine**. It suffices to show that **Track** stops changing chart after a certain point; in this case, we can apply the same argument as in Theorem 2.13 to find a contradiction. To this end, we show that if we are close to s and we change charts, then ζ will be close to 0 and by continuity at s it will stay close to 0 until s . In this case, the center of the box x also stays close to 0, thanks to the fact that $r \leq 1$, and thus its magnitude cannot exceed 4.

So let J be the set of non-zero coordinates of $\zeta(s)$. That is, $\zeta(s) \in U_i$ if and only if $i \in J$. The chart index i after the “if” block at Line 5 must always belong to J after a certain point, as x and r are bounded. Assume we switch from chart $i \in J$

to chart $j \in J$ at time t . Consider the value x before changing chart (Line 7) and write

$$z = \zeta(t) = [z_0 : \dots : z_n] \text{ and } \varphi_i^{-1}(x) = [x_0 : \dots : x_n],$$

normalized so that $z_i = x_i = 1$. We change from chart i to chart j , so $\|x\| = \|x_j\|$, as x_j is the component of largest modulus of $\varphi_i^{-1}(x)$. Because (x, r, A) is a $\frac{7}{8}$ -Moore box for F_t^i with associated zero $\varphi(t)$, we have $\varphi_i(z) \in x + \frac{7}{8}rB$, thus

$$\|x - \varphi_i(z)\| \leq \frac{7}{8}r \leq \frac{7}{8}. \quad (2.22)$$

Equation (2.22) implies that

$$\|x_j\| \leq \|z_j\| + \|x_j - z_j\| \leq \|z_j\| + \|x - \varphi_i(z)\| \leq \|z_j\| + \frac{7}{8}, \quad (2.23)$$

so $\|z_j\| \geq \|x_j\| - \frac{7}{8}$. Moreover, for all $k \neq i$, we have

$$\|z_k\| \leq \|\varphi_i(z)\| \leq \|x\| + \frac{7}{8} = \|x_j\| + \frac{7}{8}. \quad (2.24)$$

The inequality also holds for $k = i$, as $z_i = 1$ and $\|x\| \geq 4$. Recall that

$$\varphi_j(z) = \left(\frac{z_0}{z_j}, \dots, \frac{z_n}{z_j} \right).$$

Now, the “if” condition at Line 5 passes, so $\|x_j\| = \|x\| = \text{mag}(x) \geq 4$. Using (2.23) and (2.24), we obtain

$$\|\varphi_j(z)\| \leq \frac{\|x_j\| + \frac{7}{8}}{\|x_j\| - \frac{7}{8}} \leq \frac{4 + \frac{7}{8}}{4 - \frac{7}{8}} = \frac{39}{25}. \quad (2.25)$$

Now, by continuity at s of ζ , there exists $\delta > 0$ such that for all $t, t' \in (s - \delta, s)$ and for all $k \in J$, $\zeta(t)$ and $\zeta(t')$ belong to U_k , and we have

$$\|\varphi_k(\zeta(t)) - \varphi_k(\zeta(t'))\| < 1. \quad (2.26)$$

Assume we switch from chart $i \in J$ to chart $j \in J$ at $e \in (s - \delta, s)$. Let $z = \zeta(e)$, and consider values of x and t afterwards. Using (2.25) and (2.26), we obtain

$$\|x\| \leq \underbrace{\|x - \varphi_j(\zeta(t))\|}_{\leq \frac{7}{8}} + \underbrace{\|\varphi_j(\zeta(t)) - \varphi_j(z)\|}_{< 1} + \underbrace{\|\varphi_j(z)\|}_{\leq \frac{39}{25} < 2} < 4.$$

After this point, we can never reach the “if” at Line 5, thus we stay in chart j . This concludes the proof of termination. $\square$

2.6 Alpath

In this last section, we give some details about **Alpath**, a **Rust** implementation of the certified path-tracking algorithms presented above. Precisely, it implements **Track** with Hermite’s predictor from Section 2.4.

2.6.1 Interval arithmetic

Alpath uses **Arb** for interval computations. **PREC** is given as the precision argument to **Arb** operations. We choose **Arb** over **MPFI** as **SageMath** experiments, as well as Johansson [51], show that evaluating polynomials using **Arb** is roughly twice as fast compared to evaluating them using **MPFI**, even when **PREC** is low.

The software also relies on a implementation of 64-bit floating-point interval arithmetic using Single Instruction Multiple Data (SIMD) instructions, which we call *SIMD interval arithmetic*; its interval endpoints are given by two 64-bit floating-point numbers, packed in a 128-bit register. This arithmetic is implemented within **Alpath** using the AVX instruction set for the x86-64 platform, following Lambov [59], and is extended to complex intervals using 256-bits registers as well as some instructions from the AVX2 set. Note that as of now, **Alpath** only supports x86-64 CPUs with the AVX2 instruction set.

For example, the multiplication of two order-3 Taylor models with complex interval coefficients, that is 100 real interval number multiplications and 40 additions, is performed with 638 SIMD instructions in less than 300 CPU cycles, according to the analysis tool *llvm-mca*. At double precision, using SIMD interval arithmetic to track the roots of a given system is roughly 30–40 times faster than solving the same problem using **Arb**.

2.6.2 Precision decreases

In **Refine** and **Track** (Algorithms 3 and 5), although we increase **PREC** to ensure termination, we never decrease it afterwards. While this is sufficient for termination, incorporating precision decreases is beneficial for practical performance. Indeed, if we track a zero of F and a high **PREC** is required midway through tracking, the algorithm will suffer from the computational cost of high precision for what remains to be tracked, although a lower **PREC** may be sufficient. Because interval data remains unchanged when **PREC** changes (see Subsection 2.1.3), Moore boxes retain their mathematical properties when **PREC** is lowered. For this reason, we decrease precision by 1 at the beginning of each iteration of **Track**; this does not hinder correctness or termination.

2.6.3 Mixed precision

Motivation It is possible to run **Algpath** using SIMD interval arithmetic only. In this mode, all increases of **PREC** present in **Refine** and **Track** instead raise an error, meaning that the implementation may abort computations before terminating the path tracking. It is also possible to run **Algpath** using **Arb** only; this mode supports increases of **PREC** but is much slower than SIMD interval arithmetic on problems that do not require more than double precision.

To combine the advantages of both modes, **Algpath** also implements *mixed precision*: it uses SIMD interval arithmetic when $\text{PREC} \leq 53$ and switches to **Arb** otherwise.

Mixed-precision interval arithmetic Continuing the model of computation presented in Section 2.1, a natural approach is to define elements of $\square\mathbb{R}$ as either SIMD intervals or **Arb** balls and to define $\boxplus$ and $\boxtimes$ on this union of types. Each interval operation determines its output type from **PREC** while accepting SIMD intervals or **Arb** balls independently of **PREC**. For instance, adding two SIMD intervals at $\text{PREC} = 128$ should yield an **Arb** ball.

But if, for instance, we want to add two SIMD intervals and $\text{PREC} \leq 53$, the full operation will cost a SIMD interval addition plus checking that $\text{PREC} \leq 53$. The cost of this check is not negligible compared to the cost of the addition, which consists of a single SIMD instruction. This overhead spreads to all the code, as all interval operations depend on the basic interval arithmetic: experiments show that adding precision checks to complex box addition and multiplication makes **Algpath** twice as slow at double precision.

For larger data types, we can avoid this issue by requiring homogeneous representations that do not mix SIMD interval and **Arb** ball coefficients. For instance, define a mixed interval matrix to be either a matrix with SIMD interval entries or a matrix with **Arb** ball entries (rather than being a matrix with entries that are either SIMD intervals or **Arb** balls). If we multiply two mixed interval matrices with SIMD interval entries and $\text{PREC} \leq 53$, the check that $\text{PREC} \leq 53$ is performed only once, and its overhead is negligible compared with the cost of SIMD interval matrix multiplication when the matrices are large.

Using conversions Implementing mixed-precision interval arithmetic as described above is complicated due to the multiplication of union types. Instead, we chose a more pragmatic solution that relies on conversions between the two arithmetic implementations.

Note that we cannot assume exact conversions between **Arb** and SIMD intervals, even *from* SIMD intervals. For example, consider $[-2^{-50}, 2] \subseteq \mathbb{R}$, representable as a SIMD interval. This corresponds to the ball $[(1 - 2^{-51}) \pm (1 + 2^{-51})]$, but $1 + 2^{-51}$

cannot be represented by a `mag.t` (radius type for `Arb` balls) since this type has a fixed mantissa of 30 bits. However, the Moore box m and the time t in `Track` are represented by singleton intervals, for which conversions behave as expected: a SIMD singleton interval can always be converted to an `Arb` ball, and the converse holds whenever the center of the ball is given with 53 bits of precision.

Rewrite `Track` (Algorithm 5) as follows:

```

1 def Track( $F, m$ ):    //  $F$  parametrized polynomial map,  $m$  Moore box for  $F_0$ 
2      $t \leftarrow 0$ 
3     while  $t < 1$ :
4          $m, t \leftarrow \text{OneStep}(F, m, t)$     // corrector-predictor step
5     return  $m$ 

```

For SIMD interval arithmetic and `Arb`, `OneStep` is just a corrector-predictor step:

```

1 def OneStep( $F, m, t$ ):
2      $m \leftarrow \text{Refine}(F_t, m, \frac{7}{8})$ 
3      $m, t' \leftarrow \text{Extend}(F, m, t)$     // Moore box certification on  $[t, t']$ 
4     return  $m, t'$ 

```

In the SIMD interval arithmetic version of `OneStep`, precision increases in `Refine` and `Extend` instead raise an error.

```

1 def OneStep( $F, m, t$ ):    //  $m, t$  represented using SIMD intervals or Arb balls
2     try:
3         Convert  $m, t$  to SIMD intervals
4         return OneStep( $F, m, t$ )    // SIMD interval arithmetic OneStep
5     except:
6         Convert  $m, t$  to Arb balls
7         return OneStep( $F, m, t$ )    // Arb OneStep

```

Algorithm 9: Mixed precision `OneStep`.

Let $t \in [0, 1)$ and let m be a Moore box for F_t . We provide an additional mixed-precision implementation of `OneStep` (Algorithm 9). The inputs m and t are represented using either SIMD intervals or `Arb` balls. First, we attempt a corrector-predictor step using SIMD interval arithmetic. We convert m, t to SIMD intervals (Line 3); this conversion will most likely fail if `PREC` > 53. If m, t are

name	deg.	Mixed-precision	SIMD intervals
		time (s)	time (s)
dense	10	7 ms	6 ms
dense	50	132 ms	114 ms
dense	100	522 ms	446 ms
katsura	2^{10}	21 s	25 s
katsura	2^{15}	43 min	54 min
dense	5^2	51 ms	47 ms
dense	10^2	577 ms	521 ms
dense	50^2	613 s	587 s
structured *	3^4	332 ms	318 ms
structured *	3^8	1828 ms	1928 ms
structured ^N	20^{20}	1063 ms	1192 ms
structured ^N	30^{30}	3377 ms	3945 ms
structured ^N	3^4	9 ms	8 ms
structured ^N	3^8	25 ms	23 ms

Table 2.1: Comparison of **Algpath** (mixed precision) with **Algpath** using SIMD interval arithmetic only (the `--arithmetic reckless` command-line option). *deg.*: total degree of the target system; *time*: total time to track all the paths. * A hundred randomly picked starting zeros of a total degree homotopy. ^N Newton homotopy (a single path to track). For a precise description of the data set, see Subsection 2.6.5.

already represented using SIMD intervals, this line trivially succeeds. Then we run **OneStep** using SIMD interval arithmetic (Line 4), which will fail if a precision increase is required. If either operation fails, we perform a corrector-predictor step using **Arb**. Converting m, t to **Arb** balls (Line 6) and running **OneStep** with **Arb** (Line 7) will not raise precision-related errors.

This strategy leads to no significant overhead for the overall computation, as shown by Table 2.1. Moreover, the above pseudocode can easily be implemented.

2.6.4 Usage

The source code of **Algpath** is distributed under the GPLv3 license and available at

<https://gitlab.inria.fr/numag/algpath>.

After compilation, it provides a command-line interface of the same name. The input is a JSON file describing a parametrized polynomial map $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$

```

{
  "system": [
    "(1 - x^2)*(1 - t) + t*(x^2 + 1/2*x*y - 3*y^2 + 1/4*y)",
    "(1 - y^2)*(1 - t) + t*(15*x^2 - 1/6*y^2 + x + 39*y + 1)"
  ],
  "variables": ["x", "y"],
  "parameters": ["t"],
  "path": [["0"], ["0.5 + I*0.5"], ["1"]],
  "fiber": [["1", "1"], ["1", "-1"], ["-1", "1"], ["-1", "-1"]]
}

```

Figure 2.6: Input file `system.json` for `Algpath`.

and numerical approximations of starting points that are solutions of F_0 .

The system F can have more than one parameter. The coefficients of F can be rationals or floating-point numbers, the latter interpreted as rationals. It is also possible to specify interval coefficients, although the user should be aware that `Track` is not ensured to terminate in this case. Note that specifying very tight intervals has no impact on performance.

The input must specify a piecewise-linear parameter path γ as a sequence of points in the parameter space (it can be different from $[0, 1]$). `Algpath` attempts to construct a Moore box for each starting point using `FindMb`, then tracks the corresponding paths along γ .

We showcase `Algpath`'s usage on a toy example. Consider the randomly generated system

$$\begin{cases} f_1 = x^2 + \frac{1}{2}xy - 3y^2 + \frac{1}{4}y = 0, \\ f_2 = 15x^2 - \frac{1}{6}y^2 + x + 39y + 1 = 0. \end{cases} \quad (2.27)$$

To solve (2.27), we build the linear homotopy $F : \mathbb{C} \times \mathbb{C}^2 \rightarrow \mathbb{C}^2$ with components $(1 - t)(1 - x^2) + tf_1$ and $(1 - t)(1 - y^2) + tf_2$, and track its solutions from 0 to 1. As usual with linear homotopies between real systems, the naive linear parameter path $[0, 1]$ should be avoided. Here, we choose to make a detour to the point $\frac{1}{2} + i\frac{1}{2}$. We create the input file `system.json` described in Figure 2.6 and run `algpath system.json`. It outputs approximations of the endpoints of the homotopy, which are solutions to (2.27). If we run `algpath system.json --stats`, the command outputs a set of statistics on the run, as in Figure 2.7.

Optional arguments of `Algpath` are described on the source code website, or by running `algpath --help`. For instance, the option `--homogenize` enables projective path tracking, as in Algorithm 8. `Algpath`'s website provides additional example runs.

```
{
  "failures": 0,
  "maxpreclist": [53, 53, 53, 53],
  "meanpreclist": [53.0, 53.0, 53.0, 53.0],
  "overheadtime": 0.004688670858740807,
  "steplist": [53, 22, 60, 46],
  "time": 0.0018600330222398043,
  "weightedmeanpreclist": [53.0, 53.0, 52.99999999999999, 53.0]
}
```

Figure 2.7: Output of `algpath system.json --stats`. *failures*: number of zeros that the software failed to track; *maxpreclist*: maximum precision used to track each zero; *meanpreclist*: mean precision used to track each zero; *overheadtime*: total computation time; *steplist*: number of steps used to track each zero; *time*: tracking time, excluding precomputations; *weightedmeanpreclist*: mean precision used to track each zero, weighted by step size.

2.6.5 Timings

We compared `Algpath` with the `Macaulay2` package `NumericalAlgebraicGeometry` [10] (certified in the BSS model). As a state-of-the-art implementation of noncertified path tracking, we also benchmarked the Julia package `HomotopyContinuation.jl` [21]. We used a computer with an AMD EPYC 9754 CPU, on a single thread. Homotopy continuation algorithms use very little memory, but we report some out-of-memory errors from `Julia` (to be investigated) and `Macaulay2` (which tries to expand structured polynomials). The benchmarking data, scripts and raw results are available at

<https://gitlab.inria.fr/numag/algp_{ath}-bench>.

Data set We considered linear homotopies $F_t(x) = tf(x) + (1 - t)g(x)$ between a start polynomial system g and a target system f . For the target, we considered several families:

dense dense systems with random standard independent complex coefficients;

katsura the Katsura benchmark family, where Katsura n is a polynomial system in $n + 1$ variables with 2^n solutions. It is available in `SageMath` as `sage.rings.ideal.Katsura`;

resultants ill-conditioned systems obtained by choosing two random dense polynomials $h_1, h_2 \in \mathbb{C}[x_1, \dots, x_n, y]$, setting $f_1 = \text{Res}_y(h_1, h_2)$, and choosing the $n - 1$ remaining equations as random dense polynomials;

structured random systems of low Waring rank, that have a small straight-line program representation but a large dense representation. The components of the systems are of the form $\pm 1 + \sum_{i=1}^5 \left(\sum_{j=1}^n a_{i,j} x_j \right)^d$, where the coefficients $a_{i,j}$ are independent and uniformly distributed in $\{-1, 0, 1\}$.

W_d the degree d Wilkinson polynomial $\prod_{i=1}^d (x - i)$ in expanded form; for $W_{20} + 1$, we keep W_{20} factorized;

clustered one-dimensional homotopies for which $F_{1/2}$ has clustered roots, making path jumping likely. We use the construction of Telen, Van Barel, and Verschelde [89, Section 5.4], with parameters $(n_c, CS, \alpha) = (5, 5, 10)$ and $(10, 5, 10)$.

For the start system, we considered both total-degree homotopies, with $g_i(x) = \gamma_i(x_i^d - 1)$ for random $\gamma_i \in \mathbb{C}$, and Newton homotopies, with $g(x) = f(x) - f(x_0)$ for a random x_0 .

Analysis of the results By substantially reducing the number of steps required to track a path relative to **Macaulay2**, **Algpath** can solve much larger problems. Moreover, it significantly narrows the gap between certified and non-certified methods.

Comparing the median number of steps performed by **Algpath** and **HomotopyContinuation.jl**² on each problem suggests that, typically, the latter performs only 2 or 3 times more steps. Figure 2.8 examines this relationship in more detail. The numbers of steps taken by **HomotopyContinuation.jl** and **Algpath** are strongly correlated on some examples, such as random dense or structured systems of dimension 8 and degree 3, but less so on others, such as difficult paths for the degree-1000 univariate polynomial, the Katsura-15 system or the 3-dimensional resultant problem. Moreover, the number of steps taken by **Algpath** varies considerably on challenging problems; see, for example, the median and maximum columns for the dense degree-1000 instance.

Finally, on some challenging inputs that **HomotopyContinuation.jl** can solve, the performance gap is large: for the dense degree-1000 instance, **Algpath** is 100 times slower than **HomotopyContinuation.jl**. Conversely, **Algpath** quickly solves some resultant and structured examples for which **HomotopyContinuation.jl** reports failures. The instances W_{19} and W_{20} are difficult for **Algpath**, as indicated by their iteration counts. Although **HomotopyContinuation.jl** also fails on these problems, more robust implementations may suffice while performing much better: for example, **PHCpack** solves W_{19} in at most 83 steps [89].

²Rigorous benchmarking of Julia code is difficult because of run-time compilation (JIT). To reduce compilation overhead, we first perform a warm-up computation with a degree-one polynomial system of the same dimension as the input system.

name	dim.	deg.	HomotopyContinuation.jl					Algpah				Macaulay2		
			time (s)	fail.	jump.	med.	max.	time (s)	prec.	med.	max.	time (s)	fail.	med. max.
dense	1	10	1.4			6	10	< 0.1		11	31	0.1	629	2146
dense	1	100	1.7			13	51	0.5		37	794		> 3 h	
dense	1	1000	6.7			15	100	12 min	59	76	17k	32	1000	
dense	1	2000	29	3		24	79	1 h	62	95	69k	202	2000	
dense	2	10 ²	1.9			22	74	0.6		53	178	141	33k	301k
dense	2	30 ²	4.2			24	127	63		85	682		> 3 h	
dense	2	50 ²	21			27	84	10 min		117	1674		> 3 h	
katsura	11	2 ¹⁰	5.5			100	179	21		177	360	27 min	21k	88k
katsura	16	2 ¹⁵	186			153	303	43 min		304	1845		> 3 h	
katsura	21	2 ²⁰	3 h			209	469	62 h	65	427	12k		> 8 GB	
resultants	3	16 · 4 ²	5.5			54	128	93	58	438	1855		> 3 h	
resultants	2	40 · 5	3.1	200				315	106	452	1413		> 3 h	
structured *	3	20 ³	2.9	12	×	107	164	7.2	106	270	636		> 3 h	
structured *	3	30 ³	2.9	92		111	133	50	106	380	818		> 3 h	
dense *	4	3 ⁴	1.9		×	39	67	0.8		65	129	9.4	3384	9936
dense *	8	3 ⁸	3.8			74	123	15		155	297	176	21k	74k
structured *	4	3 ⁴	3.5		×	40	78	0.3		75	198	11	4531	8925
structured *	8	3 ⁸	3.4			81	122	1.8		182	281	223	36k	97k
W ₁₉	1	19	1.6	17		97	101	1 h	111	249k	1005k		> 3 h	
W ₂₀	1	20	2.0	20				2 h	118	559k	2315k		> 3 h	
W ₂₀ + 1	1	20	2.1	20				1.1	106	309	396		> 3 h	
clustered (5, 5, 10)	1	25	3.0		×	80	151	0.3		263	837	179	204k	677k
clustered (10, 5, 10)	1	50	3.0		×	80	153	86	69	954	11k		> 3 h	
structured ^N	10	10 ¹⁰	2.7			53	53	< 0.1		123	123		> 3 h	
structured ^N	20	20 ²⁰		> 8 GB				1.1		1588	1588		> 8 GB	
structured ^N	30	30 ³⁰		> 8 GB				3.4		1988	1988		> 8 GB	
dense ^N	6	3 ⁶	2.0			33	33	< 0.1		90	90	1.3	22k	22k
dense ^N	8	3 ⁸	2.7			10	10	0.3		33	33	1.3	8775	8775
structured ^N	4	3 ⁴	2.6			32	32	< 0.1		67	67	0.2	8559	8559
structured ^N	8	3 ⁸	2.6			21	21	< 0.1		71	71	1.7	19k	19k

Table 2.2: Comparison of HomotopyContinuation.jl (non-certified), Algpah and Macaulay2 using Predictor => Certified. *Dim.*: number of variables; *deg.*: total degree of the target system; *time*: total time to track all the paths; *fail.*: number of reported failures; *jump.*: path jumping occurred. *med.*: median number of iterations over all paths; *max.*: maximum number of iterations in a single path; *prec.*: maximum precision used, if more than double precision was required; * A hundred randomly picked starting zeros of a total degree homotopy. ^N Newton homotopy (a single path to track).

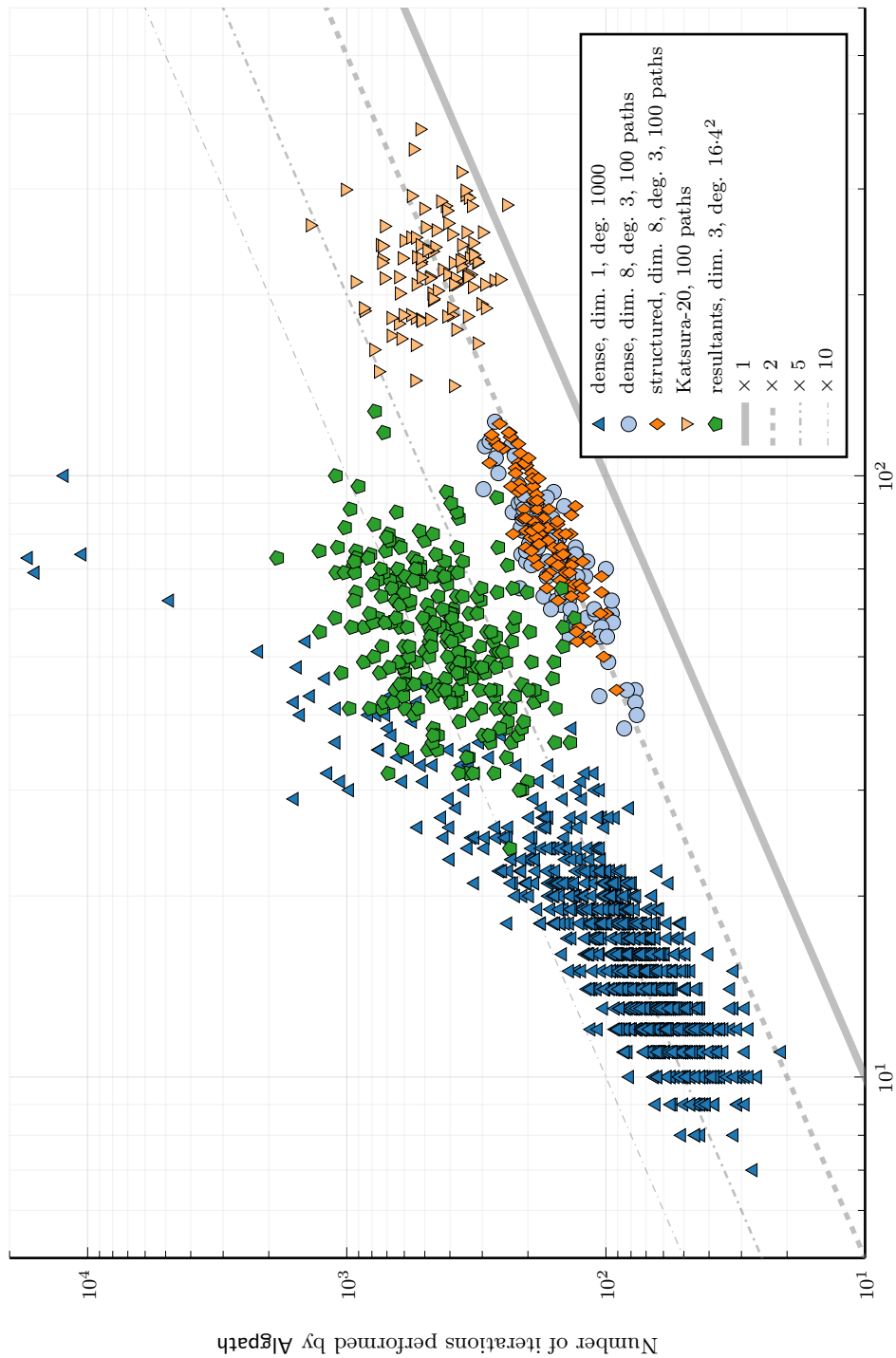


Figure 2.8: Numbers of iterations performed by Alpath and HomotopyContinuation.jl on five path-tracking problems.

Chapter 3

Monodromy and Belyi maps

This chapter is based on Guillemot and Voight [43]. Additionally, we detail how monodromy permutations can be computed in the setting of Chapter 2, and provide details on how to compute monodromy permutations using `Algpath`.

Let $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a parametrized polynomial map, and let F_t be the specialization of F at $t \in \mathbb{C}$ in the first variable. If instead of moving t from 0 to 1, we displace it along a parameter loop $\gamma : [0, 1] \rightarrow \mathbb{C}$ (that avoids a certain critical set), it induces a permutation of the solutions of $F_{\gamma(0)}$, called the *monodromy permutation associated to γ* . Computing monodromy requires certifying the entire path to ensure that path jumping or swapping does not occur; certified homotopy continuation ensures correctness of the results.

In this chapter, we provide an algorithm to compute monodromy permutations in the setting of Chapter 2. It is implemented in `Algpath`, and we apply it to the computation of the monodromy of Belyi maps. In particular, we compute the monodromy of all 1111 Belyi maps in the LMFDB.

3.1 Monodromy computations

3.1.1 Conventions

Permutations Let A be a set. If σ, τ are permutations on a set A , their product is defined by $\sigma\tau = \sigma \circ \tau$. If $a \in A$, we write $\sigma(a)$ for the evaluation of σ at a . The permutation group with this law is denoted by $S(A)$, and S_n if $A = [n] = \{1, \dots, n\}$.

Remark 3.1. Note that it is also standard to define $\sigma\tau = \tau \circ \sigma$, and in this case to write a^σ for the evaluation at a . For instance, `GAP` uses this convention.

We represent a concrete permutation $\sigma \in S_n$ either by the array of its images, denoted by $[\sigma(1), \sigma(2), \dots, \sigma(n)]$, or in the usual cycle notation. For instance,

$[4, 1, 2, 3]$ represents the permutation defined by $1 \mapsto 4$, $2 \mapsto 1$, $3 \mapsto 2$ and $4 \mapsto 3$. The latter permutation is given by $(1\ 4\ 3\ 2)$ in cycle notation.

Fundamental group Let X be a topological space, and $\gamma_1, \gamma_2 : [0, 1] \rightarrow X$ be paths. The *composite path* $\gamma_1\gamma_2 : [0, 1] \rightarrow X$ is the path obtained by traversing γ_1 first and then γ_2 , that is,

$$\gamma_1\gamma_2(t) = \begin{cases} \gamma_1(2t) & \text{if } t \in [0, \frac{1}{2}], \\ \gamma_2(2t - 1) & \text{else.} \end{cases}$$

The law on $\pi_1(X, x)$ is defined by $[\gamma_1][\gamma_2] = [\gamma_1 \cdot \gamma_2]$.

Monodromy right action Let $p : X \rightarrow B$ be a cover, and for $b \in B$, define $X_b = p^{-1}(b)$. Let $[\gamma] \in \pi_1(B, b)$, let $x \in X_b$. There exists a unique $\tilde{\gamma} : [0, 1] \rightarrow X$ such that $p \circ \tilde{\gamma} = \gamma$ and $\tilde{\gamma}(0) = x$. The monodromy right action is defined as $x \cdot [\gamma] = \tilde{\gamma}(1)$. The monodromy right action of a cover $X \rightarrow B$ induces a group homomorphism,

$$\begin{aligned} m : \pi_1(B, b) &\rightarrow S(X_b) \\ g &\mapsto \left\{ \begin{array}{ccc} X_b & \rightarrow & X_b \\ x & \mapsto & x \cdot g^{-1} \end{array} \right\}. \end{aligned} \quad (3.1)$$

Given $g \in \pi_1(B, b)$, the permutation $m(g)$ is called the *monodromy permutation associated to g* .

Assume p is a finite cover, and order the fiber $X_b = \{x_1, \dots, x_d\}$. Let $g \in \pi_1(B, b)$, with associated monodromy permutation $\sigma \in S_d$ obtained by identifying X_b with $\{1, \dots, d\}$. Then $x_{\sigma^{-1}(i)} = x_i \cdot g$, meaning that, to compute $\sigma^{-1}(i)$, we let g act on x_i and return the (unique) index $j \in [d]$ such that $x_i \cdot g = x_j$.

A rant that you can omit Usually, functions act on their domain from the *left*. Here, this convention clashes with the monodromy action of g on x , naturally defined as a *right* action. We cannot have standard permutation evaluation, the natural monodromy action and the monodromy representation map m to be a group homomorphism at the same time. In our case, the monodromy permutation is unnatural, as we would prefer $\sigma(x) = g \cdot x$ instead of $\sigma^{-1}(x) = g \cdot x$. The inverse permutation product convention (see Remark 3.1) solves this issue: permutation evaluation is then a right action, consistent with the monodromy action, although one must accept composing and evaluating permutations differently from all other functions. A more radical solution would be to redefine function evaluation as a right action and, for instance, write $(x)f$ instead of $f(x)$. This is particularly convenient: if $f : X \rightarrow Y$ and $g : Y \rightarrow Z$, we can define their composition by $(x)(f \circ g) = ((x)f)g$, reading the diagram $X \rightarrow Y \rightarrow Z$ from left to right. Obviously, this clashes with the traditional function evaluation convention.

3.1.2 Monodromy of finite maps

Let $F: \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a polynomial map, let X be the variety of pairs (t, x) such that $F_t(x) = 0$, let p be the projection onto the first variable, and denote by $X_t = p^{-1}(t)$ the fiber above $t \in \mathbb{C}$. Let $U \subseteq \mathbb{C}$ be such that X_t is finite, made of regular points and has constant cardinality over U , so that $p^{-1}(U) \rightarrow U$ defines a finite cover. Let $\gamma: [0, 1] \rightarrow U$ be a loop based at $b \in U$. We seek the monodromy permutation associated to γ .

Algorithm Let $x \in X_b$, and let $\zeta: [0, 1] \rightarrow \mathbb{C}^n$ be the solution path of F along γ . Then $t \mapsto (\gamma(t), \zeta(t))$ is the lift of γ through p , thus we can compute the monodromy permutation associated to γ by correctly tracking the solution path starting from each $x \in X_b$.

Track (Algorithm 5) is easily extended to track along general parameter segments by precomposing the system in the parameter variable with a parametrization of the segment. It can then be extended to track along a piecewise-linear path γ by tracking each piece successively. We enhance **Track** with an additional path argument; the call $\text{Track}(F, m, \gamma)$ tracks starting from m along γ , the latter being represented by a list of points in $\mathbb{C}$.

Input. A circuit $F: \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$; a piecewise-linear loop $\gamma: [0, 1] \rightarrow U$ starting at $b \in U$; X_b the fiber above b given as an array of $\frac{7}{8}$ -Moore boxes.

Output. The monodromy permutation associated to γ .

```

1  def Monodromy( $F, \gamma, X_b$ ):
2       $\sigma \leftarrow [1, \dots, d]$ 
3      for  $i$  from 1 to  $d$ :
4           $y \leftarrow \text{Track}(F, X_b[i], \gamma)$ 
5          for  $j$  from 1 to  $d$ :
6              if  $\text{Equals}(F_b, y, X_b[j])$ :
7                   $\sigma[j] \leftarrow i$ 
8                  break
9      return  $\sigma$ 
```

Algorithm 10: Monodromy computation using certified path tracking.

Assume that γ is piecewise-linear and that we are given the fiber X_b , each point being represented by a $\frac{7}{8}$ -Moore box. To compute σ , we track starting from each $x \in X_b$. If we denote the result by y , then σ maps (the zero associated

to) y to (the zero associated to) x . In practice, we order X_b and compute the monodromy permutation seen as an element of S_d . If x is the i -th point in X_b , we find $j \in [d]$ such that x_j and y represent the same zero using **Equals** (Algorithm 4), and let $\sigma(j) = i$. We do not attempt to optimize the matching of the fiber, as we observe experimentally that the bottleneck of the method lies in tracking each root. Note that the algorithm stores the monodromy permutation as the array of its images.

Computing monodromy permutations with Algp The `--monodromy` option of **Algp** computes the monodromy as **Monodromy** does, assuming that the parameter path is a loop and that the whole fiber is described in the input. Because the input fiber is given by numerical approximations instead of Moore boxes, the algorithm checks that all starting Moore boxes, computed from the numerical approximations with **FindMooreBox** (Algorithm 2), are disjoint using **Equals**. For instance, running `algp --monodromy` with input

```
{
  "system": ["x^4 - t"],
  "variables": ["x"],
  "parameters": ["t"],
  "path": [["1"], ["-1 + I"], ["-1 - I"], ["1"]],
  "fiber": [["1"], ["I"], ["-1"], ["-I"]]
}
```

yields `[3, 0, 1, 2]`. The latter corresponds to the permutation $[4, 1, 2, 3]$ (in Rust, array indices start at 0).

Remark 3.2. Because a permutation on d points is determined by its action on $d-1$ points, we perform all d path tracking runs in parallel and halt whenever $d-1$ runs have finished. This reduces the computation time when one zero is slower to track.

3.2 Belyi maps

Definition Let X be a smooth projective (irreducible) algebraic curve over $\mathbb{C}$ or equivalently a (connected) compact Riemann surface. A *Belyi map* is a non-constant map $\phi: X \rightarrow \mathbb{P}_{\mathbb{C}}^1$ unramified away from $\{0, 1, \infty\}$. We say that two Belyi maps $\phi: X \rightarrow \mathbb{P}_{\mathbb{C}}^1$, $\psi: Y \rightarrow \mathbb{P}_{\mathbb{C}}^1$ are *isomorphic* if there exists an isomorphism $\alpha: X \rightarrow Y$ such that

$$\begin{array}{ccc}
X & \xrightarrow{\alpha} & Y \\
\phi \downarrow & \swarrow \psi & \\
\mathbb{P}_{\mathbb{C}}^1 & &
\end{array}$$

commutes.

Permutation triples A permutation triple of degree d is a triple $(\sigma_0, \sigma_1, \sigma_\infty) \in S_d^3$ such that $\sigma_0 \sigma_1 \sigma_\infty = 1$. We say that two permutation triples $\sigma = (\sigma_0, \sigma_1, \sigma_\infty) \in S_d^3$ and $\sigma' = (\sigma'_0, \sigma'_1, \sigma'_\infty) \in S_d^3$ are *simultaneously conjugate* if there exists $\tau \in S_d$ such that $\sigma'_s = \tau^{-1} \sigma_s \tau$, for all $s \in \{0, 1, \infty\}$. A permutation triple $(\sigma_0, \sigma_1, \sigma_\infty)$ is *transitive* if $\langle \sigma_0, \sigma_1, \sigma_\infty \rangle = \langle \sigma_0, \sigma_1 \rangle$ is a transitive subgroup of S_d (it acts transitively on $[d]$).

Monodromy triple and monodromy group Let $U = \mathbb{P}_{\mathbb{C}}^1 \setminus \{0, 1, \infty\}$. Fix a basepoint $b \in U$ and generators g_0, g_1 and g_∞ of $\pi_1(U, b)$, represented by paths γ_0, γ_1 and γ_∞ , such that $g_0 g_1 g_\infty = 1$. Let $\phi : X \rightarrow \mathbb{P}_{\mathbb{C}}^1$ be a Belyi map. Since ϕ is unramified away from $\{0, 1, \infty\}$, it restricts to a topological cover $\phi^{-1}(U) \rightarrow U$ of degree d . Label the fiber $X_b = \phi^{-1}(b) = \{x_1, \dots, x_d\}$. From the monodromy action of $\pi_1(U, b)$ on X_b , we obtain three monodromy permutations $\sigma_0, \sigma_1, \sigma_\infty \in S_d$. Moreover, $\sigma_0 \sigma_1 \sigma_\infty = 1$, since the map (3.1) assigning g to its monodromy permutation is a group homomorphism, and the triple $(\sigma_0, \sigma_1, \sigma_\infty)$ is transitive as X is connected.

Although the permutations depend on the ordering of X_b , mapping ϕ to the triple $(\sigma_0, \sigma_1, \sigma_\infty)$ is well defined if we consider permutation triples up to simultaneous conjugacy. We call $(\sigma_0, \sigma_1, \sigma_\infty)$ (precisely, its equivalence class up to simultaneous conjugacy) the *monodromy triple associated with ϕ* . We call the subgroup $\langle \sigma_0, \sigma_1 \rangle = \langle \sigma_0, \sigma_1, \sigma_\infty \rangle \leq S_d$ the *(geometric) monodromy group*. Again, this group, as a subgroup of S_d , depends on the ordering of X_b , but is well defined up to conjugacy in S_d (thus, up to isomorphism). The cycle structure of σ_s records the ramification above $s \in \{0, 1, \infty\}$.

Proposition 3.1. *The mapping that assigns a Belyi map to its monodromy triple is a bijection between Belyi maps of degree d , up to isomorphism, and the set of transitive permutation triples of degree d , up to simultaneous conjugacy.*

A proof of Proposition 3.1 can be found in [56, Lemma 1.1], for instance. A consequence is that there are only finitely many Belyi maps of fixed degree d .

Belyi's theorem and model computation Belyi [12] proved that a smooth projective curve X admits a Belyi map $X \rightarrow \mathbb{P}^1$ if and only if X can be defined

over $\mathbb{Q}^{\text{al}}$ the algebraic closure of $\mathbb{Q}$. Together with Proposition 3.1, this result leads to the following problem:

Problem 1. Given a transitive permutation triple $\sigma = (\sigma_0, \sigma_1, \sigma_\infty)$, compute equations, possibly with algebraic number coefficients, for the Belyi map with monodromy triple σ .

Problem 1 can be tackled in various ways; see Sijsling and Voight [83] for an overview, as well as the more recent work by Roberts [77], Barth and Wenz [3], and Berghaus, Monien, and Radchenko [13]. These methods take as input a permutation triple encoding the monodromy around the branch points and return candidate equations over a number field $K \subset \mathbb{C}$. One can certify that the map is indeed a Belyi map with the correct ramification. Unfortunately, for all known methods, the monodromy triple is not certified to coincide with the given input. Algebraic techniques [37, 3] may suffice to compute the monodromy group and the triple when uniquely determined, but do not provide a general approach. On the other hand, the monodromy triple can be computed using certified homotopy continuation.

Remark 3.3. Consider a Belyi map $\phi : X \rightarrow \mathbb{P}^1$ of degree d . We relax the irreducibility assumption on X and assume that ϕ is nonconstant on every irreducible component of X . Since X may be disconnected, the monodromy group of ϕ may fail to be transitive. In this case, one can decompose $[d]$ as the disjoint union of the orbits of the action of G on $[d]$, and restrict σ , the monodromy triple associated with ϕ , to each of these orbits to recover transitive triples. Then ϕ is isomorphic to the disjoint union of Belyi maps associated to the restricted triples. Thus, computing Belyi maps with a possibly disconnected underlying curve reduces to computing Belyi maps with a connected underlying curve, as we do in this chapter.

3.3 Monodromy verification of Belyi maps

Consider a degree d Belyi map $\phi : X \rightarrow \mathbb{P}_{\mathbb{C}}^1$ and assume an affine chart $Y \subseteq \mathbb{C}^n$ of X , cut out by polynomials $g_1, \dots, g_{n-1}$. Suppose ϕ is represented on this chart by a rational function p/q . Consider the polynomial system

$$F_t(x, z) = (g_1(x), \dots, g_{n-1}(x), p(x) - tq(x), q(x)z - 1). \quad (3.2)$$

Let $Z = \{(t, x, z) \in \mathbb{C} \times \mathbb{C}^n \times \mathbb{C} : F_t(x, z) = 0\}$. Let $V \subseteq \mathbb{C} \setminus \{0, 1\} = U \cap \mathbb{C}$ be a Zariski-dense open subset over which the projection π of Z onto the t -component defines a cover. Since π identifies with the restriction of ϕ to the dense open subset $Y \setminus V(q)$ of X (as $q(x)z - 1 = 0$ excludes the vanishing locus of q), the regular zeros of F_t are in bijection with the points of $\phi^{-1}(t)$ for each $t \in V$.

Therefore, if $\gamma: [0, 1] \rightarrow V$ is a loop based at b and each regular zero of F_b is tracked correctly along γ , then the induced permutation of the zeros of F_b is the monodromy permutation of ϕ associated with γ . In particular, it is enough to compute the permutations associated to loops around 0 and 1.

3.3.1 LMFDB Belyi maps

The LMFDB provides a Belyi map section, with 1111 Belyi map entries up to degree 9. The equations for the maps were generated using the method of Klug, Musty, Schiavone, and Voight [56]. We briefly review the data representing these maps.

Smooth model. In this case, the Belyi map is given by a polynomial $f \in \mathbb{C}[x, y]$ defining the curve and a bivariate rational function $p/q \in \mathbb{C}(x, y)$ defining the map. By homogenizing, we obtain a smooth curve $X \subseteq \mathbb{P}^2$, and p/q induces a map $\phi: X \rightarrow \mathbb{P}^1$. The system we use to compute the monodromy is

$$f = p - tq = 0. \quad (3.3)$$

For a fixed $t \in \mathbb{C}$, this system captures the fiber $\phi^{-1}(t)$ together with the points of X at which p and q vanish simultaneously. Adding $qz - 1 = 0$, as in (3.2), removes the latter but may greatly slow down path tracking. In practice, we compute starting solutions using the enlarged system and then track (3.3), excluding the solutions on which q vanishes. More precisely, from a Moore box for the enlarged system with center (x, y, z) , we compute a Moore box for (3.3) centered at (x, y) ; Theorem 2.3 ensures that q does not vanish at the associated zero.

Smooth case with curve $\mathbb{P}^1$. Whenever the underlying curve is $\mathbb{P}^1$, the LMFDB provides a rational function $p/q \in \mathbb{C}(x)$ representing a map $\mathbb{P}^1 \rightarrow \mathbb{P}^1$. In this case, we compute the monodromy using $p - tq = 0$ after checking that p and q are coprime.

Plane models. In addition to the smooth model, some entries provide a polynomial equation $f \in \mathbb{C}[t, x]$ defining a possibly singular curve and a constant $\lambda \in \mathbb{C}$. If $\pi: V(f) \rightarrow \mathbb{C}$ is the projection onto the variable t , then λp induces a map from the normalization of $V(f)$ to $\mathbb{P}^1$, and we compute the monodromy by tracking the roots of $f(\lambda^{-1}t, x)$.

3.3.2 Implementation details

In this subsection, we give some details about the process of computing the monodromy triples of LMFDB entries.

Dealing with algebraic coefficients `Algpath` can handle polynomial systems with rational, floating-point, or interval coefficients, whereas the equations defining Belyi maps may have algebraic coefficients. Each LMFDB entry specifies a number field $\mathbb{Q}(\nu)$ by the minimal polynomial m_ν of ν , and the coefficients of the equations are polynomials in ν with rational coefficients. Given an embedding $\mathbb{Q}(\nu) \hookrightarrow \mathbb{C}$, corresponding to a root α of m_ν , we obtain a Belyi map by evaluating at α .

One possibility is to treat ν as an additional variable, add m_ν to the system, and extend each starting zero with α on the ν -coordinate. This ensures termination but introduces a substantial cost, and the closeness of the roots of m_ν may force small Moore boxes and hence small step sizes. In practice, we instead compute a tight interval around α and replace ν by this interval in the system. If the interval is not tight enough, the algorithm may stall, but whenever a run succeeds the result is correct. This performs much better in practice: for the Belyi map with entry 7T7-6.1.5.2.4.2.1-a, adding the number field equation did not finish within 6 days, whereas replacing α by an interval took less than 20 seconds.

Computing the fiber above the base point To compute the monodromy of a Belyi map, we first need a Moore box for each point in the fiber above the base point of the loops around 0 and 1. We numerically compute the fiber and then certify $\frac{1}{4}$ -Moore boxes around the resulting approximations. The whole fiber is captured as soon as the number of Moore boxes matches the degree of the map and they represent distinct zeros, and the latter is checked by `Algpath`.

In the smooth case where the curve is $\mathbb{P}^1$, or when a plane model is available, this amounts to solving a univariate polynomial with complex coefficients after embedding the equations into $\mathbb{C}$, so we use a standard root finder in `SageMath`. Otherwise we use `Msolve` [14] on (3.3), before embedding the equations into $\mathbb{C}$ and treating ν as a variable. This yields fibers, for all embeddings of the number field, which we sort using the ν -coordinate. For each fiber, we also get the unwanted common locus of p and q over X ; to remove it, we discard the points whose evaluation by q is the smallest, until the size of the fiber matches the degree of the map. We choose `Msolve` as it is directly accessible from `SageMath` and requires no additional setup, but we also experimented with `HomotopyContinuation.jl` [21] and found it to be a suitable alternative for this task.

Practical results We compute the triples for all 1111 entries in the LMFDB database. The certification scripts are written in `SageMath`. They can be found online, together with the results, at

<https://gitlab.inria.fr/aguillem/belyi-monodromy>.

Using plane models when available, the whole computation takes 1.2 hours of CPU

time on an Intel i7-1370P CPU, and the total wall-clock time is 17 minutes on 6 threads.

Using the resulting triples, we prove correctness of all monodromy groups in the database. The computations brought to light a few bugs; see

<https://github.com/LMFDB/lmfdb/issues/6796>,
<https://github.com/LMFDB/lmfdb/issues/6798>.

First, the triples computed using plane models only correspond to the triples on the database up to an S_3 -action, coming from post-composition by an automorphism of $\mathbb{P}^1$ that permutes $\{0, 1, \infty\}$. In this case, we check correctness only up to this S_3 -action. Second, we found two entries for which the list of embeddings of the number field is incorrect (one embedding appears twice and another is missing):

- 9T33-5.3.1_5.1.1.1.1_4.3.2-a;
- 9T34-6.1.1.1_5.2.2_4.3.1.1-a.

We also found five entries for which an error in the ordering of the triples in the table breaks the correspondence between embeddings and triples (although the set of triples itself is correct):

- 6T10-4.2_4.2_2.2.1.1-a;
- 7T6-7_5.1.1_5.1.1-c;
- 7T3-3.3.1_3.3.1_3.3.1-a;
- 7T7-6.1_6.1_3.2.2-b.
- 7T4-6.1_3.3.1_2.2.2.1-a;

Additionally, we verified the monodromy of the maps provided by Barth and Wenz [3], whose degrees range from 55 to 280, with two exceptions: the two maps of highest degrees (266 and 280) are defined over a nontrivial number field, and although their monodromy groups could be verified using algebraic techniques, their monodromy triples remain unverified.

Chapter 4

Braid computations

*This chapter is based on Guillemot and Lairez [41]. We provide additional details on the associated software **Braids**.*

First, we review configuration spaces, geometric and combinatorial braids, and revisit braid computations in the exact setting; the algorithm is similar to the one implemented in **SIROCCO** [64].

Next, we explore braid computations when the braid is only known approximately. We precisely define what is *approximate data* for a geometric braid by providing a data structure that reduces the braid to a geometric predicate, **Sep**. This data structure is well suited to braids arising from tracking the roots of a parametrized polynomial $f \in \mathbb{C}[t][x]$ using certified homotopy continuation, which typically output tubular neighborhoods around the solution paths. Then, we define arrangements, which capture the relative positions of the points inducing a geometric braid and can be computed using **Sep**. We introduce the notion of a *covering sequence of arrangements* of a geometric braid, which characterizes the braid, and provide an algorithm **Cover** to compute such a sequence from a braid given by a **Sep** predicate. Finally, we provide an algorithm, **Braid**, that computes a combinatorial braid from a sequence of arrangements using *permutation points*. Combining **Cover** and **Braid** yields an algorithm that computes the combinatorial braid associated with a geometric braid given by its **Sep** predicate.

In the last section of this chapter, we revisit braid computations from an implementation point of view. We merge **Cover** and **Braid** into a single algorithm, **Braid'**, and explain how to efficiently update pointed arrangements, as required by **Braid'**. We demonstrate how **Sep** can be implemented on the output of **Algp**, and provide details on **Braids**, a **Rust** implementation of **Braid**.

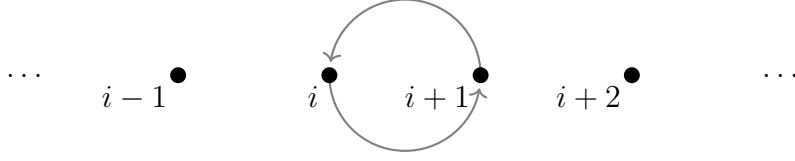


Figure 4.1: Path in C_n inducing the combinatorial braid σ_i .

4.1 Braids

Subsection 4.1.1 recalls the definitions of geometric and combinatorial braids. Subsection 4.1.2 explains how to compute combinatorial braids from geometric ones in an exact setting.

4.1.1 Configuration spaces and braids

Configuration space and geometric braids For $n \geq 1$, let OC_n denote the *ordered configuration space of n points in $\mathbb{C}$* :

$$OC_n = \{(z_1, \dots, z_n) \in \mathbb{C}^n \mid \forall i, j \in [n], i \neq j \Rightarrow z_i \neq z_j\}.$$

The permutation group S_n acts on OC_n by permuting the z_i : for a permutation $\sigma \in S_n$,

$$\sigma \cdot (z_1, \dots, z_n) = (z_{\sigma^{-1}(1)}, \dots, z_{\sigma^{-1}(n)}).$$

The (*unordered*) *configuration space of n points in $\mathbb{C}$* is the quotient space $C_n = OC_n/S_n$, endowed with the quotient topology. A *geometric braid* is the homotopy class of a path $F : [0, 1] \rightarrow C_n$.

Combinatorial braids Let us fix $[n] = \{1, \dots, n\} \in C_n$ as a base point. A geometric braid F such that $F(0) = F(1) = [n]$ defines an element of the fundamental group $\pi_1(C_n, [n])$. Artin [2] introduced the *braid group*

$$B_n = \langle \sigma_1, \dots, \sigma_{n-1} \mid \sigma_i \sigma_{i+1} \sigma_i = \sigma_{i+1} \sigma_i \sigma_{i+1}, |i - j| > 1 \Rightarrow \sigma_i \sigma_j = \sigma_j \sigma_i \rangle$$

and proved that $B_n \simeq \pi_1(C_n, [n])$. We call *combinatorial braids* the elements of B_n , as opposed to geometric braids. The isomorphism is given by mapping the generator σ_i to the path exchanging i and $i+1$, with the right point passing above the left point (Figure 4.1).

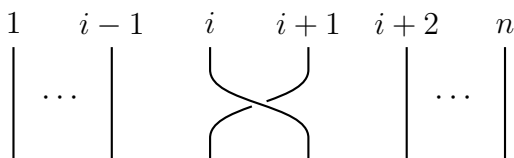
When a geometric braid does not start or end at the base point $[n]$, it is still possible to associate a combinatorial braid in a natural way. To this end, we define a canonical path R_c from $[n]$ to c for any configuration $c = \{z_1, \dots, z_n\}$ in C_n as follows: order the points z_i lexicographically, comparing the real parts

first and breaking ties using the imaginary parts. The path R_c is given by the linear interpolation

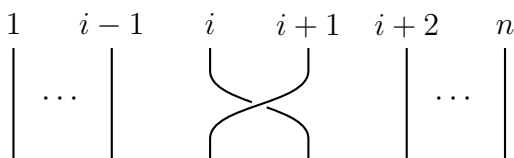
$$R_c(t) = \{(1-t)k + tz_k \mid k \in [n]\}.$$

For every $t \in [0, 1]$, the points are strictly lexicographically ordered, so they never collide. Given a geometric braid $F : [0, 1] \rightarrow C_n$, we consider the concatenation¹ $\widetilde{F} = R_{F(0)} \cdot F \cdot R_{F(1)}^{-1}$, which defines an element of $\pi_1(C_n, [n])$ and therefore induces a combinatorial braid via Artin's isomorphism. This convention works well with concatenation: for any two paths F and G in C_n such that $F(1) = G(0)$, the path $\widetilde{F \cdot G}$ is homotopic to $\widetilde{F} \cdot \widetilde{G}$.

Braid diagrams We can visualize braids using *braid diagrams*. To σ_i we associate the diagram



where strand $i + 1$ is broken when it meets strand i , to indicate the positive sign of the crossing, as if strand i passed over strand $i + 1$. This diagram corresponds to Figure 4.1 after projection onto the real axis seen from $\text{Im} = -\infty$, with time running from top to bottom. To σ_i^{-1} we associate the same picture, with strand i broken instead of strand $i + 1$.



To visualize a general combinatorial braid b , we concatenate, from top to bottom, diagrams corresponding to a decomposition of b into standard generators, as in Figure 4.2. Braid diagrams conveniently illustrate the relations of B_n arising from homotopies between geometric braids: two diagrams represent the same braid if one can be transformed into the other by deforming the strands horizontally (Figure 4.3).

¹Following the convention of Section 3.1.1.

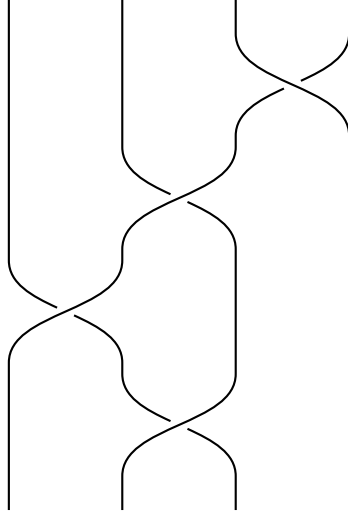


Figure 4.2: Braid diagram corresponding to $\sigma_3\sigma_2^{-1}\sigma_1^{-1}\sigma_2^{-1}$.

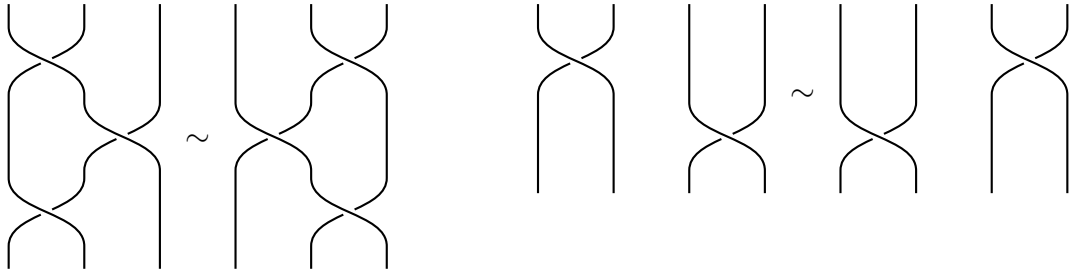


Figure 4.3: On the left, diagrams representing the braids $\sigma_1\sigma_2\sigma_1$ and $\sigma_2\sigma_1\sigma_2$. On the right, diagrams representing $\sigma_1\sigma_3$ and $\sigma_3\sigma_1$.

4.1.2 Computing braids in an exact setting

To better understand the approximate setting, first consider the problem of computing the combinatorial braid associated to an exact representation of a geometric braid. There are many possible ways to represent a continuous function $[0, 1] \rightarrow C_n$; for computing braids, a practical choice is a piecewise linear representation: choose points $p_0, \dots, p_r \in \mathbb{C}^n$ (or rather $\mathbb{Q}[\mathbf{i}]^n$, in order to perform exact arithmetic) and define the function $F : [0, 1] \rightarrow \mathbb{C}^n$ by piecewise linear interpolation:

$$F(t) = (i + 1 - rt)p_i + (rt - i)p_{i+1}, \quad \frac{i}{r} \leq t \leq \frac{i+1}{r}.$$

Assume that the n coordinates of this function stay distinct; then $F : [0, 1] \rightarrow OC_n$ is a path. This data structure is *exact*, and it allows easy computation of:

- for any $t \in [0, 1]$, the unique permutation $\text{ord}_F(t)$ such that $\text{ord}_F(t) \cdot F(t)$ is lexicographically ordered;
- the discontinuity points of ord_F^2 ;
- the value of $\text{ord}_F(t)$ on the right of a discontinuity point.

Algorithm Braid (Algorithm 11) computes the braid associated with F . In the generic situation, where there are no multiple crossings (at any time, at most two points share the same real part), the proof is straightforward because one can always choose $s = 1$ in the decomposition in Line 8. In non-generic situations, the proof is more technical, and we omit it because it is not the main focus of this chapter.

Limitations When the geometric braid F is specified only approximately (for instance, by tubular neighborhoods), the ordering function ord_F is underspecified and cannot be computed anymore. One solution is to compute an exact piecewise linear path $G : [0, 1] \rightarrow OC_n$ such that:

1. the lexicographic ordering of the coordinates of $F(0)$ (resp. $F(1)$) is identical to that of $G(0)$ (resp. $G(1)$);
2. the path $F(0) \rightarrow G(0) \xrightarrow{G} G(1) \rightarrow F(1)$ (where the first and last arrows are straight lines) is homotopic to F .

²It is sufficient to compute a finite superset of the discontinuity points, such as the union over all $1 \leq i < j \leq n$ of the boundary of the set of all $t \in [0, 1]$ such that $\text{Re}(F_i(t)) = \text{Re}(F_j(t))$.

```

1  def Braid( $F$ ):
2       $T \leftarrow$  all discontinuity points of  $\text{ord}_F$  (or a superset of those)
3       $b \leftarrow 1$  //the identity braid
4       $\alpha \leftarrow \text{ord}_F(0)$ 
5      for  $t \in T$  in increasing order:
6          //loop invariant:  $\alpha = \lim_{s \rightarrow t-} \text{ord}_F(s)$ 
7           $\beta \leftarrow \lim_{s \rightarrow t+} \text{ord}_F(s)$  //value on the right of the discontinuity point
8          write  $\beta\alpha^{-1}$  as a product of transpositions  $(j_s \ j_s + 1) \cdots (j_1 \ j_1 + 1)$ 
9          for  $i$  from 1 to  $s$ :
10              $\epsilon \leftarrow \text{sign} \left( \text{Im} \left( F_{\alpha^{-1}(j_i+1)}(t) - F_{\alpha^{-1}(j_i)}(t) \right) \right)$ 
11              $b \leftarrow b \cdot \sigma_{j_i}^\epsilon$  //update the braid
12              $\alpha \leftarrow (j_i \ j_i + 1) \cdot \alpha$  //update the strand ordering
13  return  $b$ 

```

Algorithm 11: Braid computation in an exact setting.

In this case, the braids induced by F and G are the same, and **Braid** applies to G . This is the approach taken by Marco-Buzunariz and Rodríguez [64]. But this still requires a lexicographic ordering of $F(0)$ and $F(1)$, which may not be available in a numerical context. From a practical point of view, even if the ordering of the endpoints is not ambiguous, it may still be hard to compute such a path G , for example when F is given as tubular neighborhoods of Taylor models, as in the algorithm presented in Section 2.4 and in [93]. The number of line segments required to approximate a nonlinear path within a tubular neighborhood is unbounded as the tube radius decreases.

4.2 Approximate data and arrangements

Section 4.2.1 introduces a definition of *path approximation* that provides the basic context for computing braids from approximate data. Section 4.2.2 studies *arrangements*, a combinatorial abstraction representing the positions of n points in the plane. Sequences of arrangements provide an intermediate representation between geometric and combinatorial braids.

4.2.1 Coping with numerical ambiguity

The braid induced by a path $F : [0, 1] \rightarrow \mathbb{C}^n$ is a topological object, invariant under deformation, so it should be computable from approximate data. But what does it mean to have approximate data? Concrete numerical values are not important; the goal is only to separate the points and capture their winding. To that end, it is enough to know, for any $t \in [0, 1]$ and any pair $1 \leq i < j \leq n$, a coordinate axis, Re or Im , along which $F_i(t)$ and $F_j(t)$ are separated, together with their relative order along this axis. This leads to the following definition.

Definition 4.1. An *approximation of a path* $F : [0, 1] \rightarrow OC_n$ is the data of two families of open subsets (R_{ij}) and (I_{ij}) of $[0, 1]$, indexed by distinct $i, j \in [n]$, such that, for all such pairs,

1. $\text{Re}(F_i(t)) < \text{Re}(F_j(t))$ for all $t \in R_{ij}$,
2. $\text{Im}(F_i(t)) < \text{Im}(F_j(t))$ for all $t \in I_{ij}$,
3. $R_{ij} \cup R_{ji} \cup I_{ij} \cup I_{ji} = [0, 1]$.

In computational terms, separation queries provide the only access to path approximations: given t and a pair $1 \leq i < j \leq n$, we want to obtain at least one element of $\{R_{ij}, R_{ji}, I_{ij}, I_{ji}\}$ containing t , whose existence is guaranteed by 3. This leads to the following computation-oriented variant of Definition 4.1; the rest of the chapter uses it. The two definitions are equivalent, in that an approximation for one definition gives an approximation for the other definition.

Definition 4.2 (Path approximation, computational variant). An *approximation of a path* $F : [0, 1] \rightarrow OC_n$ is the data, for each distinct $i, j \in [n]$, of a computable map

$$\text{Sep}_F(i, j, -) : [0, 1] \longrightarrow [n]^2 \times \{\text{Re}, \text{Im}\} \times [0, 1]$$

such that for all $t \in [0, 1]$, the value $(i', j', q, t') = \text{Sep}_F(i, j, t)$ satisfies:

- (sep_a) $(i', j') = (i, j)$ or (j, i) ;
- (sep_b) for all $s \in [t, t']$, $q(F_{i'}(s)) < q(F_{j'}(s))$;
- (sep_c) $t' \geq \min(1, t + \epsilon_F)$, for some $\epsilon_F > 0$ depending only on F .

4.2.2 Arrangements

Arrangements, arrangement cells An arrangement is a combinatorial structure that represents the relative position of points in the complex plane and reflects the information provided by the approximation of a path (Definitions 4.1 and 4.2).

Precisely, an *arrangement on n points* is a pair $(\prec_{\text{Re}}, \prec_{\text{Im}})$ of strict partial orders³ on $[n]$ such that any distinct $i, j \in [n]$ are comparable in at least one of them. The *arrangement cell* attached to an arrangement $(\prec_{\text{Re}}, \prec_{\text{Im}})$ is the subset $A \subset OC_n$ defined by

$$A = \{(z_1, \dots, z_n) \in \mathbb{C}^n \mid i \prec_{\text{Re}} j \Rightarrow \text{Re}(z_i) < \text{Re}(z_j) \\ \text{and } i \prec_{\text{Im}} j \Rightarrow \text{Im}(z_i) < \text{Im}(z_j) \text{ for all } i, j \in [n]\}.$$

Lemma 4.1. *Arrangement cells are nonempty, convex, open subsets of OC_n , and they cover OC_n . Moreover, any nonempty intersection of arrangement cells is itself an arrangement cell.*

Proof. Let $(\prec_{\text{Re}}, \prec_{\text{Im}})$ be an arrangement and write A for its cell. The inequalities defining A are strict, so A is open in $\mathbb{C}^n$, and they are real-linear, so A is convex.

To see that A is nonempty, choose linear extensions $\overline{\prec}_{\text{Re}}$ and $\overline{\prec}_{\text{Im}}$ of the partial orders $\prec_{\text{Re}}$ and $\prec_{\text{Im}}$, and choose permutations π and ϕ of $[n]$ such that $\pi^{-1}(1) \overline{\prec}_{\text{Re}} \dots \overline{\prec}_{\text{Re}} \pi^{-1}(n)$ and $\phi^{-1}(1) \overline{\prec}_{\text{Im}} \dots \overline{\prec}_{\text{Im}} \phi^{-1}(n)$. The configuration $(\pi(k) + \phi(k)\mathbf{i})_{1 \leq k \leq n}$ is in A .

The inclusion $A \subseteq OC_n$ is immediate: given $z \in A$ and $i \neq j$, the definition of an arrangement requires one of $i \prec_{\text{Re}} j$, $j \prec_{\text{Re}} i$, $i \prec_{\text{Im}} j$, or $j \prec_{\text{Im}} i$ to hold, which implies either $\text{Re}(z_i) \neq \text{Re}(z_j)$ or $\text{Im}(z_i) \neq \text{Im}(z_j)$, so $z_i \neq z_j$.

For the covering property, let $z \in OC_n$ and define $i \prec_{\text{Re}} j$ whenever $\text{Re}(z_i) < \text{Re}(z_j)$, and $i \prec_{\text{Im}} j$ whenever $\text{Im}(z_i) < \text{Im}(z_j)$. Distinct points of z differ in real or imaginary part, so every pair becomes comparable in at least one order. So $(\prec_{\text{Re}}, \prec_{\text{Im}})$ is an arrangement whose cell, by construction, contains z .

Finally, consider two arrangements $(\prec_{\text{Re}}, \prec_{\text{Im}})$ and $(\prec'_{\text{Re}}, \prec'_{\text{Im}})$ with cells A and A' . If some i and j satisfy $i \prec_{\text{Re}} j$ and $j \prec'_{\text{Re}} i$, then $A \cap A' = \emptyset$, and similarly for $\prec_{\text{Im}}$ and $\prec'_{\text{Im}}$. So if $A \cap A' \neq \emptyset$, then the binary relations $\prec_{\text{Re}} \cup \prec'_{\text{Re}}$ and $\prec_{\text{Im}} \cup \prec'_{\text{Im}}$ are acyclic, and their transitive closures yield partial orders. The intersection $A \cap A'$ is the arrangement cell attached to this pair of orders. $\square$

Path cover A finite sequence $(A_1, \dots, A_r)$ of arrangement cells *covers* a path $F : [0, 1] \rightarrow OC_n$ if there exists a partition $0 = t_0 \leq t_1 \leq \dots \leq t_r = 1$ such that $F([t_{i-1}, t_i]) \subseteq A_i$ for every i .

Because OC_n is an open subset of $\mathbb{C}^n$ covered by arrangement cells, compactness of $[0, 1]$ guarantees that every path in OC_n admits a finite covering sequence.

Recall that two paths $F, G : [0, 1] \rightarrow OC_n$ are *homotopic* if there exists a continuous map $H : [0, 1]^2 \rightarrow OC_n$ with $H(0, t) = F(t)$, $H(1, t) = G(t)$, and $H(s, 0) = F(0) = G(0)$, $H(s, 1) = F(1) = G(1)$ for all s, t . The next lemma states

³A *strict partial order* is a transitive, irreflexive binary relation.

that the homotopy class of a path is determined by its endpoints together with any arrangement sequence covering it.

Lemma 4.2. *Let $F, G : [0, 1] \rightarrow OC_n$ be two paths. Suppose*

1. $F(0) = G(0)$ and $F(1) = G(1)$, and
2. *there exists a sequence of arrangement cells that covers both F and G .*

Then F and G are homotopic.

Proof. Let $(A_1, \dots, A_r)$ be a common covering sequence and choose partitions $0 = t_0 \leq \dots \leq t_r = 1$ for F and $0 = s_0 \leq \dots \leq s_r = 1$ for G realizing the cover. Up to homotopy, one can reparametrize F and G so that $s_i = t_i$ and inequalities are strict.

Define $H : [0, 1]^2 \rightarrow \mathbb{C}^n$ by $H(s, t) = (1 - s)F(t) + sG(t)$. This is a homotopy between F and G : the boundary conditions are clearly satisfied, so it suffices to check that $H(s, t)$ stays inside OC_n . Fix $t \in [0, 1]$ and choose i with $t_{i-1} \leq t \leq t_i$. By construction, both $F(t)$ and $G(t)$ lie in A_i , and A_i is convex; hence $H(s, t) \in A_i \subset OC_n$ for all s . $\square$

Covering sequence from a path approximation *Cover* (Algorithm 12) computes a covering sequence of arrangements from a path approximation. It is illustrated in Figure 4.4. In this example, the algorithm produces a sequence of five arrangements. Arrangements are represented by subsets of $[n]^2 \times \{\text{Re}, \text{Im}\}$, which we interpret as graphs with vertex set $[n]$ and edges labeled by Re or Im. The partial orders $\prec_{\text{Re}}$ and $\prec_{\text{Im}}$ are the reachability relations restricted to Re- and Im-labeled edges, respectively. The map *lifetime* stores the time until which each edge remains valid.

Proposition 4.3. *Cover (Algorithm 12) terminates and outputs a sequence covering F .*

Proof. For termination, let $\epsilon > 0$ be given by Item (*sep_c*) and, for each integer $k \geq 0$, let n_k denote the number of edges in A whose lifetime lies in $[k\epsilon, (k + 1)\epsilon)$. Considering the tuple $(n_k)_{0 \leq k \leq \frac{1}{\epsilon}}$ at the end of each iteration, we obtain a strictly decreasing sequence for the lexicographic ordering. Thus only finitely many iterations occur.

Correctness rests on the following invariant coming from the defining properties of **Sep**: whenever (i, j, q) belongs to A with lifetime t' , the inequality $q(F_i(s)) < q(F_j(s))$ holds for all $s \in [t, t']$. $\square$

Input. An approximation of a path $F : [0, 1] \rightarrow OC_n$, given as a **Sep** method.

Output. A sequence of arrangements covering F .

```

1  def Cover( $F$ ):
2       $A \leftarrow \{\}$  //edge set of a graph encoding the current arrangement
3       $lifetime \leftarrow \{\}$  //map edges of  $A$  to their lifetime
4       $\mathcal{L} \leftarrow []$  //list of arrangements covering  $F$ 
5       $t \leftarrow 0$ 
6      while  $t < 1$ :
7          //Complete the graph until it represents an arrangement containing  $F(t)$ .
8          while  $A$  does not define an arrangement:
9               $i, j \leftarrow$  a non-comparable pair
10              $e, t' \leftarrow \text{Sep}(F, i, j, t)$  // $e \in [n]^2 \times \{\text{Re}, \text{Im}\}$ 
11              $A \leftarrow A \cup \{e\}$  // $i$  and  $j$  are now comparable
12              $lifetime[e] \leftarrow t'$ 
13             append a copy of  $A$  to  $\mathcal{L}$ 
14             //Advance time until some edge expires.
15              $e \leftarrow$  edge of  $A$  with smallest lifetime
16              $A \leftarrow A \setminus \{e\}$ 
17              $t \leftarrow lifetime[e]$ 
18  return  $\mathcal{L}$ 

```

Algorithm 12: Computation of an arrangement sequence covering a path.

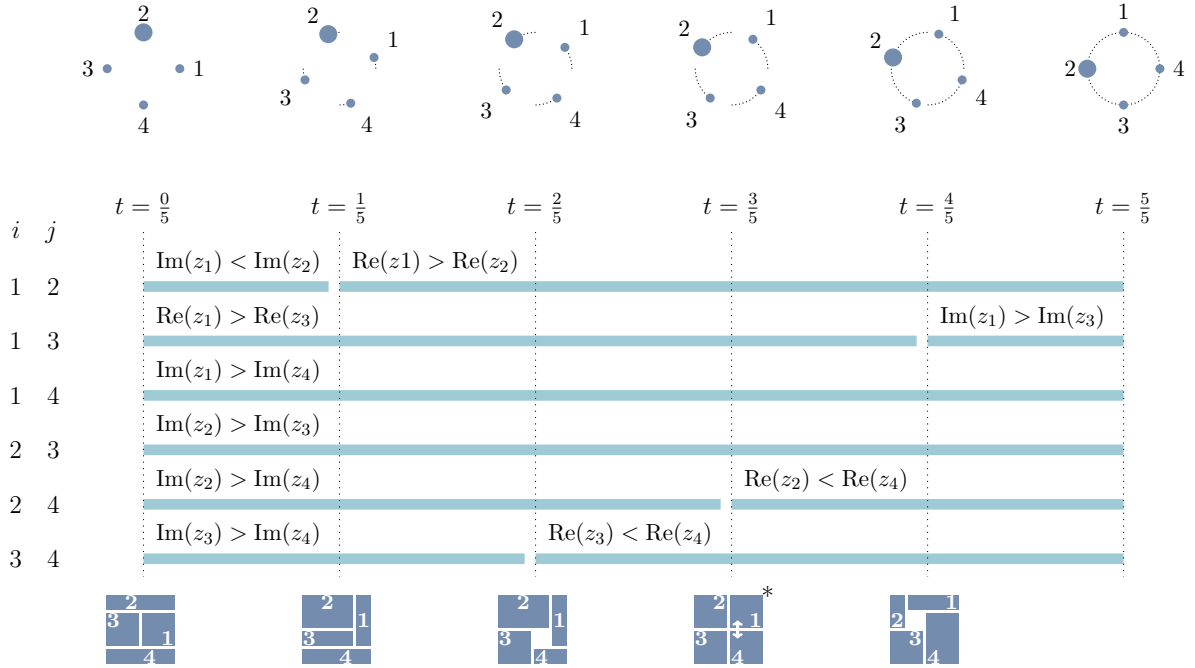


Figure 4.4: Illustration of **Cover** (Algorithm 12) for four points moving along a circle, with position known only approximately (as represented by the blue discs). The bars represent the lifetime of an edge in the graph representing the current arrangement. The diagrams in the last row represent the resulting cover. The arrangement cell marked with $*$ is not representable as a configuration of boxes, it is defined by

$$\{\max(\text{Re}(z_2), \text{Re}(z_3)) < \min(\text{Re}(z_1), \text{Re}(z_4)), \text{Im}(z_3) < \text{Im}(z_2) \text{ and } \text{Im}(z_4) < \text{Im}(z_1)\}.$$



Figure 4.5: All four combinatorial arrangements of two axis-aligned disjoint rectangles in the plane. Two configurations are equivalent if, for every pair of rectangles, the same relative position holds in both configurations (namely, whether one lies strictly to the left of the other or strictly above the other).

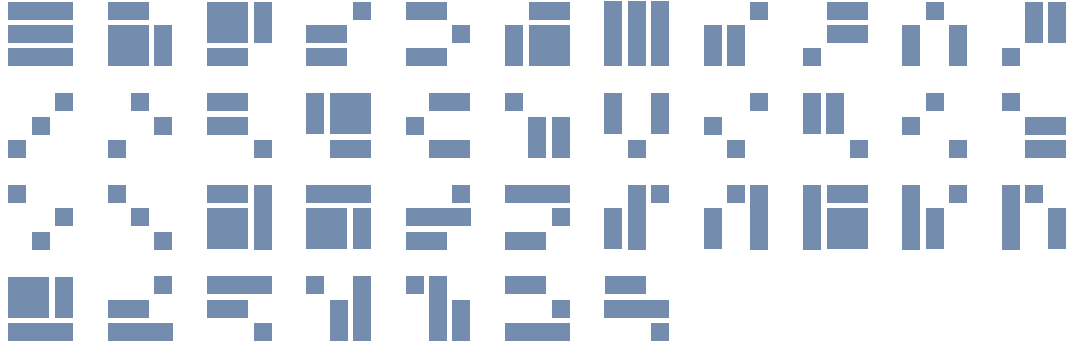


Figure 4.6: All 40 combinatorial arrangements of three axis-aligned disjoint rectangles in the plane.

Combinatorics of arrangements The combinatorics of points in an exact setting is simpler than its approximate counterpart. Fox and Neuwirth [38] described a CW-complex structure on OC_n , whose cells give a combinatorial description of configurations. For example, with three points, the ordered configuration space OC_3 admits the partition formed by the following cells, up to permutation of z_1, z_2 and z_3 :

$$\begin{aligned} &\{\operatorname{Re}(z_1) < \operatorname{Re}(z_2) < \operatorname{Re}(z_3)\}, \\ &\{\operatorname{Re}(z_1) < \operatorname{Re}(z_2) = \operatorname{Re}(z_3) \text{ and } \operatorname{Im}(z_2) < \operatorname{Im}(z_3)\}, \\ &\{\operatorname{Re}(z_1) = \operatorname{Re}(z_2) < \operatorname{Re}(z_3) \text{ and } \operatorname{Im}(z_1) < \operatorname{Im}(z_2)\}, \\ &\{\operatorname{Re}(z_1) = \operatorname{Re}(z_2) = \operatorname{Re}(z_3) \text{ and } \operatorname{Im}(z_1) < \operatorname{Im}(z_2) < \operatorname{Im}(z_3)\}. \end{aligned}$$

Yet, determining a Fox–Neuwirth cell containing a triple (z_1, z_2, z_3) from a numerical approximation, even a certified one, may be impossible because equality cannot be decided. This motivates using arrangement cells instead, which are open in OC_n but lead to much richer combinatorics.

The combinatorial diversity of arrangement cells can be pictured by those coming from arrangements of disjoint axis-aligned rectangles (or *boxes*) in the plane, as they arise in interval arithmetic. In this context, a configuration $(z_1, \dots, z_n) \in OC_n$ is described by an interval for each coordinate $\operatorname{Re}(z_1)$, $\operatorname{Im}(z_1)$, $\operatorname{Re}(z_2)$, etc., so each z_i lies in a box, and we assume that the boxes are pairwise disjoint. Two boxes can be separated by a vertical line, or a horizontal line. So a tuple of disjoint boxes induces an arrangement. Figures 4.5, 4.6, and 4.7 display all the arrangements that can be obtained in this way, with 2, 3 and 4 points, respectively.



Figure 4.7: All 772 combinatorial configurations of four disjoint rectangles in the plane.

4.3 Computing braids from arrangements

A sequence of arrangement cells covering a path $F : [0, 1] \rightarrow OC_n$ entirely determines the homotopy type of F (Lemma 4.2), and **Cover** (Algorithm 12) computes such a sequence. This section focuses on the next step: computing the braid induced by F .

4.3.1 Permutation points and elementary braids

Permutation points To compute a braid from a sequence of arrangements covering a path, we use a piecewise linear path whose vertices are permutation points. Given two permutations π and ϕ of $[n]$, the *permutation point* $p(\pi, \phi)$ is the point in OC_n defined by

$$p(\pi, \phi) = (\pi(1) + \phi(1)\mathbf{i}, \dots, \pi(n) + \phi(n)\mathbf{i}).$$

Note that $\sigma \cdot p(\pi, \phi) = p(\pi\sigma^{-1}, \phi\sigma^{-1})$ for any permutation $\sigma \in S_n$.

Lemma 4.4. *Any arrangement cell contains a permutation point.*

Proof. Same argument as the nonemptiness proof in Lemma 4.1. $\square$

We also consider the point $q(\pi) = (\pi(1), \dots, \pi(n)) \in OC_n$. The collection of these points forms the fiber of the projection $OC_n \rightarrow C_n$ above the base point $[n] \in C_n$. A permutation point $p(\pi, \phi)$ is connected to $q(\pi)$ by a straight-line path in OC_n . Given a path P between two permutation points $p(\pi, \phi)$ and $p(\pi', \phi')$, we can extend P as a path P' from $q(\pi)$ to $q(\pi')$:

$$q(\pi) \rightarrow p(\pi, \phi) \xrightarrow{P} p(\pi', \phi') \rightarrow q(\pi').$$

This denotes the path that starts from $q(\pi)$, goes to $p(\pi, \phi)$ in a straight line, follows P to reach $p(\pi', \phi')$, and then goes to $q(\pi')$ in a straight line. The projection of P' onto C_n induces a loop based at $[n]$; therefore, its homotopy class is described by a combinatorial braid $b \in B_n$. Conversely, a braid $b \in B_n$ is the homotopy class of a loop in C_n . We may lift this loop to a path P' in OC_n starting from $q(\pi)$ and ending at another point of the fiber over $[n]$, say $q(\pi')$. Given two permutations ϕ and ϕ' , we may form the extended path

$$p(\pi, \phi) \rightarrow q(\pi) \xrightarrow{P'} q(\pi') \rightarrow p(\pi', \phi'),$$

which is well defined up to homotopy. We denote this path $p(\pi, \phi) \xrightarrow{b} p(\pi', \phi')$. These two constructions, from the homotopy class of P to b , and from b , π , ϕ and ϕ' to the homotopy class of P , are inverse to one another. Moreover, they are compatible with composition:

$$p(\pi, \phi) \xrightarrow{b} p(\pi', \phi') \xrightarrow{b'} p(\pi'', \phi'') \equiv_{\text{homotopy}} p(\pi, \phi) \xrightarrow{b \cdot b'} p(\pi'', \phi'').$$

Elementary braids Given two permutations π, ϕ , the piecewise-linear path $q(\pi) \rightarrow p(\pi, \phi) \rightarrow p(\pi, \text{id}) \rightarrow q(\pi)$ lies in OC_n and induces the trivial braid: since the real parts remain constant, the path is contractible. However, the linear path $p(\pi, \phi) \rightarrow p(\text{id}, \phi)$, which also lies in OC_n , may induce a nontrivial braid. Let $B(\pi, \phi)$ denote its associated braid, called an *elementary braid*. In other words,

$$p(\pi, \phi) \xrightarrow{\text{straight line}} p(\text{id}, \phi) \equiv_{\text{homotopy}} p(\pi, \phi) \xrightarrow{B(\pi, \phi)} p(\text{id}, \phi).$$

Lemma 4.5. *Let A be an arrangement and let π, ϕ, π' and ϕ' be permutations of $[n]$ such that the permutation points $p(\pi, \phi)$ and $p(\pi', \phi')$ lie in A . Then the straight-line path $p(\pi, \phi) \rightarrow p(\pi', \phi')$ is contained in OC_n and its associated braid is $B(\pi\pi'^{-1}, \phi\phi'^{-1})$.*

Proof. By definition of an arrangement cell, if both $p(\pi, \phi)$ and $p(\pi', \phi')$ are in A , then so is $p(\pi', \phi)$. So the path $p(\pi, \phi) \rightarrow p(\pi', \phi)$ is homotopic to

$$p(\pi, \phi) \rightarrow p(\pi', \phi) \rightarrow p(\pi', \phi'),$$

by convexity of A . Under the action of π' , which does not change the associated braid, this path becomes

$$p(\pi\pi'^{-1}, \phi\pi'^{-1}) \rightarrow p(\text{id}, \phi\pi'^{-1}) \rightarrow p(\text{id}, \phi'\pi'^{-1}).$$

The first part of the path induces the braid $B(\pi\pi'^{-1}, \phi\pi'^{-1})$, by definition, while the second part induces the trivial braid, since the real parts remain fixed. $\square$

The following statement gives identities to compute $B(\pi, \phi)$ in terms of a decomposition of π as a product of elementary transpositions $(i \ i+1)$.

Lemma 4.6. *For any permutations π, π' and ϕ :*

1. $B(\text{id}, \phi) = 1$;
2. $B(\pi'\pi, \phi) = B(\pi', \phi\pi^{-1})B(\pi, \phi)$;
3. $B((i \ i+1), \phi) = \sigma_i$ if $\phi(i) > \phi(i+1)$, for any $1 \leq i < n$;
4. $B((i \ i+1), \phi) = \sigma_i^{-1}$ if $\phi(i) < \phi(i+1)$, for any $1 \leq i < n$.

Proof. The first point is trivial since the linear path $p(\text{id}, \phi) \rightarrow p(\text{id}, \phi)$ is constant.

For the second point, consider the piecewise straight-line path P

$$p(\pi'\pi, \phi) \rightarrow p(\pi, \phi) \rightarrow p(\text{id}, \phi),$$

and the arrangement $A = (\emptyset, \prec_\phi)$ where $\emptyset$ is the empty partial order and where $\prec_\phi$ is the total order defined by $i \prec_\phi j$ when $\phi(i) < \phi(j)$. The path P entirely lies

in the cell attached to this arrangement. The convexity of this cell (Lemma 4.1) implies that P is homotopic to the straight-line path $p(\pi'\pi, \phi) \rightarrow p(\text{id}, \phi)$, which induces the braid $B(\pi'\pi, \phi)$, by definition.

On the other hand, the part $p(\pi, \phi) \rightarrow p(\text{id}, \phi)$ induces the braid $B(\pi, \phi)$, by definition, and the part $p(\pi'\pi, \phi) \rightarrow p(\pi, \phi)$ induces the same braid as the path

$$p(\pi', \phi\pi^{-1}) \rightarrow p(\text{id}, \phi\pi^{-1}),$$

by invariance under the action of the permutation group. This latter path induces the braid $B(\pi', \phi\pi^{-1})$, by definition. This gives the identity $B(\pi'\pi, \phi) = B(\pi', \phi\pi^{-1})B(\pi, \phi)$.

For the third point, let τ denote the transposition $(i \ i+1)$, and consider the path P

$$q(\tau) \rightarrow p(\tau, \phi) \rightarrow p(\text{id}, \phi) \rightarrow q(\text{id}).$$

It is covered by three arrangement cells: $(\prec_\tau, \emptyset)$, A , and $(\prec_{\text{id}}, \emptyset)$, where A is the arrangement $(\prec_{\text{Re}}, \prec_{\text{Im}})$ with $\prec_{\text{Re}} = (\prec_\tau \cap \prec_{\text{id}})$ and $\prec_{\text{Im}}$ contains the only relation $i+1 \prec_{\text{Im}} i$. We check easily that these three arrangements also cover the path defining σ_i (Figure 4.1). The last point is similar. $\square$

4.3.2 Braids from sequences of arrangements

Algorithm The procedure **Braid** (Algorithm 13) takes a sequence of arrangements $A_1, \dots, A_r$ as input, with $A_{i-1} \cap A_i$ nonempty. It computes a sequence of permutation points $p_1, \dots, p_r$ such that $p_1 \in A_1$ and $p_i \in A_{i-1} \cap A_i$, for $i \geq 2$. Simultaneously, it computes the braid b induced by the piecewise linear path $p_1 \rightarrow \dots \rightarrow p_r$ as a composition of elementary braids $B(\pi, \phi)$ (Section 4.3.1). Finally, it outputs the triple (p_1, b, p_r) .

Figure 4.8 shows an example, continuing that of Figure 4.4. In this case, the algorithm computes (among several possible choices) the following permutation points:

$$\begin{aligned} p_1 &= p([4, 3, 1, 2], [3, 4, 2, 1]) & p_2 &= p([4, 2, 1, 3], [3, 4, 2, 1]) \\ p_3 &= p_2 & p_4 &= p([4, 1, 2, 3], [4, 3, 2, 1]), \end{aligned}$$

and outputs the triple $(p_1, \sigma_2\sigma_1, p_4)$. For permutations, we use the notation introduced in Section 3.1.1: for example, $[4, 3, 1, 2]$ denotes the permutation defined by $1 \mapsto 4, 2 \mapsto 3, 3 \mapsto 1$, and $4 \mapsto 2$.

Analysis

Theorem 4.7 (Correctness of **Braid**). *Let $(A_1, \dots, A_r)$ be a sequence of arrangements covering a path $F : [0, 1] \rightarrow OC_n$. Let (p, b, p') be the output triple of **Braid** (Algorithm 13) on input $A_1, \dots, A_r$. Then:*

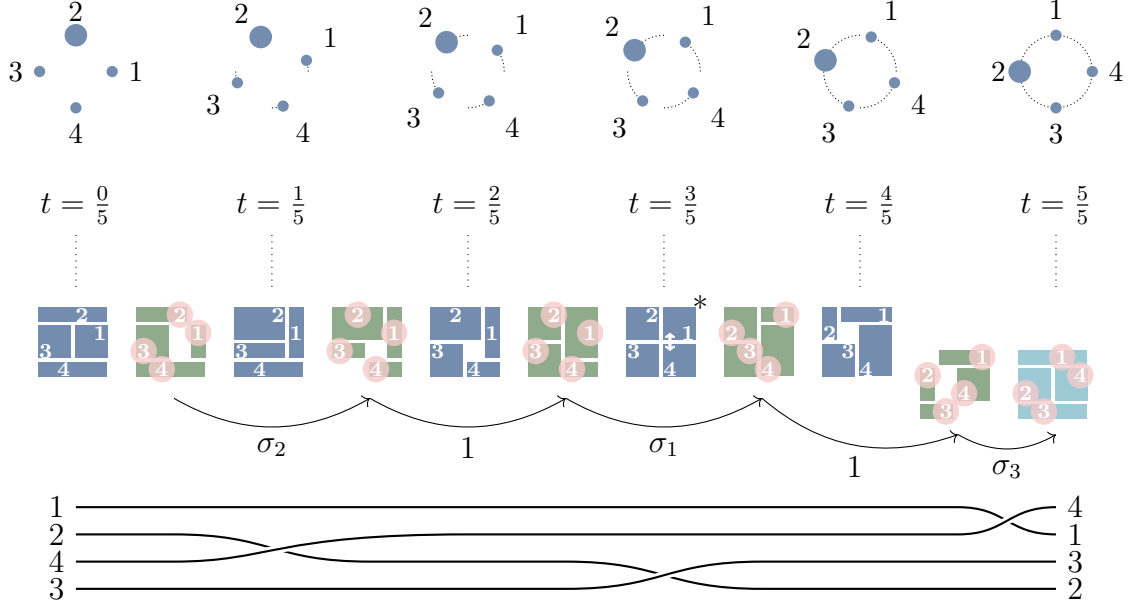


Figure 4.8: Illustration of Algorithm 13 for four points moving along a circle, continuing Figure 4.4. The arrangement diagrams in blue are the input covering sequence. The arrangement diagrams in green represent the intersection of the two neighboring blue diagrams. The pink circled digits highlight a permutation point lying in the arrangement cell. The last blue diagram represents an extension of the cover so that the last arrangement is a permutation of the first one (see Section 4.3.4).

- $F(0)$ and p are in A_1 ;
- $F(1)$ and p' are in A_r ;
- the path $F(0) \rightarrow p \xrightarrow{b} p' \rightarrow F(1)$ lies in OC_n and is homotopic to F .

Proof. By definition of a cover, the intersections $A_{i-1} \cap A_i$ are nonempty and therefore each contains a permutation point, by Lemma 4.1 and Lemma 4.4. So the algorithm does not fail. The first two points follow by construction.

Let $p_1 = p(\pi_{\text{start}}, \phi_{\text{start}})$ and let p_i be the permutation point chosen in $A_{i-1} \cap A_i$. It is clear that the piecewise linear path $p_1 \rightarrow \dots \rightarrow p_r$ is also covered by $A_1, \dots, A_r$. Its associated braid is b , by repeated application of Lemma 4.5. Since $F(0) \in A_1$ and $F(1) \in A_r$, the piecewise linear path

$$F(0) \rightarrow p_1 \rightarrow \dots \rightarrow p_r \rightarrow F(1)$$

is also covered by the same sequence. By Lemma 4.2, this path is homotopic to F . $\square$

Input. A sequence of arrangements.

Output. A triple (p, b, p') where p and p' are permutation points and b is a braid.

```

1  def Braid( $A_1, \dots, A_r$ ):
2       $(\pi_{\text{start}}, \phi_{\text{start}}) \leftarrow$  a permutation point in  $A_1$ 
3       $(\pi, \phi) \leftarrow (\pi_{\text{start}}, \phi_{\text{start}})$ 
4       $b \leftarrow 1$  //the identity braid
5      for  $i$  from 2 to  $r$ :
6          if  $A_{i-1} \cap A_i = \emptyset$ :
7              error
8           $(\pi', \phi') \leftarrow$  a permutation point in  $A_{i-1} \cap A_i$ 
9           $b \leftarrow b \cdot B(\pi\pi'^{-1}, \phi\phi'^{-1})$ 
10          $(\pi, \phi) \leftarrow (\pi', \phi')$ 
11     return  $p(\pi_{\text{start}}, \phi_{\text{start}}), b, p(\pi, \phi)$ 

```

Algorithm 13: Computation of a braid associated to an arrangement sequence.

4.3.3 Composition of paths

Given two paths $F, G : [0, 1] \rightarrow OC_n$ with $F(1) = G(0)$, the goal is to compute the braid attached to the concatenation $F \cdot G$ using the algorithms above. Applying **Cover** (Algorithm 12) to F and G yields covering sequences of arrangements $(A_1, \dots, A_r)$ and $(B_1, \dots, B_s)$. Their concatenation

$$A_1, \dots, A_r, B_1, \dots, B_s$$

covers $F \cdot G$, so Algorithm 13 directly produces the desired braid. However, storing only the outputs of **Braid** $(A_1, \dots, A_r)$ and **Braid** $(B_1, \dots, B_s)$ is preferable; let us denote them

$$((\pi_F, \phi_F), b_F, (\pi'_F, \phi'_F)) \text{ and } ((\pi_G, \phi_G), b_G, (\pi'_G, \phi'_G)).$$

When an arrangement cell A contains both $p(\pi'_F, \phi'_F)$ and $p(\pi_G, \phi_G)$ together with the common endpoint $F(1) = G(0)$, the path composition

$$p(\pi_F, \phi_F) \xrightarrow{b_F} p(\pi'_F, \phi'_F) \rightarrow F(1) = G(0) \rightarrow p(\pi_G, \phi_G) \xrightarrow{b_G} p(\pi'_G, \phi'_G) \quad (4.1)$$

is homotopic, by convexity of A , to

$$p(\pi_F, \phi_F) \xrightarrow{b_F} p(\pi'_F, \phi'_F) \rightarrow p(\pi_G, \phi_G) \xrightarrow{b_G} p(\pi'_G, \phi'_G),$$

whose braid equals

$$b_F \cdot B(\pi'_F \pi_G^{-1}, \phi'_F \pi_G^{-1}) \cdot b_G.$$

The equality $F(1) = G(0)$ alone, however, does not ensure that $p(\pi'_F, \phi'_F)$ and $p(\pi_G, \phi_G)$ lie in a common arrangement cell; the straight-line segment joining them may even exit OC_n . We suggest two approaches to circumvent the issue:

1. When future compositions are known in advance, append B_1 to $(A_1, \dots, A_r)$. The augmented sequence still covers F , and this will guarantee that $F(1)$, $p(\pi'_F, \phi'_F)$ and $p(\pi_G, \phi_G)$ lie in the common cell B_1 .
2. Retain the endpoint cells A_r and B_1 and compute their intersection, which is nonempty because it contains $F(1) = G(0)$. Choose $p(\pi, \phi)$ a permutation point in $A_r \cap B_1$; then the path (4.1) is homotopic to

$$p(\pi_F, \phi_F) \xrightarrow{b_F} p(\pi'_F, \phi'_F) \rightarrow p(\pi, \phi) \rightarrow p(\pi_G, \phi_G) \xrightarrow{b_G} p(\pi'_G, \phi'_G)$$

with associated braid

$$b_F \cdot B(\pi'_F \pi^{-1}, \phi'_F \pi^{-1}) \cdot B(\pi \pi_G^{-1}, \phi \pi_G^{-1}) \cdot b_G.$$

Next, we relax the condition $F(1) = G(0)$ and only require equality in C_n , namely, there is a permutation σ such that $\sigma \cdot F(1) = G(0)$. We consider the paths $\tilde{F}$ and $\tilde{G}$ in C_n induced by F and G respectively, so that $\tilde{F}(1) = \tilde{G}(0)$. How do we compute the braid induced by the composition $\tilde{F} \cdot \tilde{G}$ from the braid data computed for F and G ? If the permutation σ is known, then the situation reduces to the previous case $F(1) = G(0)$. Unfortunately, the separation method **Sep** is not enough to compute σ . For example, on two points, assume that $F(1) = \sigma \cdot G(0)$ for some σ (which is either identity or the transposition $(1\ 2)$), and that

$$\text{Sep}(F, 1, 2, 1) = (2, 1, \text{Re}, [\dots]) \text{ and } \text{Sep}(G, 1, 2, 0) = (1, 2, \text{Im}, [\dots]),$$

meaning that $\text{Re}(F_1(1)) > \text{Re}(F_2(1))$ and $\text{Im}(G_1(0)) < \text{Im}(G_2(0))$. Is this enough information to recover σ ? The answer is no: we could have $\sigma = \text{id}$ with $F(1) = G(0) = (1, \mathbf{i})$, or $\sigma = (1\ 2)$ with $F(1) = (1 + \mathbf{i}, 0)$ and $G(0) = (0, 1 + \mathbf{i})$. Therefore, we need to compute σ by other means, such as path tracking.

4.3.4 Braids along loops

Consider a path $F : [0, 1] \rightarrow OC_n$ inducing a loop in C_n . In other words, there exists a permutation σ with $F(0) = \sigma \cdot F(1)$. We seek a permutation point $p(\pi, \phi)$ and a braid b such that

$$F(0) \rightarrow p(\pi, \phi) \xrightarrow{b} \sigma^{-1} \cdot p(\pi, \phi) \rightarrow F(1)$$

is homotopic to F .

This problem is similar to the composition problem above. In particular, we must determine σ by other means, since the **Sep** method alone does not provide sufficient information to recover it.

Once σ is known, we compute a cover $A_1, \dots, A_r$ of F , which we extend to

$$A_1, \dots, A_r, \sigma^{-1}A_1.$$

Because $F(0) = \sigma \cdot F(1)$ and $F(0) \in A_1$, the extended sequence still covers F . The method **Braid**($A_1, \dots, A_r, \sigma^{-1}A_1$) outputs a triple (π, ϕ) , b , and (π', ϕ') such that

$$F(0) \rightarrow p(\pi, \phi) \xrightarrow{b} p(\pi', \phi') \rightarrow F(1)$$

is homotopic to F . Moreover, $F(0) = \sigma \cdot F(1) \in A_1$, $p(\pi, \phi) \in A_1$ and $p(\pi', \phi') \in \sigma^{-1}A_1$. Hence the path above is homotopic to

$$F(0) \rightarrow p(\pi, \phi) \xrightarrow{b} p(\pi', \phi') \rightarrow \sigma^{-1} \cdot p(\pi, \phi) \rightarrow F(1),$$

by convexity of $\sigma^{-1}A_1$, and it simplifies to

$$F(0) \rightarrow p(\pi, \phi) \xrightarrow{\tilde{b}} \sigma^{-1} \cdot p(\pi, \phi) \rightarrow F(1),$$

with $\tilde{b} = b \cdot B(\pi' \sigma^{-1} \pi^{-1}, \phi' \sigma^{-1} \pi^{-1})$, by Lemma 4.5.

Assume we have already computed **Braid**($A_1, \dots, A_r$) and retained A_1 and A_r . Analogously to the path composition problem, we can compute a permutation point in the intersection $A_r \cap \sigma^{-1}A_1$ and use it to close the loop (see Figure 4.8).

4.4 Implementation

The algorithm presented in the previous sections to compute the braid induced by a path $F : [0, 1] \rightarrow OC_n$ works in two steps: first compute an arrangement sequence covering F using **Cover** (Algorithm 12); then find a permutation point in the intersection of each pair of consecutive arrangements in the sequence and compute (on the fly) the braid associated to the straight-line interpolation of those permutation points using **Braid** (Algorithm 13).

This section revisits **Cover** and **Braid** from an implementation point of view. At Line 13 of Algorithm 12, the working arrangement is copied, thus creating a list of arrangements later processed by Algorithm 13. To avoid those copies and simplify implementation, **Cover** and **Braid** are merged: a current arrangement A and a permutation point p in A are maintained, and whenever A is updated, p is updated so that it lies in both the old A and the new A . Explicit data structures

for arrangements and permutation points are given, and this update is described in detail (Section 4.4.2). Finally, the loop at Line 8 of Algorithm 12, which needs to find incomparable pairs (i, j) after the deletion of an edge, is revisited (Section 4.4.3). All in all, this leads to simple algorithms that can be implemented with basic data structures.

4.4.1 Data structure for pointed arrangements

Representing arrangements Recall that an arrangement is a pair $(\prec_{\text{Re}}, \prec_{\text{Im}})$ of partial orders on $[n]$ such that all $i \neq j$ are comparable in either $\prec_{\text{Re}}$ or $\prec_{\text{Im}}$ (see Section 4.2.2).

A partial order $\prec$ on $[n]$ is encoded by a directed acyclic graph (DAG) G with vertex set $[n]$, where $i \prec j$ holds if and only if there is a directed path from i to j in G . An arrangement $(\prec_{\text{Re}}, \prec_{\text{Im}})$ is encoded by a single graph G whose edges are labeled by Re or Im, so its edge set is a subset of $[n]^2 \times \{\text{Re}, \text{Im}\}$. An output (i, j, q, t) of **Sep** corresponds to an edge (i, j, q) with lifetime t . Let G_{Re} (resp. G_{Im}) denote the subgraph of G with edges labelled by Re (resp. Im), and write $i \rightarrow_{\text{Re}} j$ (resp. $i \rightarrow_{\text{Im}} j$) when (i, j) is an edge of G_{Re} (resp. G_{Im}). By metonymy, such a graph representing $(\prec_{\text{Re}}, \prec_{\text{Im}})$ is called an *arrangement* if $(\prec_{\text{Re}}, \prec_{\text{Im}})$ is an arrangement.

Pointed arrangement Let $(\prec_{\text{Re}}, \prec_{\text{Im}})$ be the pair of partial orders associated to G and let A be the associated cell, as defined in Section 4.2.2. Let $\pi, \phi \in S_n$. A permutation point $p(\pi, \phi)$ is in A if and only if π increases along the edges of G_{Re} and ϕ increases along the edges of G_{Im} ; that is, for all $i, j \in [n]$,

$$i \rightarrow_{\text{Re}} j \Rightarrow \pi(i) < \pi(j) \quad \text{and} \quad i \rightarrow_{\text{Im}} j \Rightarrow \phi(i) < \phi(j). \quad (4.2)$$

We write $G \vdash p(\pi, \phi)$ if this condition holds. In other words, identifying permutations of $[n]$ with total orders on $[n]$, $G \vdash p(\pi, \phi)$ if and only if the permutations π and ϕ are topological sorts of G_{Re} and G_{Im} respectively.

When computing a braid, we will maintain an arrangement with a permutation point lying in its associated cell; that is a *pointed arrangement*.

4.4.2 Pointed arrangement updates

Edge insertion The two operations on pointed arrangements are the deletion and the insertion of an edge in G , while maintaining the condition $G \vdash p(\pi, \phi)$. Interpreting the latter condition as π and ϕ being topological orders on G_{Re} and G_{Im} , this is the *dynamic topological sort* problem. If G' is G with an edge deleted, then $G' \vdash p(\pi, \phi)$ provided that $G \vdash p(\pi, \phi)$, so π and ϕ can remain unchanged

when an edge is deleted from G . On the other hand, inserting an edge in G may require updates to π and ϕ : this is what **Insert** does (Algorithm 14). Because G does not change entirely, updating π and ϕ is preferable to recomputing them from scratch. The algorithm of Marchetti-Spaccamela, Nanni, and Rohnert [63] [see also 71] is adapted for this purpose (Figure 4.9). It modifies G, π and ϕ *in-place*, so they are global variables in the pseudocode.

Analysis We consider a run of **Insert** (Algorithm 14). Let G', π', ϕ' be the state of the pointed arrangement before the run of **Insert**, and let $(i, j, q) \in [n]^2 \times \{\text{Re}, \text{Im}\}$ be a valid edge for G' , meaning that inserting (i, j, q) in G' does not introduce any cycle in G'_{Re} nor G'_{Im} . Let G, π, ϕ be the state of the arrangement after running **Insert** (i, j, q) , and b be the output.

Proposition 4.8. *Insert (Algorithm 14) is correct, assuming that the initial graph G' is an arrangement: with the notation above, $G = G' \cup \{(i, j, q)\}$, $G \vdash p(\pi, \phi)$, and b is the braid induced by the well-defined piecewise straight-line path $p(\pi', \phi') \rightarrow p(\pi, \phi)$.*

Proof. When $q = \text{Im}$, the procedure **Insert** calls **UpdateImag**, which updates the topological sort ϕ to be compatible with the new edge and returns the identity braid. Indeed, the path $p(\pi', \phi') \rightarrow p(\pi, \phi)$ induces the identity braid because $\pi = \pi'$.

When $q = \text{Re}$, the procedure **Insert** calls **UpdateReal**, which performs two tasks simultaneously. First, it updates the topological sort π to be compatible with the new edge, following the strategy of Marchetti-Spaccamela, Nanni, and Rohnert [63] (see Figure 4.9) by left-multiplying π with suitable elementary transpositions (Line 13). Second, it computes the braid induced by $p(\pi', \phi) \rightarrow p(\pi, \phi)$, where π' is the initial value of π .

We claim that, at the end of the procedure, b is the braid induced by $p(\pi', \phi) \rightarrow p(\pi, \phi)$. By construction of the topological sorting algorithm, $G' \vdash p(\pi, \phi)$ throughout the procedure, so the path $p(\pi', \phi) \rightarrow p(\pi, \phi)$ lies in the arrangement cell defined by G' , and Lemma 4.5 applies. The path induces the braid $B(\pi' \pi^{-1}, \phi \pi^{-1})$. The permutation $\pi' \pi^{-1}$ decomposes as a product of elementary transpositions, accumulated on Line 13, and the braid is computed using Lemma 4.6: each transposition $(y-1 \ y)$ contributes a factor σ_{y-1} or σ_{y-1}^{-1} to b , which is exactly what is computed on Line 8. $\square$

4.4.3 Braid computation implementation

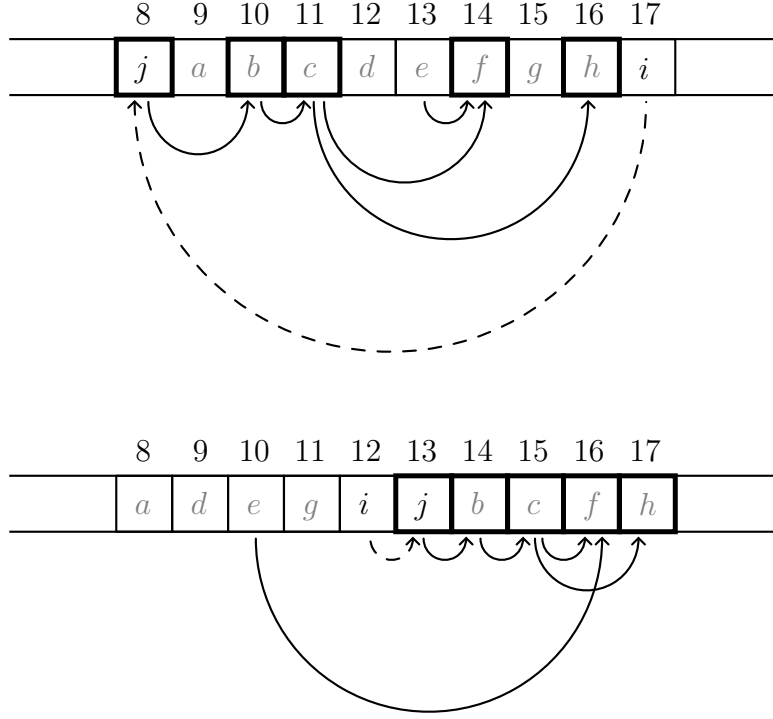


Figure 4.9: Illustration of $\text{UpdateReal}(i, j)$. We draw π^{-1} as a horizontal array mapping indices to vertices (for instance, $\pi(j) = 8$), and draw the Re-labeled edges in G . The topological sort is correct if all edges are pointing to the right. The dashed arrow from i to j represents the edge to be inserted. Using the variable names from UpdateReal (Algorithm 14), $m = 8$, $M = 17$, and δ is the set of vertices corresponding to bold squares. Vertex i cannot be in δ , as it is assumed that the new edge does not introduce a cycle. The vertices in the interval $[m, M]$ are reordered, pushing the elements of δ to the right and the others to the left. Once done, the dashed arrow from i to j is pointing to the right, since i is not in δ . The procedure always keeps existing edges pointing to the right. Although more efficient methods exist for this task [71], it is required to update π with elementary transpositions to be able to express the braid in standard generators.

```

1  def UpdateReal( $i, j$ ): //updates  $\pi$  after inserting  $(i, j, \text{Re})$  in  $G$ 
2       $m \leftarrow \pi(j)$ ;    $M \leftarrow \pi(i)$ ;    $c \leftarrow 1$ ;    $b \leftarrow 1$ 
3       $\delta \leftarrow \{k \in [n] : j \prec_{\text{Re}} k \text{ and } \pi(k) < M\}$  //Bounded dfs
4      for  $x$  from  $m + 1$  to  $M$ :
5          if  $\pi^{-1}(x) \in \delta$ :
6               $c \leftarrow c + 1$ ;   continue
7          for  $y$  from  $x$  to  $x - c + 1$ : //Reverse order iteration
8              if  $\phi(\pi^{-1}(y - 1)) < \phi(\pi^{-1}(y))$ :
9                   $b \leftarrow b \cdot \sigma_{y-1}$ 
10             else:
11                  $b \leftarrow b \cdot \sigma_{y-1}^{-1}$ 
12                  $\pi \leftarrow (y - 1 \ y)\pi$ 
13     return  $b$ 
14
15 def UpdateImag( $i, j$ ): //updates  $\phi$  after inserting  $(i, j, \text{Im})$  in  $G$ 
16     Update  $\phi$  with a dynamic topological sorting algorithm
17     return 1
18
19 def Insert( $i, j, q$ ):
20      $G \leftarrow G \cup \{(i, j, q)\}$ 
21     if  $q = \text{Re}$ :
22         return UpdateReal( $i, j$ )
23     else:
24         return UpdateImag( $i, j$ )

```

Algorithm 14: Edge insertion in a pointed arrangement.

```

1  def Braid'(F):
2       $G \leftarrow \{\}$ 
3       $lifetime \leftarrow \{\}$  //map edges of  $G$  to their lifetime
4       $b \leftarrow 1$ 
5      for  $i$  from 1 to  $n$ :
6          for  $j$  from  $i + 1$  to  $n$ :
7               $i', j', q, t \leftarrow \text{Sep}(F, i, j, 0)$ 
8               $G \leftarrow G \cup \{(i', j', q)\}$ 
9               $lifetime[i', j', q] \leftarrow t$ 
10          $\pi \leftarrow \text{TopologicalSort}(G_{\text{Re}}); \quad \phi \leftarrow \text{TopologicalSort}(G_{\text{Im}})$ 
11          $G_0, \pi_0, \phi_0 \leftarrow G, \pi, \phi$  //save initial pointed arrangement
12
13     loop:
14          $(i, j, q) \leftarrow$  edge of  $G$  with smallest lifetime
15          $t \leftarrow lifetime[i, j, q]$ 
16         if  $t \geq 1$ :
17             break
18         if there is no  $q$ -labeled path from  $i$  to  $j$  in  $G \setminus \{(i, j, q)\}$ :
19              $i', j', q', t' \leftarrow \text{Sep}(F, i, j, t)$ 
20             if  $q = q'$ : //necessarily,  $i = i'$  and  $j = j'$ 
21                  $lifetime[i, j, q] \leftarrow t'$ 
22                 continue
23             else:
24                 //insert  $(i', j', q')$  in  $G$  and update  $\pi$  and  $\phi$ .
25                  $b \leftarrow b \cdot \text{Insert}(i', j', q')$ 
26                  $lifetime[i', j', q'] \leftarrow t'$ 
27                 //insert additional edges to prepare deletion
28                 for  $k \in [n]$  s.t.  $k \rightarrow_q i$  and not  $k \rightarrow_q j$ :
29                      $G \leftarrow G \cup \{(k, j, q)\}$ 
30                      $lifetime[k, j, q] \leftarrow t$ 
31                 for  $k \in [n]$  s.t.  $j \rightarrow_q k$  and not  $i \rightarrow_q k$ :
32                      $G \leftarrow G \cup \{(i, k, q)\};$ 
33                      $lifetime[i, k, q] \leftarrow t$ 
34          $G \leftarrow G \setminus \{(i, j, q)\}$ 
35
36     return  $(G_0, \pi_0, \phi_0), b, (G, \pi, \phi)$ 

```

Algorithm 15: Braid computation algorithm, revised.

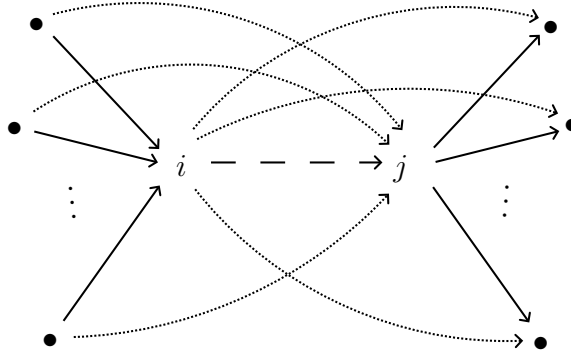


Figure 4.10: Procedure to prepare the deletion of an edge. We assume that the edge to be deleted is $i \rightarrow_{\text{Re}} j$, and we only represent Re-labeled edges. Initially, only the plain edges and the edge from i to j are present in G_{Re} . After inserting the dotted edges in G_{Re} , inserting an edge $i \rightarrow j$ or $j \rightarrow i$ in G_{Im} , and deleting the edge $i \rightarrow_{\text{Re}} j$, the graph G is still an arrangement.

Algorithm **Cover** and **Braid** (Algorithms 12 and 13) are merged into **Braid'** (Algorithm 15). It relies on **Insert** (Algorithm 14) to maintain a permutation point within the current arrangement and computes the braid on the fly.

The procedure **Braid'** first computes an initial arrangement G containing $F(0)$ and a permutation point in this arrangement. Each edge of G is associated with a lifetime. When all lifetimes are at least 1, the algorithm terminates.

Theorem 4.9 (Correctness of **Braid'**). *On input $F : [0, 1] \rightarrow OC_n$, given as an approximation, Algorithm 15 outputs a triple $((G_0, \pi_0, \phi_0), b, (G_1, \pi_1, \phi_1))$, such that*

- G_0 is an arrangement containing $p(\pi_0, \phi_0)$ and $F(0)$;
- G_1 is an arrangement containing $p(\pi_1, \phi_1)$ and $F(1)$;
- the path $F(0) \rightarrow p(\pi_0, \phi_0) \xrightarrow{b} p(\pi_1, \phi_1) \rightarrow F(1)$ lies in OC_n and is homotopic to F .

Basic invariant We show that the main loop maintains the following invariants: G is an arrangement and $G \vdash p(\pi, \phi)$.

At each iteration of the main loop, an edge in G with the smallest lifetime is picked for deletion, say (i, j, q) . Deleting the edge may break the invariant that G is an arrangement by creating incomparable pairs. If there is a q -labeled path from i to j in $G \setminus \{(i, j, q)\}$, then deleting the edge does not create incomparable

pairs. Otherwise, the procedure inserts new edges to make the deletion possible (the block following Line 18).

To achieve this, the procedure queries **Sep** (Line 19), which outputs a new edge (i', j', q') with a lifetime t' . If $q = q'$ (Line 20), then $i = i'$ and $j = j'$, since otherwise $i = j'$ and $j = i'$, which would mean that both $q(F_i(t)) < q(F_j(t))$ and $q(F_i(t)) > q(F_j(t))$ hold. So in this case, the procedure does not delete the edge but extends its lifetime to t' , and proceeds to the next iteration.

In the case where $q \neq q'$ (Line 23), the new edge is inserted, using the **Insert** procedure to keep π and ϕ updated and to compute the braid on the fly. But the expired edge (i, j, q) is not yet ready for deletion: $G \setminus \{(i, j, q)\}$ may still contain incomparable pairs. Assume that there is indeed one such pair $\{u, v\}$, which means that there is a Re- or Im-labeled path from u to v (or the other way) in G but not in $G \setminus \{(i, j, q)\}$. This cannot be $\{i, j\}$ since i and j are comparable thanks to the newly inserted edge (i', j', q') . To fix this, assume that there is a Re- or Im-labeled path from u to v in G and $v \neq j$ (the case $u \neq i$ is similar). This path does not exist in $G \setminus \{(i, j, q)\}$, so it is q -labeled and goes through the edge $i \rightarrow_q j$. Since $j \neq v$, there is some edge $j \rightarrow_q k$ following $i \rightarrow_q j$ in the path from u to v . The procedure precisely inserts all the edges $i \rightarrow_q k$ (whenever $j \rightarrow_q k$) and $k \rightarrow_q i$ (whenever $k \rightarrow_q j$), see Figure 4.10. Therefore, after Line 33, the segment $i \rightarrow_q j \rightarrow_q k$ in the path from u to v can be replaced by the edge $i \rightarrow_q k$, and we obtain a path from u to v in $G \setminus \{(i, j, q)\}$. Therefore, there are no incomparable pairs in $G \setminus \{(i, j, q)\}$. Note that the edges $i \rightarrow_q k$ and $k \rightarrow_q i$ can be safely inserted, with lifetime t , because they do not change the relation $\prec_q$.

Termination Let $\epsilon > 0$ be given by Condition (sep_ϵ) . At each iteration, the loop selects an edge with the smallest lifetime t . When **Sep** is called on this edge, the returned lifetime satisfies $t' \geq \min(1, t + \epsilon)$, so each call increases the lifetime of the corresponding relation by at least ϵ until it reaches 1. Since there are finitely many ordered pairs and labels, this can only happen finitely many times.

When inserting additional edges without lifetime increase (Lines 23–33), the number of paths in G_q going through the new edges (i, k, q) or (k, j, q) is strictly less than the number of paths going through (i, j, q) , which is deleted. This ensures that the main loop does not stall at a given t .

Correctness It remains to check that **Braid'** computes what we want. Let G_k , π_k , ϕ_k be the values of the corresponding variables at the end of the k th iteration. Considering the state of the variables just after Line 34, the basic invariant (Section 4.4.3) tells us that G_k contains $p(\pi_k, \phi_k)$. Considering the state just before Line 34, it tells us that G_{k-1} also contains $p(\pi_k, \phi_k)$. Moreover, since new edges are inserted following **Sep**, or using transitivity of partial orders (Lines 28–33), the

arrangement sequence (G_k) covers F .

By Proposition 4.8, at the k th iteration, $\text{Insert}(i', j', q')$ returns the braid induced by the path $p(\pi_{k-1}, \phi_{k-1}) \rightarrow p(\pi_k, \phi_k)$. The iterations where Insert is not called do not modify π and ϕ . Then, correctness follows in the same way as that of **Braid** (Algorithm 13, Theorem 4.7).

Arrangement initialization Let us comment on the computation of an initial arrangement G (Lines 5-11). For each distinct $i, j \in [n]$, we compute an edge between i and j using **Sep**, resulting in a quadratic cost in n . This ensures that G is an arrangement, though there may be redundant edges. Depending on the specific implementation of **Sep** and with additional conditions on $F(0)$, this initialization can be improved. For example, if **Sep** only returns Re-labeled edges at time 0, a way to initialize the arrangement is to sort the components of $F(0)$ by real part using **Sep**. The arrangement G for which G_{Im} is empty and G_{Re} contains edges $(i, i+1, \text{Re})$ for $i \in [n-1]$ will contain $F(0)$. This initialization process is quasi-linear in n , as opposed to the quadratic cost of the naive method. This strategy can be adapted to the case where **Sep** only returns Re-labeled edges most of the time. Moreover, we may use a priori information on $F(0)$ to directly compute an arrangement containing it, such as the components of $F(0)$ being lexicographically sorted.

4.4.4 Practical implementation of Sep

Our main motivation is to compute the braid defined by the displacement of the roots of a parametrized polynomial $f \in \mathbb{C}[t][x]$, when t moves along a continuous path $\gamma : [0, 1] \rightarrow \mathbb{C}$ that stays away from the discriminant locus of f . To compute a braid using the methods presented above, we need an approximation of the path in OC_n formed by the roots of $f(\gamma(t), -)$, in the sense of Definition 4.2.

Certified homotopy continuation software (such as the software described in [11, 42, 34]) takes f and γ as input and returns disjoint tubular neighborhoods around the root paths. We consider **Alpath**, as presented in Section 2.6. Although **Track** (Algorithm 6) returns only the endpoint of the solution path, we can also save intermediate computations to obtain a piecewise tubular neighborhood of the solution path, with each piece described by a Taylor model (see Section 2.4).

Let $F : [0, 1] \rightarrow OC_n$ be a path describing the roots of $f(\gamma(t), -)$ as t varies. Denote by $\square[0, 1]$ the closed subintervals of $[0, 1]$, and by $\square\mathbb{C}$ the set of pairs of real compact intervals (that is, boxes in the plane). For each $i \in [n]$, the output of **Alpath** induces an interval extension $\square F_i : \square[0, 1] \rightarrow \square\mathbb{C}$ of F_i . We can ensure that for all $t \in [0, 1]$ and distinct $i, j \in [n]$,

1. $\square F_i(\{t\}) \cap \square F_j(\{t\}) = \emptyset$;

t	G	π^{-1}	ϕ^{-1}	b	$F(t)$
0		$[3, 4, 2, 1]$	$[4, 3, 1, 2]$	1	
$\frac{1}{5}$		$[3, 4, 2, 1]$	$[4, 3, 1, 2]$	1	
$\frac{2}{5}$		$[3, 4, 2, 1]$	$[4, 3, 1, 2]$	1	
$\frac{3}{5}$		$[3, 2, 4, 1]$	$[4, 3, 1, 2]$	σ_2	
$\frac{4}{5}$		$[3, 2, 4, 1]$	$[4, 3, 1, 2]$	σ_2	
1		$[3, 2, 4, 1]$	$[3, 4, 2, 1]$	σ_2	
		$[2, 3, 1, 4]$	$[3, 2, 4, 1]$	$\sigma_2\sigma_1\sigma_3$	

Figure 4.11: Braid computation of the path described by Figure 4.4. t : time in $[0, 1]$. G : arrangement represented by a graph. Plain edges correspond to Re-labeled edges, dashed edges correspond to Im-labeled edges. π^{-1}, ϕ^{-1} : permutations on $[4]$ that represent a permutation point, given by the array of their evaluations. b : braid on 4 strands. F : state of the path F at time t . The upper half of the table records the values of these variables in **Braid'** (Algorithm 15) after initialization and at the end of each iteration. The column F suggests that $F(0) = \sigma \cdot F(1)$ with $\sigma = [2, 3, 4, 1]$. In the lower half, we close the loop by adding $\sigma^{-1}G_0$ to the arrangement sequence, as described in 4.3.4. Denote by π_0, ϕ_0 (resp. π_1, ϕ_1) the permutations at $t = 0$ (resp. $t = 1$). To wrap up the computation, we multiply b by $\sigma_1\sigma_3$, the braid induced by the straight-line path $p(\pi_1, \phi_1) \rightarrow \sigma^{-1} \cdot p(\pi_0, \phi_0)$ which can be computed using Lemma 4.6. The output $\sigma_2\sigma_1\sigma_3$ is the braid induced by F , in the sense of Theorem 4.9.

2. $\square F_i$ is continuous at $\{t\}$ for the Hausdorff topology.

The procedure **Sep** is implemented as follows. Let $i, j \in [n]$ be distinct, and let $t \in [0, 1]$. Set $\delta = 1 - t$, and then:

1. compute $X_i = \square F_i([t, t + \delta]) \subseteq \mathbb{C}$ and $X_j = \square F_j([t, t + \delta]) \subseteq \mathbb{C}$;
2.
 - if $\operatorname{Re} X_i < \operatorname{Re} X_j$, return $(i, j, \operatorname{Re}, t + \delta)$;
 - if $\operatorname{Re} X_i > \operatorname{Re} X_j$, return $(j, i, \operatorname{Re}, t + \delta)$;
 - if $\operatorname{Im} X_i < \operatorname{Im} X_j$, return $(i, j, \operatorname{Im}, t + \delta)$;
 - if $\operatorname{Im} X_i > \operatorname{Im} X_j$, return $(j, i, \operatorname{Im}, t + \delta)$;
3. else divide δ by 2 and go back to step 1.

Since $\square F_i(\{t\})$ and $\square F_j(\{t\})$ are disjoint, continuity of $\square F_i$ and $\square F_j$ at $\{t\}$ ensures that for small enough δ , $\square F_i([t, t + \delta])$ and $\square F_j([t, t + \delta])$ are disjoint. The latter being complex intervals, one of the conditions of step 2 must hold, so the procedure terminates. As for Definition 4.1, Items (sep_a) and (sep_c) are straightforward, while Item (sep_b) follows from compactness of $[0, 1]$ and Conditions 1 and 2.

4.4.5 Braids

Braids is a Rust implementation of **Braid'** (Algorithm 15) and is available at

<https://gitlab.inria.fr/aguillem/braids>.

It provides a command-line interface, **braids**, using **Algp** as the certified homotopy continuation backend. The method **Sep** is implemented as explained in Subsection 4.4.4.

Roots of unity Consider the polynomial $f = x^4 - t \in \mathbb{C}[t][x]$ and the piecewise-linear loop γ defined by the sequence of points $1, -1 + \mathbf{i}, -1 - \mathbf{i}$, and 1 . To compute the braid induced by the motion of the roots of f along γ , we run **braids** with the following input file:

```
{
  "system": ["x^4 - t"],
  "variables": ["x"],
  "parameters": ["t"],
  "path": [["1"], ["-1 + I"], ["-1 - I"], ["1"]],
  "fiber": [["-1"], ["-I"], ["I"], ["1"]]
}
```

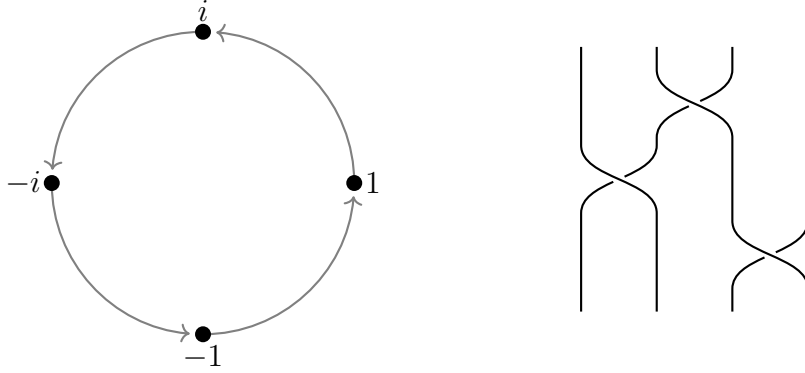


Figure 4.12: On the left, the displacement of the roots of $x^4 - t$ when t circles around 0 counterclockwise. On the right, a diagram representing the associated braid $\sigma_2\sigma_1\sigma_3$.

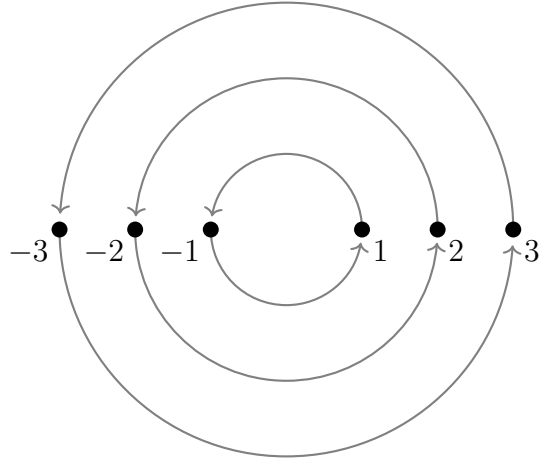


Figure 4.13: Displacement of the roots of $(x^2 - t)(x^2 - 4t)(x^2 - 9t)$ when t circles around 0 counterclockwise.

Note that compared to the input of Subsection 3.1.2, the fiber is reordered with respect to complex lexicographic order (sort by real part, then break ties using imaginary part). This is required for **Braids** to succeed as well as to correctly return the braid in the sense of Subsection 4.1.1. The command returns $\sigma_2\sigma_1\sigma_3$, whose associated diagram is depicted in Figure 4.12.

Non-generic crossing Consider the polynomial $f = (x^2 - t)(x^2 - 4t)(x^2 - 9t) \in \mathbb{C}[t][x]$ and the piecewise linear loop γ passing through 1 , $-1 + \mathbf{i}$, $-1 - \mathbf{i}$, and 1 , as shown in Figure 4.13.

The particularity of this example is that the strands are all aligned vertically when $t = -1$. In terms of braids diagram, this means that the six points cross

when projected onto the real axis. On input

```
{
  "system": ["(x^2 - t)*(x^2 - 4*t)*(x^2 - 9*t)"],
  "variables": ["x"],
  "parameters": ["t"],
  "path": [[ "1" ], [ "-1 + I" ], [ "-1 - I" ], [ "1" ]],
  "fiber": [[ "-3" ], [ "-2" ], [ "-1" ], [ "1" ], [ "2" ], [ "3" ]]
}
```

`braids` returns

$$\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5\sigma_4\sigma_3\sigma_2\sigma_1\sigma_2\sigma_3\sigma_4\sigma_3\sigma_2\sigma_3$$

which corresponds to the diagram of Figure 4.14.

On the complement of a plane curve In this last paragraph, we revisit a classical application of braid computations, namely computing the fundamental group of the complement of a plane algebraic curve. The approach dates back to Zariski [98] and van Kampen [94], and since then has been implemented in SIROCCO [64].

Let $f \in \mathbb{C}[t, x]$ be square-free with $d = \deg_x f \geq 1$, and let $\mathcal{C} = V(f)$. For simplicity, assume that f is monic seen as an element of $\mathbb{C}[t][x]$ and that $[0 : 1 : 0]$ does not belong to the projectivization of $\mathcal{C}$. Let $p : \mathcal{C} \rightarrow \mathbb{C}$ be the projection onto t . Let Σ be the zero set of $\text{disc}_x(f)$. For $t \in \mathbb{C} \setminus \Sigma$, the fiber of p consists of $d = \deg_x f$ distinct regular points, so the map

$$\begin{array}{ccc} F : \mathbb{C} \setminus \Sigma & \rightarrow & C_d \\ t & \mapsto & \{x \in \mathbb{C} : f(t, x) = 0\} \end{array}$$

is well defined. Let $b \in \mathbb{C} \setminus \Sigma$. F induces a map $\rho : \pi_1(\mathbb{C} \setminus \Sigma, b) \rightarrow \pi_1(C_d, F(b))$, which is called the *braid monodromy* associated to $\mathcal{C}$. The group $\pi_1(C_d, F(b))$ can be identified with B_d ; see the discussion on braids whose initial and final configurations differ from $[d]$ at the beginning of Subsection 4.1.1. Moreover, one can associate to $\beta \in B_d$ an automorphism of F_d (see Kassel and Turaev [53], for instance), called *the braid automorphism associated to β* . This association yields an isomorphism between B_d and the subgroup of $\text{Aut}(F_d)$ consisting of braid automorphisms. By abuse of notation, we also denote by ρ the extended map $\pi_1(\mathbb{C} \setminus \Sigma, b) \rightarrow \text{Aut}(F_d)$.

The braid monodromy map ρ can be used to recover a presentation of the fundamental group of $\mathbb{C}^2 \setminus \mathcal{C}$. Let $\gamma_1, \dots, \gamma_r$ generate $\pi_1(\mathbb{C} \setminus \Sigma, b)$. van Kampen [94] showed that $\pi_1(\mathbb{C}^2 \setminus \mathcal{C})$ has a presentation with generators $x_1, \dots, x_d$ and

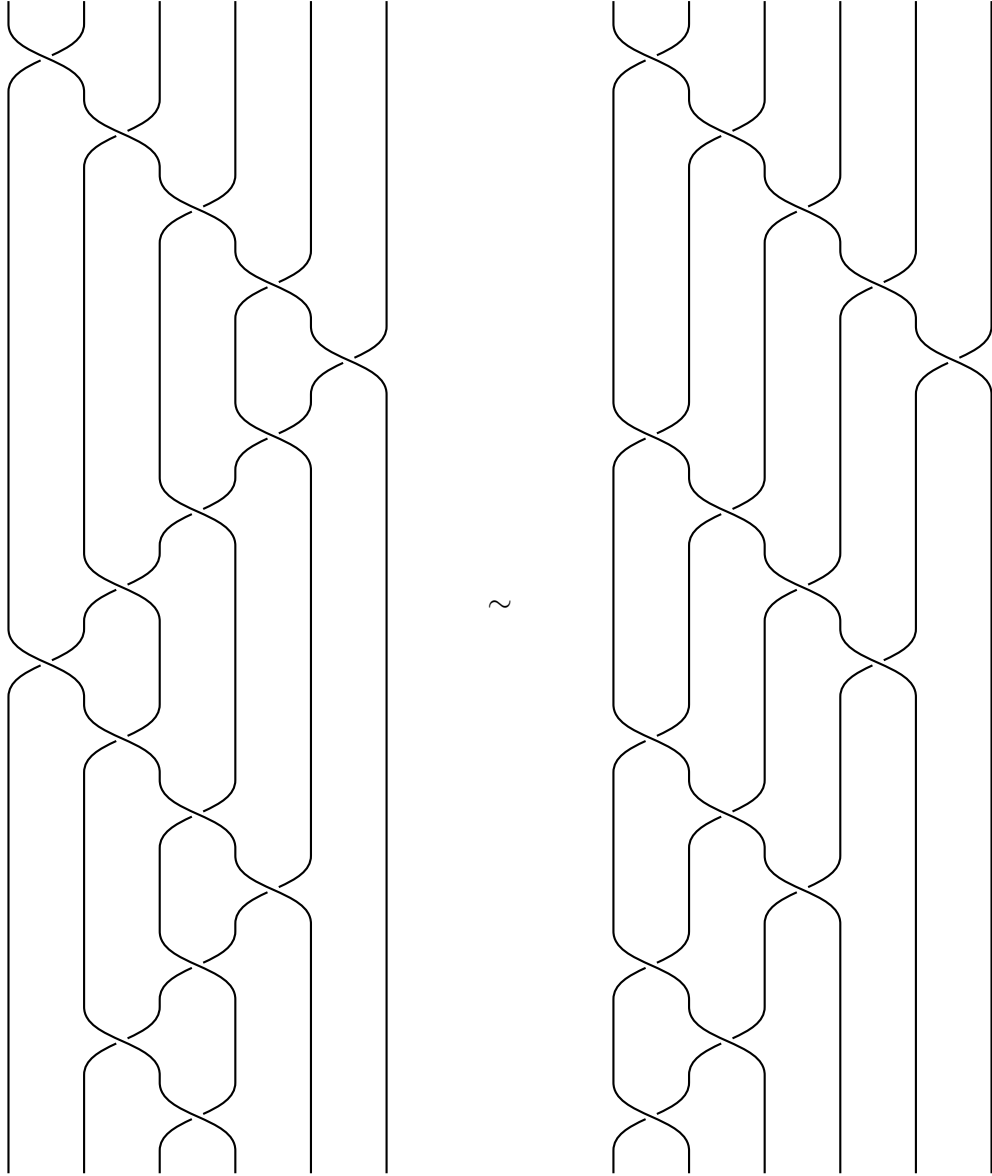


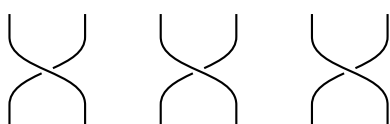
Figure 4.14: On the left, the diagram associated with $\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5\sigma_4\sigma_3\sigma_2\sigma_1\sigma_2\sigma_3\sigma_4\sigma_3\sigma_2\sigma_3$. The diagram on the right represents the same braid.

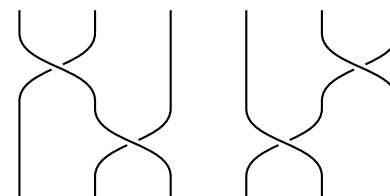
relations $x_i = \rho(\gamma_j)(x_i)$, for all $i \in [d]$ and $j \in [r]$. The core idea is that the standard generators $x_1, \dots, x_d$ of the fundamental group of $(\{b\} \times \mathbb{C}) \setminus \mathcal{C}$ —a complex plane punctured at d points—map under inclusion to generators of $\pi_1(\mathbb{C}^2 \setminus \mathcal{C})$. Transporting these loops as the parameter t follows loops in $\mathbb{C} \setminus \Sigma$ imposes relations on the x_i in $\pi_1(\mathbb{C}^2 \setminus \mathcal{C})$, and actually all relations between the x_i come from those braid relations.

We now work out an example using **Braids**. Consider the bivariate polynomial

$$f = x^6 - 2x^4 + 9tx^2 + x^2 - 8t,$$

with associated curve $\mathcal{C}$. Its discriminant in x is $8192 \cdot t^3(27t - 1)^4$, so $\Sigma = \{0, c\}$, where $c = \frac{1}{27}$. We choose $b = \frac{1}{100}$ and compute the braid monodromy map by computing the braids associated with loops around 0 and c . Using **Braids** and after simplification, we respectively obtain

$$b_0 = \sigma_1 \sigma_3 \sigma_5,$$


$$b_c = \sigma_1 \sigma_2 \sigma_5 \sigma_4.$$


To obtain $\pi_1(\mathbb{C}^2 \setminus \mathcal{C})$, we compute the (braid) automorphisms of F_6 associated with b_0 and b_c . We denote by $x_1, \dots, x_6$ the generators of F_6 . The automorphism ϕ_0 associated to b_0 is

$$[x_2, x_2^{-1}x_1x_2, x_4, x_4^{-1}x_3x_4, x_6, x_6^{-1}x_5x_6],$$

described as the list of its values on the generators $x_1, \dots, x_6$. Similarly, ϕ_c is given by

$$[x_2, x_3, x_3^{-1}x_2^{-1}x_1x_2x_3, x_6, x_6^{-1}x_4x_6, x_6^{-1}x_5x_6].$$

The group $\pi_1(\mathbb{C}^2 \setminus \mathcal{C})$ is given by generators $x_1, \dots, x_6$ and relations $x_i = \phi_0(x_i) = \phi_c(x_i)$. Either by looking closely at ϕ_0 and ϕ_c , or by using the `simplified()` method on finitely presented groups in **SageMath**, we ultimately obtain that $\pi_1(\mathbb{C}^2 \setminus \mathcal{C})$ is the free group on one generator, that is, $\mathbb{Z}$.

Note that this is also the result returned by the `fundamental_group` function provided by the **SageMath** interface to **SIROCCO**. Moreover, we could have guessed the result from the beginning: f factors as

$$\underbrace{x^2(x-1)^2(x+1)^2}_{a \in \mathbb{C}[x]} - \underbrace{(-9x^2 + 8)t}_{b \in \mathbb{C}[x]}.$$

That is, $\mathcal{C}$ is the graph of the rational function $\frac{a}{b}$, and removing it from the plane suggests that $\pi_1(\mathbb{C}^2 \setminus \mathcal{C})$ should be $\mathbb{Z}$ (although one needs to account for the zeros of b).

Chapter 5

Writing mathematical code in Rust

Why Rust? Rust is a compiled, low-level programming language with performance comparable to that of C++. One of its key features is memory management: the compiler enforces strict rules (ownership, rules on references, and lifetimes) to prevent memory-safety violations that may lead to hard-to-debug bugs (segmentation faults, double frees, ...), while automating memory deallocation without a garbage collector. These rules are also the main drawback of the language: they can make programming feel unnatural and challenging, and they introduce concepts absent from other standard low-level languages, making Rust difficult to learn. In terms of quality of life, Rust belongs to a modern ecosystem that includes, for instance, the Cargo package manager.

Generic programming The algorithms of Chapter 2 (`Refine`, `Track`, ...) are based on an abstract interval arithmetic over $\mathbb{R}$ satisfying a set of assumptions. If, for instance, we implement `Track` for several concrete interval arithmetics, we will end up writing the same function several times. This duplication can be avoided using *generic programming*: a function can be written using abstract data types that are later specified to obtain a concrete executable function. Generic programming is featured in languages such as Python, Julia, C++, Rust, although taking different forms (dynamic typing, classes, templates, traits, ...).

Even more abstraction A practical implementation of interval arithmetic must provide a data structure and methods for real intervals, but may also provide extensions such as complex-box and complex-box matrix-vector arithmetic. For instance, the double-precision interval arithmetic implemented in `Algpath` provides complex-box arithmetic using 256-bit registers and the AVX and AVX2 instruction sets. On the other hand, `Arb`, which is also used in `Algpath`, provides its own complex-ball and complex-ball matrix types. This means that a function such as `Track`, which is generic with respect to the interval arithmetic, should depend

on the types of real intervals, complex boxes, and complex-box matrices at least. The resulting large number of type parameters and abstract methods makes the implementation challenging.

Generics in Rust Rust provides tools for writing generic code, which are used extensively in *Algpath* to reduce code duplication. Generics are particularly important in computer algebra because an algorithm may apply in several contexts, and we want to avoid rewriting it for each valid context.

In this chapter, we review the basic features of generics in *Rust* and list pitfalls that we encountered while implementing *Algpath*. We conclude by outlining an implementation strategy, deduced from our experience implementing *Algpath*. This chapter is mostly heuristic: it summarizes our experience implementing *Algpath* and presents ideas that have yet to be tested on a larger scale, to have a clearer idea of their limitations.

5.1 Generics

Rust's generics revolve around type parameters for functions and structures, as well as traits. For more details about generics and *Rust* in general, see [55].

5.1.1 Generic functions and structures

Consider the following function, which reverses an array of 64-bit integers in place.

```
1 fn reverse(l: &mut [u64]) {
2     let n = l.len(); //length of l
3     for i in 0..n / 2 {
4         l.swap(i, n - 1 - i); //swap entry i and entry n - 1 - i
5     }
6 }
```

This function is already present in the standard library of *Rust*; we only consider it for the sake of explanation.

A digression on references The variable *l* represents a reference to an array of integers; its precise type is `&mut [u64]`, that is, a mutable slice of 64-bit integers. The `&` symbol denotes that *l* is a reference to some data. Most references are represented by a pointer guaranteed to point to valid data. Slices are references to contiguous collections of data that additionally record the length of the collection. The keyword `mut` indicates that the data that *l* refers to can be modified. In *Rust*, variables and references are *immutable* by default. For instance, the code

does not compile if we replace `&mut [u64]` with `&[u64]`—a plain, and therefore immutable, slice—because `l` is modified in place. In this case, the compiler suggests reintroducing `mut`.

```
error[E0596]: cannot borrow `*l` as mutable, as it is behind a `&` reference
--> src/main.rs:4:9
|
4 |         l.swap(i, n - 1 - i); //swap entry i and entry n - 1 - i
|         ^ `l` is a `&` reference, so it cannot be borrowed as mutable
|
help: consider changing this to be a mutable reference
|
1 | fn reverse(l: &mut [u64]) {
|               +++
```

Borrowing and *lifetimes* are central concepts in Rust associated with references. We omit these notions from the remainder of this discussion; for details, see [55].

Type parameters If we want to reverse a slice `l` containing 32-bit unsigned integers (of type `u32`), `reverse` does not apply because `l` does not have the required type.

```
1 fn main() {
2     let l: &mut [u32] = &mut [1, 2, 3, 4];
3     reverse(l);
4 }

error[E0308]: mismatched types
--> src/main.rs:3:13
|
3 |     reverse(l);
|     ^ expected `&mut [u64]`, found `&mut [u32]`
|
|     arguments to this function are incorrect
|
= note: expected mutable reference `&mut [u64]`
       found mutable reference `&mut [u32]`
```

Remark 5.1. The expression `[1, 2, 3, 4]` yields an array of length 4, allocated on the stack. Taking a mutable reference to this array via `&mut [1, 2, 3, 4]` yields a reference of type `&mut [u32; 4]`, not `&mut [u32]`. In the first case, the static length is part of the type, whereas in the second case the length is stored in the reference. Sweeping details under the rug, Rust automatically coerces array references to slices, so Line 2 of `main` causes no issue.

To avoid rewriting this function for each possible element type of `l`, Rust allows functions to have *generic type parameters*.

```

1 fn reverse<T>(l: &mut [T]) {
2     let n = l.len();
3     for i in 0..n / 2 {
4         l.swap(i, n - 1 - i);
5     }
6 }

```

The type parameter `T` is added between angle brackets after the name of the function. This function can reverse slices of `u32`, slices of `u64`, or slices containing any concrete type `T`. During compilation, Rust generates a version of `reverse` for each concrete slice type used as an argument: it *monomorphizes* the code.

Calling a generic function When calling a generic function, type parameters can be instantiated using the *turbofish operator* `::<>`.

```

1 fn main() {
2     let l: &mut [u32] = &mut [1, 2, 3, 4];
3     reverse::<u32>(l);
4 }

```

In some cases, Rust can automatically infer the type parameters from the arguments of the generic function, in which case the turbofish operator can be omitted. For `reverse`, the type `T` can be inferred from the type of `l`, and we can simply call `reverse` as follows.

```

1 fn main() {
2     let l: &mut [u32] = &mut [1, 2, 3, 4];
3     reverse(l);
4 }

```

Structures with type parameters We can define *structures* and implement *methods* over them. The syntax is as follows.

```

1 struct MyStruct { field1: u64, .. }
2
3 impl MyStruct {
4     fn method( .. ) .. { .. }
5 }

```

If `x` is of type `MyStruct`, we call `method` via the syntax `x.method(..)`. In the `impl` block above, `method` should have `self`, `&self`, or `&mut self` as its first argument, denoting the value of type `MyStruct` on which the method is called. A function defined in the `impl` block of `MyStruct` may omit `self` (or one of its reference

variants) as its first argument; such functions are called *associated functions* of `MyStruct`.

Similarly to functions, structures can depend on generic type parameters. Here is an example.

```
1 struct Poly<T> {
2     coeffs: Vec<T>,
3 }
```

Methods can be implemented for a structure with type parameters, generically with respect to those parameters. For instance, we may want to define the degree of an element of type `Poly<T>` generically with respect to `T`.

```
1 impl<T> Poly<T> {
2     fn degree(&self) -> u64 { .. }
3 }
```

Implementing this operation as a method of `Poly<T>` or as a generic function depending on `T` is roughly equivalent: in the first case, we call `degree` using `f.degree()`, and in the second using `degree(&f)`.

5.1.2 Traits

Traits form the backbone of generic programming in Rust. The notion of traits is not specific to Rust; for instance, they also appear in PHP, Scala, and in C++ with concepts. In this section, we review the fundamental features of traits in Rust.

An example We can try to implement generic polynomial addition over `Poly<T>`.

```
1 fn poly_add<T>(mut f: Poly<T>, mut g: Poly<T>) -> Poly<T> {
2     if f.coeffs.len() < g.coeffs.len() {
3         mem::swap(&mut f, &mut g); //swap f and g
4     }
5     //iterate over the coefficients of g
6     for (i, bi) in g.coeffs.into_iter().enumerate() {
7         f.coeffs[i] += bi
8     }
9     return f;
10 }
```

Compilation produces the following error.

```

error[E0368]: binary assignment operation `+=` cannot be applied to type `T`
--> src/main.rs:7:9
|
7 |         f.coeffs[i] += bi
|         ~~~~~~^~~~~~
|         |
|         cannot use `+=` on type `T`
|
help: consider restricting type parameter `T` with trait `AddAssign`
|
1 | fn poly_add<T: std::ops::AddAssign>(mut f: Poly<T>, mut g: Poly<T>) ->
  ↳ Poly<T> {
  |         ++++++

```

At Line 7 of `poly_add`, we use the `+=` operator with values of type `T`, and the compiler complains because we did not state that `+=` can be applied to `T`. As suggested by the compiler, we can introduce a *trait bound* for `T` in `poly_add` by adding

```
1 fn poly_add<T: AddAssign>(..) -> Poly<T> { .. }
```

or equivalently, using a `where` clause.

```
1 fn poly_add<T>(..) -> Poly<T> where T: AddAssign { .. }
```

The trait bound `T: AddAssign` indicates that `T` implements the `AddAssign` trait, which provides the `add_assign` method aliased by the `+=` operator. When `poly_add` is used with a concrete type `S`, Rust checks that `S` implements `AddAssign` and raises an error otherwise. For instance, addition is not implemented for threads (what would the addition of two threads even mean?).

```

1 fn main() {
2     let f: Poly<Thread> = Poly { coeffs: vec![] };
3     let g: Poly<Thread> = Poly { coeffs: vec![] };
4     let h = poly_add(f, g);
5 }

```

```

error[E0277]: cannot add-assign `Thread` to `Thread`
--> src/main.rs:4:13
|
4 |     let h = poly_add(f, g);
|               ~~~~~~ no implementation for `Thread += Thread`
|
= help: the trait `AddAssign` is not implemented for `Thread`

```

Traits and trait bounds Rust generics revolve around *traits* and *trait bounds*. A trait defines some abstract behavior and can be implemented for concrete types. In a generic function, we can impose trait bounds on a type parameter `T` to state that `T` harbors the behavior specified by the traits. Before monomorphizing the code, Rust type checks all generic functions using their trait bounds. This contrasts with how C++ templates work, where one can omit assumptions about type parameters and errors occur after monomorphizing the code.

In practice, traits allow for clear error messages, as no implicit requirement is discovered only after instantiation, unlike with the infamous C++ template errors. The obvious downside is the verbosity, as most generic functions require trait bounds on their type parameters.

Abstract methods A trait can define an abstract method signature that can be implemented for concrete types. For example, we can have a trait for the width function over real intervals.

```
1 trait Width {
2     fn width(&self) -> Rational64;
3 }
```

We can implement the `Width` trait for concrete interval types.

```
1 // Exact intervals
2 struct Interval { lo: Rational64, hi: Rational64 }
3
4 // Exact balls
5 struct Ball { mid: Rational64, rad: Rational64 }
6
7 // "implement trait Width over Interval"
8 impl Width for Interval {
9     fn width(&self) -> Rational64 { return self.hi - self.lo; }
10 }
11
12 impl Width for Ball {
13     fn width(&self) -> Rational64 { return self.rad * 2; }
14 }
```

The trait can be used in generic functions.

```
1 fn is_singleton<T: Width>(x: &T) -> bool {
2     return x.width().is_zero();
3 }
```

Associated types Traits can also provide abstract types, called *associated types*. We can modify the `Width` trait to have an associated type for the output of `width`, instead of fixing it to `Rational64`.

```
1 trait Width {  
2     type Output;  
3  
4     fn width(&self) -> Self::Output;  
5 }
```

When implementing the trait above, one specifies `Output`.

```
1 struct F64Interval { lo: f64, hi: f64 }  
2  
3 impl Width for F64Interval {  
4     type Output = f64; //set Output to f64  
5  
6     fn width(&self) -> f64 { self.hi - self.lo } //!rounding  
7 }
```

When implementing `is_singleton`, we can recover the type `Output` associated with the implementation of `Width` for `T` using the syntax `T::Output`.

```
1 fn is_singleton<T>(x: &T) -> bool  
2 where  
3     T: Width,  
4     T::Output: Zero, //the Zero trait defines is_zero  
5 {  
6     return x.width().is_zero();  
7 }
```

Here we require `T::Output` to implement the `Zero` trait, which provides the method `is_zero`. When calling `is_singleton`, `T` is automatically inferred from the type of the input `x`.

```
1 let x = F64Interval { .. }  
2 if is_singleton(&x) { .. }
```

Generic type parameters Like functions and structures, traits can have generic type parameters. This is the case for `AddAssign`, which depends on a type parameter `Rhs`, set by default to `Self` (the type for which `AddAssign` is being implemented).

```

1 trait AddAssign<Rhs = Self> {
2     fn add_assign(&mut self, rhs: Rhs);
3 }

```

We can add a generic type parameter for the output type of `Width` instead of an associated type.

```

1 trait Width<Output> {
2     fn width(&self) -> Output;
3 }

```

We can then implement it for exact balls, setting the output type to a rational number.

```

1 impl Width<Rational64> for Ball {
2     fn width(&self) -> Rational64 { .. }
3 }
4
5 impl Width<f64> for Ball {
6     fn width(&self) -> f64 { .. }
7 }

```

To implement `is_singleton` similarly to the implementation with an associated type, we need an additional type parameter.

```

1 fn is_singleton<T, Output>(x: &T) -> bool
2 where
3     T: Width<Output>,
4     Output: Zero,
5 {
6     return x.width().is_zero();
7 }
8
9 fn main() {
10     let x = Ball { .. };
11     if is_singleton::<_, Rational64>(&x) { .. }
12 }

```

Here, `Output` cannot be deduced from the input `x`, as the compiler cannot decide which implementation of `Width` for `Ball` should be used. Thus, a call to `is_singleton` requires the turbofish operator. Note that the first type parameter can be deduced from the type of `x`, so we can insert a “`_`” instead of “`Interval`”.

Associated types versus type parameters What is the difference between associated types and type parameters? The main point is that a trait cannot be implemented twice for the same type. For instance, it is not possible to implement `Width` twice for `F64Interval`, even if `Output` is different.

```
1 impl Width for F64Interval {
2     type Output = f64;
3
4     fn width(&self) -> Self::Output { .. }
5 }
6
7 impl Width for F64Interval {
8     type Output = Rational64;
9
10    fn width(&self) -> Self::Output { .. }
11 }
```

```
error[E0119]: conflicting implementations of trait `Width` for type
↳ `F64Interval`
--> src/main.rs:7:1
|
1 | impl Width for F64Interval {
| ----- first implementation here
...
7 | impl Width for F64Interval {
| ~~~~~~ conflicting implementation for `F64Interval`
```

But if `Output` is a type parameter, then `Width<f64>` and `Width<Rational64>` are different traits, and we can implement both:

```
1 impl Width<f64> for F64Interval {
2     fn width(&self) -> f64 { .. }
3 }
4
5 impl Width<Rational64> for F64Interval {
6     fn width(&self) -> Rational64 { .. }
7 }
```

The downside is the introduction of an additional type parameter, which may lead to heavier syntax involving the turbofish operator, for instance. For `Width`, an associated type seems to be the better choice. For instance, this is consistent with the `Add` trait from the Rust standard library, for which the output type of the `add` method (`+` operator) is given as an associated type.

Default implementations When writing a trait that provides a function, it is possible to specify a default implementation. For example, such a default implementation is present in the `Zero` trait provided by the `num` package (see <https://docs.rs/num/latest/num/>).

```

1 trait Zero /*--snipped, kept for later--*/ {
2     // Required methods
3     fn zero() -> Self;
4     fn is_zero(&self) -> bool;
5
6     // Provided method
7     fn set_zero(&mut self) { *self = Zero::zero(); }
8 }

```

The method `set_zero` has a default body. To implement `Zero` on a type `T`, it is sufficient to specify the body of `zero` and `is_zero`. The default implementation of `set_zero` can be overridden by specifying a body for `set_zero`.

5.2 Specialization pitfalls

5.2.1 Conflicting implementations

Specialization in computer algebra Consider matrices over a set R . The algorithm associated with the naive formula for multiplication only requires two operations $+$ and $\times$. For this reason, matrices can be multiplied in very general contexts, although one should make sense of what matrix multiplication means when R does not have a ring structure (think of R as being real intervals for instance). For Strassen’s algorithm, it is sufficient to have a $-$ operation in addition to $+$ and $\times$, although further properties on $-$, $+$ and $\times$ (for instance, they form a ring) are required to ensure that it yields the same result as naive multiplication.

On the other hand, polynomial matrices over a base field K , that is, matrices with entries in $K[x]$, benefit from specialized algorithms with lower complexity than general-purpose algorithms such as naive or Strassen’s multiplication. For instance, let d be an upper bound on the degrees of the entries of the input matrices, and assume a geometric sequence of $2d + 1$ points in K . In this case, using an evaluation-interpolation approach, matrices can be multiplied in $O(n^\omega d + n^2 M(d))$ operations in K , where $2 \leq \omega \leq 3$ is such that we can multiply two matrices over K in $O(n^\omega)$ and M represents the cost of multiplying polynomials of degree d with coefficients in K . In this case, naive multiplication requires $\Theta(n^3)$ multiplications over $K[x]$, each with a cost of at least $M(d)$.

Specialization? Ideally, we would like to have a default implementation of matrix multiplication, using naive multiplication, that applies to a broad variety of concrete coefficient sets. Whenever more information is known about R , we override this implementation with more specialized algorithms.

```

1 struct Matrix<T> { .. }
2
3 impl<T: Add + Mul> Mul for Matrix<T> {
4     //implement naive multiplication
5 }
6
7 impl<T: Add + Mul + Sub> Mul for Matrix<T> {
8     //implement Strassen's algorithm
9 }

```

Note that the trait bounds in the code above are simplified, for the sake of explanation. If we compile this code, we get:

```

error[E0119]: conflicting implementations of trait `Mul` for type `Matrix<_>`
--> src/main.rs:7:1
|
3 | impl<T: Add + Mul> Mul for Matrix<T> {
| ----- first implementation here
...
7 | impl<T: Add + Mul + Sub> Mul for Matrix<T> {
| ^

```

A type could satisfy the bounds of both implementations (in fact it will satisfy `Add + Mul` if it satisfies `Add + Mul + Sub`) and Rust does not allow multiple implementations of the same trait. But specialization is obvious here, as one set of constraints is included in the other. Should Rust therefore be able to determine which implementation is the most specialized and use it accordingly? Unfortunately, stable Rust does not provide such specialization tools (such as specialization of templates in C++). One blocking factor comes from a technical issue with *lifetimes*; for reference, see [91].

Aside from language considerations, specialization may lead to ambiguities when two generic implementations of the same function apply to a concrete type but it is unclear which is more specific. Languages such as C++ and Julia allow specialization but raise an error in case of ambiguity.

Large scope generic implementations Assume that we can multiply matrices of type `Matrix<T>`, implemented using the `Mul` trait.

```

1  impl<T> Mul for Matrix<T> where .. {
2      type Output = Self;
3      fn mul(self, rhs: Matrix<T>) -> Self::Output { .. }
4  }

```

Assume the trait bound `T: Mul<S, Output = T>`: elements of type `T` can be multiplied with elements of type `S`, and the result has type `T`. In this case, we can multiply matrices of type `Matrix<T>` by elements of type `S`, coefficient by coefficient. You can imagine that `T` represents univariate polynomials with rational coefficients and `S` represents rational numbers: we can multiply a polynomial matrix A by a rational coefficient λ by multiplying each polynomial coefficient of A by λ . For this reason, we may be tempted to write the following.

```

1  impl<S, T> Mul<S> for Matrix<T> where T: Mul<S, Output = T> {
2      type Output = Self;
3      fn mul(self, rhs: S) -> Self::Output { .. }
4  }

```

Unfortunately, in the code above, we may instantiate `S` with `Matrix<T>`, which leads to another implementation of matrix multiplication that conflicts with the initial `Mul` implementation. After instantiation, the trait bound in the above code snippet is `T: Mul<Matrix<T>, Output = T>`, but from the mathematical point of view we do not expect a scalar value multiplied with a matrix to result in a scalar value, so this constraint should never hold. Still, (stable) Rust does not provide a way to ensure that `T: Mul<Matrix<T>, Output = T>` never holds (the notion of *negative trait bounds*).

It seems difficult to anticipate such conflicts: defining an implementation that has a broad scope may backfire later. Trait implementations that involve a free type parameter, or that apply to all types (*blanket implementations*), should be avoided. We will come back to this issue in Section 5.4.

5.2.2 Newtype pattern

A way to avoid conflicting implementations of a trait for a type is to duplicate the type using a tuple struct. For example, suppose that we have a generic matrix type `Matrix<T>` for which naive multiplication is implemented via the `Mul` trait (with appropriate trait bounds on `T`). We may want to reimplement `Mul` for `Matrix<T>` in a different way, but, as mentioned above, this new implementation will most likely conflict with naive multiplication. To circumvent this conflict, we can create a new type that contains the exact same information as a `Matrix<T>` by using a *tuple struct*.

```

1 // A structure containing a single field of type Matrix<T>.
2 struct NewMatrix<T>(Matrix<T>);

```

Given `x` of type `NewMatrix<T>`, we can access the inner value via the syntax `x.0`. Because `NewMatrix` is a different type from `Matrix`, we can implement the `Mul` trait without conflicts.

Limitations First, when wrapping a type with a tuple struct, we may want to *derive* functions from the inner type to the outer type. For instance, accessing a coefficient of `NewMatrix` amounts to accessing the coefficient of the inner `Matrix`. In fact, many basic trait implementations for `Matrix` should be exported to `NewMatrix`. Although tools exist to automate this process in `Rust` (such as the `#[derive(...)]` macro), they only apply to a limited set of traits. This leads to code in which only a few methods are new, while all other methods are derived manually, which is not ideal.

Second, `Matrix` and `NewMatrix` both represent the same object, while their methods differ. Having two types for the same object is not ideal, and additional levels of specialization will lead to even more wrappers and types representing the same objects. It is preferable to have a single type that represents matrices `Matrix<T>`, and different implementations of the multiplication, although it is not possible to achieve this with the `Mul` trait because of conflicts.

Last but not least, if we are given a reference `v` to a vector of matrices, of type `&Vec<Matrix<T>>`, it is not possible to convert it to a reference of type `&Vec<NewMatrix<T>>` without the use of `unsafe`. Although both types look alike, `Rust` cannot safely convert from one type to the other as it cannot ensure that the memory layout of `Vec<Matrix<T>>` and `Vec<NewMatrix<T>>` are the same. In `Alghath`, we relied too heavily on the newtype pattern, which led to this issue, and we resorted to using `unsafe`. Hopefully, this can be removed after revising the code, now that we have a clearer understanding of how to specialize properly.

5.3 Trait pitfalls

5.3.1 Where clause swell

Consider the following abstract code.

```

1 // first generic function
2 fn f<T>(x: T)
3 where
4     //hash and debug print for T
5     T: Hash + Debug,

```

```

6      //operators == and <, >, <=, >= for Vec<T>
7      Vec<T>: Eq + Ord,
8  { .. }
9
10 // second generic function that calls f
11 fn g<T>(x: T) { f(x); }

```

If we try to compile, we get a set of errors of the following form.

```

error[E0277]: the trait bound `T: Hash` is not satisfied
--> src/main.rs:7:19
   |
7 | fn g<T>(x: T) { f(x); }
   |               - ^ the trait `Hash` is not implemented for `T`
   |               |
   |               required by a bound introduced by this call
   |

```

That is, we cannot use `f` in `g` because we need to impose the appropriate trait bounds on `T` in `g`. This can be repaired as follows:

```

1  fn f<T>(x: T)
2  where
3      T: Hash + Debug, Vec<T>: Eq + Ord
4  { .. }
5
6  fn g<T>(x: T)
7  where
8      T: Hash + Debug, Vec<T>: Eq + Ord
9  { f(x); }

```

This is far from ideal, because the `where` clause of `f` is duplicated in the `where` clause of `g`. Aside from code duplication, additional levels of function calls will cause the `where` clause to swell.

Using a supertrait A partial solution to this problem is to define a *supertrait* that encompasses a set of traits.

```

1  trait BT: Hash + Debug {}
2  trait BVecT: Eq + Ord {}

```

Introducing `BT` and `BVecT` simplifies the `where` clauses of `f` and `g`.

```

1  fn f<T>(x: T)
2  where

```

```

3     T: BT, Vec<T>: BVecT
4 { .. }
5
6 fn g<T>(x: T)
7 where
8     T: BT, Vec<T>: BVecT
9 { f(x); }

```

The trait bound `T: BT` implies that `T: Hash + Debug`. But the converse is not true: `BT` is not automatically implemented for `T` if `T: Hash + Debug` (thus, `BT` is not yet an alias for `Hash + Debug`). For instance, consider the following call to `g`.

```

1 fn main() {
2     g(1u64) //1u64 is 1 of type u64
3 }

```

Compilation produces the following error:

```

error[E0277]: the trait bound `u64: BT` is not satisfied
--> src/main.rs:2:7
   |
2  |     g(1u64)
   |     ~~~~~ the trait `BT` is not implemented for `u64`
   |     |
   |     required by a bound introduced by this call
   |

```

although `u64` satisfies `Hash + Debug` and `Vec<u64>` satisfies `Eq + Ord`. To turn `BT` and `BVecT` into aliases, we use a blanket implementation.

```

1 impl<T> BT for T where T: Hash + Debug {}
2 impl<T> BVecT for T where T: Eq + Ord {}

```

Here, `BT` is implemented for all types that implement `Hash` and `Debug`, and we can call `g(1u64)`.

Introducing traits to simplify `where` clauses makes the `where` clauses more manageable, but it is not possible to have a single trait that acts as an alias of the whole `where` clause: we need a new trait for each type that is constrained in the `where` clause, here `T` and `Vec<T>`. For this reason, the `where` clause may still swell if the number of constrained types grows with the depth of function calls.

Using a trait for `f` Instead of using a generic function for `f`, we can define a trait for `f`.

```

1 trait F<T> {
2     fn f(x: T);
3 }

```

We implement this trait for a *generic implementor type*, represented here by the empty struct `FImpl<T>`.

```

1 struct FImpl<T> { types: PhantomData<T> }
2
3 impl<T> F<T> for FImpl<T>
4 where
5     T: Hash + Debug, Vec<T>: Eq + Ord
6 {
7     fn f(x: T) { .. }
8 }

```

Note the field `types: PhantomData<T>` in `FImpl<T>`. This is because every type parameter of the structure must appear in at least one field. `PhantomData<T>` can be used to ensure this property: it simulates a field of type `T` while consuming no space. For details, see <https://doc.rust-lang.org/std/marker/struct.PhantomData.html>.

To use `f` in `g`, we only need to require `FImpl<T>` to implement the `F<T>` trait:

```

1 fn g<T>(x: T)
2 where
3     FImpl<T>: F<T>
4 {
5     FImpl::f(x);
6 }

```

In all future calls of `f`, its `where` clause can be summarized using the bound `FImpl<T>: F<T>`, thus solving the `where` clause swell issue. Moreover, we can have additional implementor types for `F<T>` using different trait bounds on `T`, and let `g` take the implementor type of `F<T>` as a type parameter.

```

1 struct FImplBis<T> { types: PhantomData<T> }
2
3 impl<T> F<T> for FImplBis<T>
4 where
5     //bounds differ from FImpl
6     T: Add + Mul, Vec<T>: Hash + Debug
7 {
8     fn f(x: T) { .. }

```

```

9  }
10
11 fn g<FImplT, T>(x: T)
12 where
13     //the abstract implementor type implements F<T>
14     FImplT: F<T>,
15 {
16     FImplT::f(x);
17 }
18
19 fn main() {
20     //use different implementations of f
21     //by specifying the implementor type
22     g::<FImpl<_>, _>(1);
23     g::<FImplBis<_>, _>(1);
24 }

```

It is a particular case of externalization with a context, which we explore in Section 5.4.

A downside of this approach is that trait bounds errors can become difficult to understand. If this strategy is used over many layers of function calls, an unsatisfied bound at the lower level can cause an issue at the higher level, which may lead to a cascade of errors. A good practice is to check incrementally that the implementor types implement their associated trait for the expected concrete types. For instance, after implementing $F<T>$ for $FImpl<T>$, we could first check whether $FImpl<F64>$ satisfies the bound $F<f64>$. The strategy of using traits and implementor types for generic functions is used extensively in **Algpath** (and we also obtained horrendous error messages by not respecting the previous advice).

5.3.2 Large versus small traits

Consider the following code.

```

1  pub trait WidthUpper {
2      fn width_upper(&self) -> f64;
3  }
4  pub trait MagUpper {
5      fn mag_upper(&self) -> f64 ;
6  }
7  pub trait Endpoints {
8      fn endpoints(&self) -> (f64, f64);
9  }

```

```

10
11 fn f<T>( .. ) where T: WidthUpper + MagUpper + Endpoints { .. }

```

It can be transformed into the following simpler code.

```

1 pub trait IntervalMethods {
2     fn width_upper( .. ) -> f64;
3     fn mag_upper( .. ) -> f64;
4     fn endpoints( .. ) -> (f64, f64);
5 }
6
7 fn f<T>( .. ) .. where T: IntervalMethods { .. }

```

One immediate benefit is the simplification of the `where` clause of `f`. The drawback is that `width_upper`, `mag_upper` and `endpoints` are now grouped together and cannot be implemented separately. While we can implement `endpoints` for double-precision intervals $[a, b] \subseteq \mathbb{R}$, it is not possible to implement it for balls $[r \pm s]$, as the endpoints cannot be known without approximation errors. Still, we may want to implement `width_upper` and `mag_upper` for real balls. Although we can fake the implementation of `endpoints` for balls by filling its body with a runtime error (using the `unimplemented!()` macro), this moves a trait error, which would be detected at compile time, to a runtime error that is harder to analyze.

For the above reason, traits should remain small. It seems preferable to split a large trait `LargeTrait` into single-method traits `Trait1`, ..., `Traitn`. The simplification of `where` clauses can then be recovered by defining a supertrait that encompasses `Trait1`, ..., `Traitn`.

5.3.3 Where clauses in traits

In a similar fashion to generic functions, traits can have a `where` clause with trait bounds. Where clauses in traits may allow a default implementation to be written and can enforce trait semantics on the implementing type. For instance, the trait `Zero` requires `Add<Output = Self>`; the notion of “zero” does not make much sense without an addition operation, although it is debatable if the constraint `Add<Output = Self>` should be part of the trait or not.

Bounds on type parameters It is possible to bound the type parameters of a trait.

```

1 trait Contains<R> where R: Into<f64> {
2     fn contains(&self, x: R) -> bool;
3 }

```

As with generic functions, `where` clauses involving types other than associated types and `Self` are not inherited.

```
1 fn f<R, BoxR>() where BoxR: Contains<R> { .. }

error[E0277]: the trait bound `f64: From<R>` is not satisfied
--> src/main.rs:1:29
|
1 | fn f<R, BoxR>() where BoxR: Contains<R> {}
|                                ~~~~~ the trait `From<R>` is not
  ↳ implemented for `f64`
|
= note: required for `R` to implement `Into<f64>`
```

Composition of traits leads to exactly the same problem as generic functions: `where` clause swell.

Bounds on associated types We can also bound associated types, and in this case the `where` clause swell issue does not occur. Still, adding a `where` clause to a trait makes it less versatile because of the stacked constraints, similarly to large traits. For instance, assume the following trait.

```
1 trait Track {
2     type boxR;
3
4     fn track( .. ) .. { .. }
5 }
```

We may be tempted to introduce all necessary bounds on `boxR` to be able to generically implement `track` using a default implementation in the trait. But if we want to reimplement `Track` in another way, all constraints on `boxR` will still be required, although they may not be relevant.

The conclusion is that `where` clauses in traits should be avoided, as they only reduce the usability of the trait. Still, they can be used on a small scale to enforce trait semantics.

5.4 Towards computer algebra in Rust

5.4.1 Generic algorithms, specific implementations

To circumvent the lack of specialization while reducing code duplication, we propose a strategy that goes only one step beyond not using generics at all. The idea is the following: we only implement traits on concrete types to avoid conflicts, while

we provide generic functions that implement algorithms. These functions can be used with concrete types when implementing traits. This rule mostly applies to code that is intended for reuse; local and internal code is free to use all the tools available in Rust.

For instance, returning to the example of Subsection 5.2.1, consider a type `Matrix<T>` representing generic matrices. We can implement two generic functions for naive and Strassen’s multiplication algorithms.

```

1 fn mul_naive<T>(x: Matrix<T>, y: Matrix<T>) -> Matrix<T>
2 where
3     T: Add + Mul,
4 {
5     //implement naive multiplication
6 }
7
8 fn mul_strassen<T>(x: Matrix<T>, y: Matrix<T>) -> Matrix<T>
9 where
10     T: Add + Mul + Sub,
11 {
12     //implement Strassen's multiplication
13 }
```

For each concrete type, we implement the `Mul` trait by choosing a multiplication function.

```

1 impl Mul for Matrix<i64> {
2     type Output = Self;
3
4     fn mul(self, rhs: Self) -> Self::Output {
5         mul_strassen(self, rhs)
6     }
7 }
8
9 impl Mul for Matrix<u32> {
10     type Output = Self;
11
12     fn mul(self, rhs: Self) -> Self::Output {
13         mul_naive(self, rhs)
14     }
15 }
```

As `Matrix<u32>` and `Matrix<i64>` implement the `Mul` trait, their multiplication can be reused in other generic functions.

Because the body of the `impl` block for `Matrix<u32>` and `Matrix<i64>` only depends on `T` and on the multiplication function, we can write a simple macro that generates this block.

```

1  //a macro that generates the impl block of Mul for matrices
2  macro_rules! matrix_mul {
3      ($T:ty, $f:path) => {
4          impl Mul for Matrix<$T> {
5              type Output = Self;
6
7              fn mul(self, rhs: Self) -> Self::Output {
8                  $f(self, rhs)
9              }
10         }
11     };
12 }
13
14 //implement matrix Mul for a variety of coefficient types
15 matrix_mul!(f64, mul_strassen);
16 matrix_mul!(f32, mul_strassen);
17 matrix_mul!(u64, mul_naive);
18 matrix_mul!(u32, mul_naive);
19 matrix_mul!(i64, mul_strassen);

```

Pros and cons The main advantage of this approach is that it avoids implementation conflicts, while using some generic programming tools to reduce code duplication. But if the number of types increases, one must specify all implementations manually, even if each implementation consists of only one macro call. At the scale of `Algpath`, this instantiation process should be doable by hand, as the number of types involved is not too large. On a larger scale, automating trait implementations for concrete types would be required. It seems that logic programming could be used for this purpose, with a language such as (λ) Prolog.

Refactoring `Algpath` would be a good way to test the limitations of this approach; so far, we have conducted only partial experiments, which showed promising results. Another good benchmark would be to wrap `Flint` in `Rust` and provide an interface that relies on traits, allowing generic code to be written using `Flint` methods.

5.4.2 Context objects

In a compiled programming language, the type associated with the data structure representing a given object may not suffice, as some information may be known only at runtime. For instance, the type that represents a modular integer can only represent the modulus if it is known at compile time. For this reason, we need to add a *context* to the values: in the case of modular integers with word-size modulus n , the context consists (at least) in a 64-bit integer representing n . The context may also contain precomputations, such as the inverse of n modulo 2^{64} when n is odd.

Context objects play two roles. First, they allow us to interpret raw data: for instance, a modular integer is just a 64-bit integer. A context representing $\mathbb{Z}/n\mathbb{Z}$ allows us to interpret this integer as a modular integer. Second, they allow additional arguments to be provided to a function. For instance, multiplying two raw integers does not require additional arguments. But multiplying two integers interpreted as modular integers requires an additional argument representing the modulus, which is provided by the context.

The case of Flint If we look at Flint’s documentation, many operations have context arguments in addition to their operands. For instance, `arb_mul`, which multiplies two `Arb` balls, additionally takes as input the precision of computations. Other modules contain more complex *context object* types: for instance, `fq_ctx_t` is a type for finite field context objects that stores a prime number p , an irreducible polynomial in $\mathbb{F}_p[x]$, and additional precomputed data. All methods for finite fields take a context object of type `fq_ctx_t`.

On the other hand, `nmod_mat_mul`, the multiplication of matrices of integers modulo n (word-size n), does not take an additional argument that would contain n ; the context object for integers modulo n is stored in each matrix.

Attaching contexts to data While in some cases it is reasonable to attach the context to the raw data representing an object, it may lead to a non-trivial computational overhead for small types. For instance, integers modulo n (word-size n) are stored as single 64-bit integers. Gluing the modulus to each value doubles the size of every element, even though all the elements involved in a computation usually share the same modulus. In intensive computations, this halves the number of coefficients that can fit in caches and registers, which degrades performance. In this case, half of the cache and registers will end up containing redundant repetitions of the same modulus. On larger raw data types (dense polynomials, dense matrices, . . .), this overhead becomes negligible. This approach has the advantage that values are always stored together with their context, so no

additional context arguments are required for functions. In **Rust**, this allows us to use binary operators for instance.

Separating contexts from data Let us review the advantages of separating raw data from its context.

First and foremost, we saw that, in some cases, attaching the context to the raw data (e.g. integers modulo n) degrades performance. From the perspective of generic programming, it is inconvenient to have both data attached to a context and data separated from its context, as the function signatures will differ. Always separating contexts from raw data avoids the performance degradation caused by redundant storage of the context for small types, such as modular integers.

Second, attaching contexts to data leads to unnecessary redundancy in collections. For instance, consider a matrix with generic coefficients. The context for all the coefficients should be the same, but if a context is attached to each coefficient, it will appear redundantly throughout the matrix. Obviously, each value should store a pointer to the context instead of the context itself, meaning that the size of each object increases by only one machine word. Instead, we would prefer the matrix to contain only the raw data representing its coefficients and to provide the coefficient context object to the matrix operations that require it. For instance, this is how matrices over finite fields are implemented in **Flint**.

Lastly, the raw data can be kept bare, without a wrapper, so that all basic functions can be applied directly to the data without having to unwrap anything. For instance, the data representing a modular integer x is just a 64-bit integer, and we can directly apply all integer operations (addition, multiplication, modular reduction, ...) to x .

Context objects in Rust Context objects rule out the use of **Rust**'s binary operators because we cannot provide the additional context argument to those operators. More generally, already-existing **Rust** implementations are not designed to work with contexts. For instance, the **nalgebra** crate (<https://docs.rs/nalgebra/latest/nalgebra/>) provides generic algorithms for linear algebra but relies on traits such as **Add** and **Mul** (associated with $+$ and $*$) from the **Rust** standard library, which cannot be adapted to separate context objects.

Conclusion on an example We finish by implementing generic polynomial multiplication using generics as explained in Subsection 5.4.1, and context objects. This is merely a toy example intended to give a practical idea of the implementation strategy explained in this section. Let R be a set. To write a generic algorithm for multiplying polynomials over R , we need to add and multiply elements of R . To this end, we define two traits.

```

1  //x += y
2  pub trait AddAssign<L, R> {
3      fn add_assign(&self, x: &mut L, y: R);
4  }
5
6  //x * y
7  pub trait Mul<L, R> {
8      type Output;
9
10     fn mul(&self, x: L, y: R) -> Self::Output;
11 }

```

We also implement a macro that, given operand types `L` and `R`, a context type `Ctx`, an output type `Output`, and a method `method`, writes an `impl` block for `Mul` that calls `method`.

```

1  macro_rules! mul {
2      ($L:ty, $R:ty, $Ctx:ty, $Output:ty, $method:path) => {
3          impl Mul<$L, $R> for $Ctx {
4              type Output = $Output;
5
6              fn mul(&self, x: $L, y: $R) -> Self::Output {
7                  $method(&self, x, y)
8              }
9          }
10     };
11 }

```

Next, we define a type representing polynomials and a type representing polynomial context objects.

```

1  //T: parametric type for the coefficients
2  pub type Poly<T> = Vec<T>;
3
4  //TCtx: parametric context object type for the coefficients
5  pub struct PolyCtx<TCtx> {
6      t_ctx: TCtx,
7  }

```

Note that a generic polynomial is just a vector, so all vector methods can be applied directly to the raw data representing the polynomial (constructors, indexing, length, ...).

Next, we implement a generic function for the naive multiplication of polynomials.

```

1 fn naive_mul<TCtx, T>(poly_ctx: &PolyCtx<TCtx>, f: &Poly<T>, g: &Poly<T>) ->
  ↳ Poly<T>
2 where
3   TCtx: AddAssign<T, T> + for<'l, 'r> Mul<&'l T, &'r T, Output = T>,
4 {
5   let t_ctx = &poly_ctx.t_ctx;
6   let n = f.len();
7   let m = g.len();
8   if n == 0 && m == 0 {
9     return vec![];
10  }
11  let mut res = Vec::with_capacity(m + n - 1);
12  for i in 0..n {
13    let a = &f[i];
14    for j in 0..m {
15      if res.len() <= i + j {
16        res.push(t_ctx.mul(a, &g[j]));
17      } else {
18        t_ctx.add_assign(&mut res[j + i], t_ctx.mul(a, &g[j]));
19      }
20    }
21  }
22  res
23 }

```

Note that we could define a more generic `naive_mul`, for instance by abstracting the polynomial type itself, although more abstraction means more trait bounds in the `where` clause.

Let us define some concrete mathematical objects.

```

1 // modular integers
2 pub struct NmodCtx { modulus: usize }
3
4 pub type Nmod = usize;
5
6 impl AddAssign<Nmod, Nmod> for NmodCtx {
7   fn add_assign(&self, x: &mut Nmod, y: Nmod) {
8     *x += y;
9     *x %= self.modulus;
10  }
11 }
12
13 impl Mul<&Nmod, &Nmod> for NmodCtx {
14   type Output = Nmod;
15
16   fn mul(&self, x: &Nmod, y: &Nmod) -> Self::Output {

```

```

17         (x * y) % self.modulus
18     }
19 }
20
21 // integers
22 pub struct IntegerCtx;
23
24 pub type Integer = isize;
25
26 impl AddAssign<Integer, Integer> for IntegerCtx {
27     fn add_assign(&self, x: &mut Integer, y: Integer) {
28         *x += y;
29     }
30 }
31
32 impl Mul<&Integer, &Integer> for IntegerCtx {
33     type Output = Integer;
34
35     fn mul(&self, x: &Integer, y: &Integer) -> Self::Output {
36         x * y
37     }
38 }

```

We make no particular effort regarding the implementation; for instance, nothing prevents overflow in the operations on modular integers in the code snippet above. Moreover, the implementation of integers is based on machine integers. A true implementation would require arbitrary precision integers, as provided by **GMP**.

Implementing naive multiplication of polynomials over integers or modular integers can be done using the previously defined `mul!` macro.

```

1 mul!(&Poly<Nmod>, &Poly<Nmod>, PolyCtx<NmodCtx>, Poly<Nmod>,
    ↪ naive_mul);
2 mul!(&Poly<Integer>, &Poly<Integer>, PolyCtx<IntegerCtx>,
    ↪ Poly<Integer>, naive_mul);

```

The macro calls above avoid rewriting naive multiplication for each concrete type. Moreover, if we wish to use another multiplication method for a concrete coefficient type, we can again use the `mul!` macro without causing an implementation conflict. For instance, we can replace the implementation of polynomial multiplication for `Poly<Integer>` (that is, the second macro invocation in the code snippet above) with the following.

```

1  //another, very incorrect, multiplication method
2  fn bogo_mul<TCtx, T>(poly_ctx: &PolyCtx<TCtx>, f: &Poly<T>, g:
   ↪ &Poly<T>) -> Poly<T> {
3      vec! []
4  }
5
6  mul!(&Poly<Integer>, &Poly<Integer>, PolyCtx<IntegerCtx>,
   ↪ Poly<Integer>, bogo_mul);

```

Again, this implementation does not clash with the implementation of polynomial multiplication over `Poly<Nmod>`, that uses a different multiplication method.

To finish, let us actually multiply two polynomials.

```

1  fn main() {
2      let f = vec![1, 2, 3]; //1 + 2x + 3x^2
3      let g = vec![4, 5]; //4 + 5x
4
5      let z2z_poly = PolyCtx { t_ctx: NmodCtx { modulus: 2 } };
6      println!("{:?}", z2z_poly.mul(&f, &g));
7
8      let z_poly = PolyCtx { t_ctx: IntegerCtx {} };
9      println!("{:?}", z_poly.mul(&f, &g));
10 }

```

This code yields the multiplication of `f` and `g`, first seen as elements of $\mathbb{F}_2[x]$, then as elements of $\mathbb{Z}[x]$.

Chapter 6

Perspectives

6.1 Faster certified path tracking

Motivation Let $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a parametrized polynomial map. The step size in **Track** (Algorithm 5) is automatically determined by the ability to check Moore’s criterion for F_t over an interval (Theorem 2.3 and Proposition 2.12). In principle, any root-isolation criterion that can be checked using interval arithmetic can lead to a certified path tracking algorithm similar to **Track**. If the criterion is more optimistic than Moore’s criterion, it should allow longer steps, thus reducing the overall computation time. In what follows, we shall present an alternative to Moore’s criterion which we believe could potentially lead to improvements.

Order-0 criterion Let $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a polynomial map. To isolate zeros of f , one can consider the quasi-Newton iteration map φ for f with matrix $A \in \mathbb{C}^{n \times n}$ and try to apply a fixed-point theorem: if the application is successful and A is invertible, a fixed point of φ corresponds to a zero of f . For instance, we have the following result.

Proposition 6.1 (Order-0 root-isolation criterion). *Let $x \in \mathbb{C}^n$, $r \in \mathbb{R}_{>}$, and $A \in \mathbb{C}^{n \times n}$ be an invertible matrix. If for all $u \in rB$,*

$$u - Af(x + u) \in rB, \tag{6.1}$$

then there exists a zero of f in $x + rB$.

Proof. This is a direct consequence of Brouwer’s theorem applied to the quasi-Newton iteration map $\varphi(y) = y - Af(y)$ on $x + rB$, together with the invertibility of A . $\square$

Similarly, checking Moore’s criterion ensures that Banach’s fixed point theorem applies to the quasi-Newton iteration.

Order-0 boxes We call (x, r, A) an *order-0 box* for f if (6.1) holds and A is invertible. In theory, if Moore’s criterion is satisfied, then the order-0 criterion must also be satisfied, which suggests that the latter should allow longer step sizes. Obviously, this becomes less clear when using interval arithmetic.

Weaknesses The first weakness compared to Moore’s criterion is that the isolating region associated with an order-0 box is not guaranteed to contain a *unique* zero of f , unlike a Moore box. This problem can be addressed using the following observation: suppose instead that, for all $u \in rB$,

$$u - Af(x + u) \in \rho rB, \quad (6.2)$$

with $\rho \in (0, 1)$; in this case call (x, r, B) a ρ -*order-0 box*. We can apply the multivariate Theorem of Rouché (for instance, see [96, Theorem 2.1]): for u on the border of rB , we have

$$\|u - Af(x + u)\| \leq \rho r < \|u\|.$$

Thus the identity and $u \mapsto Af(x + u)$ have the same number of zeros in (the interior of) rB .

The second weakness is the lack of a way to converge to a zero of f . Convergence of the quasi-Newton iteration can be recovered using Cauchy’s inequalities:

Proposition 6.2. *Let (x, r, A) be a ρ -order-0 box for $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$. If $\rho \in (0, \frac{1}{2})$, then the zero of f in $x + rB$ is unique and is obtained as the limit of repeated quasi-Newton iterations $y \mapsto y - Af(y)$ starting from a point in $x + \rho rB$.*

Sketch of proof. From Cauchy’s integral formula, the differential of $\varphi(y) = y - Af(y)$ on $x + \rho rB$ is bounded (in operator norm) by $\frac{\rho}{1-\rho} < 1$. The result then follows from Banach’s fixed-point theorem. $\square$

Remark 6.1. It seems that the upper bound of $\frac{1}{2}$ for ρ can be improved to a value closer to 1.

When performing a predictor-corrector step, we first refine the endpoint of the prediction until we obtain a ρ -order-0 box with $\rho < \frac{1}{2}$. We can then try to certify the predictor using the order-0 criterion over an interval with ρ close to 1; hopefully, this suffices to prove termination.

The last difficulty with the order-0 criterion is Equation (6.1) itself: to check it, the natural approach is to replace u by rB :

$$rB - Af(x + rB) \subseteq rB. \quad (6.3)$$

However, (6.3) can hold only if $f = 0$. The original inclusion may hold only if we preserve the dependence on u in $u - Af(x + u)$. For an informal analysis, assume that $f(x) = 0$ and $A = df^{-1}(x)$. Expanding to order 1 gives

$$u - Af(x + u) = u - A(f(x) + df(x)(u) + O(\|u\|^2)) = O(r^2),$$

which belongs to rB when r is sufficiently small. This suggests expanding $u - Af(x + u)$ to order 1 before evaluating it using interval arithmetic. To this end, (6.1) should be evaluated using order-1 multivariate Taylor models in u .

Goal The first step is to compare the two criteria experimentally: how do the sizes of the isolating regions obtained from Moore’s criterion and the order-0 criterion compare? How do the criteria compare in terms of practical efficiency? If the results favor the order-0 criterion, the next step is to build a certified homotopy continuation algorithm based on the ideas of Chapter 2 and implement it, for instance as part of *Algpath*.

6.2 Connectivity queries in real algebraic curves

The ideas presented here are part of a research project initiated with Rémi Prébet.

Real algebraic geometry Let $S \subseteq \mathbb{R}^n$ be a real semi-algebraic set, that is, a set defined by a finite Boolean combination of polynomial equations and inequalities with real coefficients. The *connectivity query* problem asks whether two given points are path-connected in S . It is fundamental in real algebraic geometry and has applications in algorithmic geometry [5] and robotics [27, 28]. In robotics, semi-algebraic sets are used to describe the possible positions of a robot, and determining whether the robot can move from one position to another amounts to a connectivity query in the associated semi-algebraic set.

Connectivity queries using roadmaps A *roadmap* of S is a real semi-algebraic curve $R \subseteq S$ whose intersection with each connected component of S is non-empty and connected (see [26, 6, 75] for references on computing roadmaps). Roadmaps are useful for answering connectivity queries in S : assuming that x and y belong to R , they are connected in S if and only if they are connected in R . Thus roadmaps reduce the connectivity query problem to dimension 1.

Symbolic approach Connectivity queries in dimension 1 (studied, for instance, in [50]) can be solved by computing a graph embedded in $\mathbb{R}^n$ that captures the geometry of the curve R . Such a graph is usually obtained by first computing a

graph for the projection of R onto a plane symbolically and then lifting it back to the ambient space. This process can be costly because projecting R may introduce new singularities. Moreover, it computes a graph isotopic to the entire curve, whereas checking connectivity between $x, y \in R$ should require exploring only the connected components containing x and y .

Numerical approach Instead, we could perform *curve tracking* on R , starting from x . The idea is similar to path tracking: we progress numerically along R in small steps while correcting the approximation using Newton iterations. For instance, curve tracking is studied by Burr, Byrd, and Lee [25]. Certification ensures that we do not jump between branches of the curve, which could compromise correctness by jumping from one connected component to another. This setting introduces difficulties compared with standard path tracking, particularly because we need to handle singularities of the curve.

6.3 Topology of algebraic varieties

Using certified homotopy continuation, and more specifically braid computations, we aim to compute topological invariants of higher-dimensional algebraic varieties. For example, Pichon-Pharabod [73] briefly describes how to compute the action of the monodromy on the homology of a fibration using braids. A first goal would be to describe the approach precisely and provide an implementation that relies on the braid computation software presented earlier. More generally, if a smooth projective complex algebraic variety X has dimension 1, we can build a simplicial complex homeomorphic to X by computing the monodromy of a generic projection onto $\mathbb{CP}^1$. If X has higher dimension, this approach needs to be refined. In this case, braids seem to be the appropriate tool for describing a simplicial structure on X . Computing such a simplicial structure would allow us to compute topological invariants of X directly, a topic of interest in computer algebra [72, 73].

Bibliography

- [1] G. Alefeld and G. Mayer. “Interval Analysis: Theory and Applications”. In: *Journal of Computational and Applied Mathematics* 121.1–2 (2000), pp. 421–464.
- [2] E. Artin. “Theory of Braids”. In: *Annals of Mathematics* 48.1 (1947), pp. 101–126. JSTOR: 1969218.
- [3] D. Barth and A. Wenz. “Computation of Belyi Maps with Prescribed Ramification and Applications in Galois Theory”. In: *Journal of Algebra* 569 (2021), pp. 616–642.
- [4] L. Bartholdi, X. Buff, H.-C. Graf Von Bothmer, and J. Kröker. “Algorithmic Construction of Hurwitz Maps”. In: *Experimental Mathematics* 24.1 (2015), pp. 76–92.
- [5] S. Basu, R. Pollack, and M.-F. Roy. *Algorithms in Real Algebraic Geometry*. Ed. by A. M. Cohen, H. Cohen, D. Eisenbud, M. F. Singer, and B. Sturmfels. Vol. 10. Algorithms and Computation in Mathematics. Berlin, Heidelberg: Springer, 2006.
- [6] S. Basu, R. Pollack, and M.-F. Roy. “Computing roadmaps of semi-algebraic sets on a variety”. In: *Journal of the American Mathematical Society* 13.1 (2000), pp. 55–82.
- [7] D. J. Bates, J. D. Hauenstein, A. J. Sommese, and C. W. Wampler. *Numerically Solving Polynomial Systems with Bertini*. Software, Environments, and Tools. Society for Industrial and Applied Mathematics, 2013.
- [8] R. Becker, M. Sagraloff, V. Sharma, and C. Yap. “A Near-Optimal Subdivision Algorithm for Complex Root Isolation Based on the Pellet Test and Newton Iteration”. In: *Journal of Symbolic Computation* 86 (2018), pp. 51–96.
- [9] C. Beltrán. “A Continuation Method to Solve Polynomial Systems and Its Complexity”. In: *Numer. Math.* 117.1 (2011), pp. 89–113.
- [10] C. Beltrán and A. Leykin. “Certified Numerical Homotopy Tracking”. In: *Experimental Mathematics* 21.1 (2012), pp. 69–83.

- [11] C. Beltrán and A. Leykin. “Robust Certified Numerical Homotopy Tracking”. In: *Found Comput Math* 13.2 (2013), pp. 253–295.
- [12] G. V. Belyi. “On Galois Extensions Of a Maximal Cyclotomic Field”. In: *Math. USSR Izv.* 14.2 (1980), pp. 247–256.
- [13] D. Berghaus, H. Monien, and D. Radchenko. “On the computation of modular forms on noncongruence subgroups”. In: *Math. Comp.* 93.347 (2024), pp. 1399–1425.
- [14] J. Berthomieu, C. Eder, and M. Safey El Din. “Msolve: A Library for Solving Polynomial Systems”. In: *Proceedings of the 2021 International Symposium on Symbolic and Algebraic Computation*. ISSAC ’21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 51–58.
- [15] M. Berz and G. Hoffstätter. “Computation and Application of Taylor Polynomials with Interval Remainder Bounds”. In: *Reliable Computing* 4.1 (1998), pp. 83–97.
- [16] S. Billey and R. Vakil. “Intersections of Schubert Varieties and Other Permutation Array Schemes”. In: *Algorithms in Algebraic Geometry*. Ed. by A. Dickenstein, F.-O. Schreyer, and A. J. Sommese. New York, NY: Springer, 2008, pp. 21–54.
- [17] L. Blum, F. Cucker, M. Shub, and S. Smale. *Complexity and Real Computation*. New York, NY: Springer, 1998.
- [18] L. Blum, M. Shub, and S. Smale. “On a Theory of Computation and Complexity over the Real Numbers: NP-Completeness, Recursive Functions and Universal Machines”. In: *Bulletin (New Series) of the American Mathematical Society* 21.1 (1989), pp. 1–46.
- [19] T. Brazelton and S. Raman. *Monodromy in the Space of Symmetric Cubic Surfaces with a Line*. 2024. arXiv: 2410.09270 [math].
- [20] P. Breiding, K. Rose, and S. Timme. “Certifying Zeros of Polynomial Systems Using Interval Arithmetic”. In: *ACM Trans. Math. Softw.* 49.1 (2023), 11:1–11:14.
- [21] P. Breiding and S. Timme. “HomotopyContinuation.Jl: A Package for Homotopy Continuation in Julia”. In: *Mathematical Software – ICMS 2018*. Ed. by J. H. Davenport, M. Kauers, G. Labahn, and J. Urban. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, pp. 458–465.
- [22] N. Bruin, J. Sijssling, and A. Zotine. “Numerical Computation of Endomorphism Rings of Jacobians”. In: *Open Book Series* 2.1 (2019), pp. 155–171.

- [23] P. Bürgisser. *Completeness and Reduction in Algebraic Complexity Theory*. Ed. by E. Becker, M. Bronstein, H. Cohen, D. Eisenbud, and R. Gilman. Vol. 7. Algorithms and Computation in Mathematics. Berlin, Heidelberg: Springer, 2000.
- [24] P. Bürgisser and F. Cucker. “On a Problem Posed by Steve Smale”. In: *Ann. Math.* 174.3 (2011), pp. 1785–1836.
- [25] M. Burr, M. Byrd, and K. Lee. “Certified Algebraic Curve Projections by Path Tracking”. In: *Proceedings of the 2025 International Symposium on Symbolic and Algebraic Computation*. ISSAC ’25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 87–96.
- [26] J. Canny. *The complexity of robot motion planning*. MIT press, 1988.
- [27] J. Capco, M. Safey El Din, and J. Schicho. “Positive dimensional parametric polynomial systems, connectivity queries and applications in robotics”. In: *Journal of Symbolic Computation* 115 (2023), pp. 320–345.
- [28] D. Chablat, R. Prébet, M. Safey El Din, D. Salunkhe, and P. Wenger. “Deciding Cuspidality of Manipulators through Computer Algebra and Algorithms in Real Algebraic Geometry”. In: *2022 International Symposium on Symbolic and Algebraic Computation*. Lille, France, 2022, pp. 439–448. URL: <https://hal.science/hal-03596704>.
- [29] T. Chen, T.-L. Lee, and T.-Y. Li. “Hom4PS-3: A Parallel Numerical Solver for Systems of Polynomial Equations Based on Polyhedral Homotopy Continuation Methods”. In: *Mathematical Software – ICMS 2014*. Ed. by H. Hong and C. Yap. Berlin, Heidelberg: Springer, 2014, pp. 183–190.
- [30] B. Deconinck and M. van Hoeij. “Computing Riemann Matrices of Algebraic Curves”. In: *Physica D: Nonlinear Phenomena*. Advances in Nonlinear Mathematics and Science: A Special Issue to Honor Vladimir Zakharov 152–153 (2001), pp. 28–46.
- [31] J.-P. Dedieu, G. Malajovich, and M. Shub. “Adaptive Step-Size Selection for Homotopy Methods to Solve Polynomial Equations”. In: *IMA J Numer Anal* 33.1 (2013), pp. 1–29.
- [32] F. J. Drexler. “Eine Methode zur Berechnung sämtlicher Lösungen von Polynomgleichungssystemen.” In: *Numerische Mathematik* 29 (1977), pp. 45–58.
- [33] T. Duff, V. Korotynskiy, T. Pajdla, and M. H. Regan. “Galois/Monodromy Groups for Decomposing Minimal Problems in 3D Reconstruction”. In: *SIAM Journal on Applied Algebra and Geometry* 6.4 (2022), pp. 740–772. eprint: <https://doi.org/10.1137/21M1422872>.

- [34] T. Duff and K. Lee. “Certified Homotopy Tracking Using the Krawczyk Method”. In: *Proc. ISSAC 2024*. New York, NY, USA: Association for Computing Machinery, 2024, pp. 274–282.
- [35] T. Duff and K. Lee. *Certifying Galois/Monodromy Actions via Homotopy Graphs*. 2026. arXiv: 2603.17288 [math].
- [36] D. S. Dummit. “Solving Solvable Quintics”. In: *Mathematics of Computation* 57.195 (1991), pp. 387–401.
- [37] N. D. Elkies. “The complex polynomials $P(x)$ with $\text{Gal}(P(x)-t) \cong M_{23}$ ”. In: *ANTS X—Proceedings of the Tenth Algorithmic Number Theory Symposium*. Vol. 1. Open Book Ser. Math. Sci. Publ., Berkeley, CA, 2013, pp. 359–367.
- [38] R. Fox and L. Neuwirth. “The Braid Groups”. In: *Mathematica Scandinavica* 10 (1962), pp. 119–126.
- [39] C. B. Garcia and W. I. Zangwill. “Finding All Solutions to Polynomial Systems and Other Systems of Equations”. In: *Mathematical Programming* 16.1 (1979), pp. 159–176.
- [40] A. Guillemot. “Certified Algebraic Path Tracking with Alpath”. In: *ACM Commun. Comput. Algebra* 59.3 (2026), pp. 53–56.
- [41] A. Guillemot and P. Lairez. *Computing Braids from Approximate Data*. 2026. arXiv: 2601.23073 [cs].
- [42] A. Guillemot and P. Lairez. “Validated Numerics for Algebraic Path Tracking”. In: *Proc. ISSAC 2024*. New York, NY, USA: Association for Computing Machinery, 2024, pp. 36–45.
- [43] A. Guillemot and J. Voight. *Belyi Map Verification Using Certified Path Tracking*. 2026. arXiv: 2604.15562 [math].
- [44] T. Gunji, S. Kim, M. Kojima, A. Takeda, K. Fujisawa, and T. Mizutani. “PHoM—A Polyhedral Homotopy Continuation Method for Polynomial Systems”. In: *Computing* 73.1 (2004), pp. 57–77.
- [45] J. Harris. “Galois Groups of Enumerative Problems”. In: *Duke Mathematical Journal* 46.4 (1979), pp. 685–724.
- [46] J. D. Hauenstein, I. Haywood, and A. C. Liddell. “An a Posteriori Certification Algorithm for Newton Homotopies”. In: *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*. ISSAC ’14. New York, NY, USA: Association for Computing Machinery, 2014, pp. 248–255.
- [47] J. D. Hauenstein and A. C. Liddell. “Certified Predictor–Corrector Tracking for Newton Homotopies”. In: *Journal of Symbolic Computation* 74 (2016), pp. 239–254.

- [48] J. D. Hauenstein, J. I. Rodriguez, and F. Sottile. “Numerical Computation of Galois Groups”. In: *Found Comput Math* 18.4 (2018), pp. 867–890.
- [49] B. Huber and B. Sturmfels. “A Polyhedral Method for Solving Sparse Polynomial Systems”. In: *Mathematics of Computation* 64.212 (1995), pp. 1541–1555.
- [50] M. N. Islam, A. Poteaux, and R. Prébet. “Algorithm for Connectivity Queries on Real Algebraic Curves”. In: *Proceedings of the 2023 International Symposium on Symbolic and Algebraic Computation*. ISSAC ’23. Tromsø, Norway: Association for Computing Machinery, 2023, pp. 345–353.
- [51] F. Johansson. “Arb: Efficient Arbitrary-Precision Midpoint-Radius Interval Arithmetic”. In: *IEEE Transactions on Computers* 66.8 (2017), pp. 1281–1292.
- [52] M. Joldes. “Rigorous Polynomial Approximations and Applications”. PhD thesis. Ecole normale supérieure de lyon - ENS LYON, 2011.
- [53] C. Kassel and V. Turaev. *Braid Groups*. Vol. 247. Graduate Texts in Mathematics. New York, NY: Springer, 2008.
- [54] R. B. Kearfott and Z. Xing. “An Interval Step Control for Continuation Methods”. In: *SIAM J. Numer. Anal.* 31.3 (1994), pp. 892–914.
- [55] S. Klabnik and C. Nichols. *The Rust Programming Language*. USA: No Starch Press, 2018.
- [56] M. Klug, M. Musty, S. Schiavone, and J. Voight. “Numerical Calculation of Three-Point Branched Covers of the Projective Line”. In: *LMS J. Comput. Math.* 17.1 (2014), pp. 379–430.
- [57] S. Kranich. *An Epsilon-Delta Bound for Plane Algebraic Curves and Its Use for Certified Homotopy Continuation of Systems of Plane Algebraic Curves*. 2016. arXiv: 1505.03432 [math].
- [58] R. Krawczyk. “Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken”. In: *Computing* 4.3 (1969), pp. 187–201.
- [59] B. Lambov. “Interval Arithmetic Using SSE-2”. In: *Reliab. Implement. Real Number Algorithms*. Ed. by P. Hertling, C. M. Hoffmann, W. Luther, and N. Revol. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2008, pp. 102–113.
- [60] S. Lang. *Undergraduate Analysis*. Ed. by S. Axler, F. W. Gehring, and P. R. Halmos. Undergraduate Texts in Mathematics. New York, NY: Springer, 1997.
- [61] A. Leykin. “Numerical Algebraic Geometry”. In: *J. Softw. Algebra Geom.* 3.1 (2011), pp. 5–10.

- [62] A. Leykin and F. Sottile. “Galois Groups of Schubert Problems via Homotopy Computation”. In: *Math. Comp.* 78.267 (2009), pp. 1749–1765.
- [63] A. Marchetti-Spaccamela, U. Nanni, and H. Rohnert. “Maintaining a Topological Order under Edge Insertions”. In: *Inf. Process. Lett.* 59.1 (1996), pp. 53–58.
- [64] M. Á. Marco-Buzunariz and M. Rodríguez. “SIROCCO: A Library for Certified Polynomial Root Continuation”. In: *Mathematical Software – ICMS 2016*. Ed. by G.-M. Greuel, T. Koch, P. Paule, and A. Sommese. Vol. 9725. Cham: Springer International Publishing, 2016, pp. 191–197.
- [65] R. E. Moore. “A Test for Existence of Solutions to Nonlinear Systems”. In: *SIAM Journal on Numerical Analysis* 14.4 (1977), pp. 611–615.
- [66] R. E. Moore, R. B. Kearfott, and M. J. Cloud. *Introduction to Interval Analysis*. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics, 2009.
- [67] A. Morgan. *Solving Polynomial Systems Using Continuation for Engineering and Scientific Problems*. Englewood Cliffs, N.J. : Prentice-Hall, 1987.
- [68] M. Musty, S. Schiavone, J. Sijsling, and J. Voight. “A Database of Belyi Maps”. In: *Open Book Series* 2.1 (2019), pp. 375–392.
- [69] A. Neumaier. *Interval Methods for Systems of Equations*. Encyclopedia of Mathematics and Its Applications. Cambridge: Cambridge University Press, 1991.
- [70] A. Neumaier. “Taylor Forms—Use and Limits”. In: *Reliable Computing* 9.1 (2003), pp. 43–79.
- [71] D. J. Pearce and P. H. J. Kelly. “A Dynamic Topological Sort Algorithm for Directed Acyclic Graphs”. In: *ACM J. Exp. Algorithmics* 11 (2007), 1.7–es.
- [72] E. Pichon-Pharabod. “A Semi-Numerical Algorithm for the Homology Lattice and Periods of Complex Elliptic Surfaces over $\mathbb{P}^1$ ”. In: *Journal of Symbolic Computation* 127 (2025), p. 102357.
- [73] E. Pichon-Pharabod. “Periods in Algebraic Geometry : Computations and Application to Feynman Integrals”. Theses. Université Paris-Saclay, 2024.
- [74] E. Pichon-Pharabod and S. Telen. *Galois Groups of Symmetric Cubic Surfaces*. 2025. arXiv: 2509.06785 [math.AG].
- [75] R. Prébet, M. Safey El Din, and É. Schost. “Computing roadmaps in unbounded smooth real algebraic sets II: Algorithm and complexity”. In: *Journal of Symbolic Computation* 135 (2026), p. 102532.
- [76] N. Revol and F. Rouillier. *MPFI*. 1999.

- [77] D. P. Roberts. “Hurwitz–Belyi Maps”. In: *Publications mathématiques de Besançon. Algèbre et théorie des nombres* (2018), pp. 25–67.
- [78] J. I. Rodriguez and B. Wang. *Numerical Computation of Braid Groups*. 2017. arXiv: 1711.07947 [math].
- [79] S. M. Rump. “Solving algebraic problems with high accuracy”. In: *A New Approach to Scientific Computation*. Ed. by U. W. Kulisch and W. L. Miranker. Academic Press, 1983, pp. 51–120.
- [80] L. Schneps. “Dessins d’enfants on the Riemann sphere”. In: *The Grothendieck Theory of Dessins d’Enfants*. London Mathematical Society Lecture Note Series. Cambridge University Press, 1994, pp. 47–78.
- [81] M. Shub and S. Smale. “Complexity of Bezout’s Theorem V: Polynomial Time”. In: *Theoretical Computer Science* 133.1 (1994), pp. 141–164.
- [82] M. Shub and S. Smale. “Complexity of Bezout’s Theorem I: Geometric Aspects”. In: *Journal of the American Mathematical Society* 6.2 (1993), pp. 459–501.
- [83] J. Sijsling and J. Voight. “On Computing Belyi Maps”. In: *Publications mathématiques de Besançon. Algèbre et théorie des nombres* 1 (2015), pp. 73–131.
- [84] S. Smale. “Newton’s Method Estimates from Data at One Point”. In: *The Merging of Disciplines: New Directions in Pure, Applied, and Computational Mathematics*. Ed. by R. E. Ewing, K. I. Gross, and C. F. Martin. New York, NY: Springer, 1986, pp. 185–196.
- [85] A. J. Sommese, J. Verschelde, and C. W. Wampler. “Introduction to Numerical Algebraic Geometry”. In: *Solving Polynomial Equations: Foundations, Algorithms, and Applications*. Ed. by M. Bronstein, A. M. Cohen, H. Cohen, D. Eisenbud, B. Sturmfels, A. Dickenstein, and I. Z. Emiris. Algorithms and Computation in Mathematics. Berlin, Heidelberg: Springer, 2005, pp. 301–337.
- [86] A. J. Sommese and C. W. Wampler. *The Numerical Solution of Systems of Polynomials Arising in Engineering and Science*. Hackensack, NJ : World Scientific, 2005.
- [87] B. Sturmfels. “Solving Algebraic Equations in Terms of A-Hypergeometric Series”. In: *Discrete Math.* 210.1–3 (2000), pp. 171–181.
- [88] H.-J. Su, J. M. McCarthy, M. Sosonkina, and L. T. Watson. “Algorithm 857: POLSYS_GLP—a Parallel General Linear Product Homotopy Code for Solving Polynomial Systems of Equations”. In: *ACM Trans. Math. Softw.* 32.4 (2006), pp. 561–579.

- [89] S. Telen, M. Van Barel, and J. Verschelde. “A Robust Numerical Path Tracking Algorithm for Polynomial Homotopy Continuation”. In: *SIAM J. on Sci. Comput.* 42.6 (2020), A3610–A3637.
- [90] The LMFDB Collaboration. *The L-functions and modular forms database*. <https://www.lmfdb.org>. 2026.
- [91] A. Turon. *Shipping Specialization: A Story of Soundness*. 2017. URL: <https://aturon.github.io/blog/2017/07/08/lifetime-dispatch/>.
- [92] J. van der Hoeven. *Ball arithmetic*. 2009. URL: <https://hal.science/hal-00432152>.
- [93] J. van der Hoeven. *Reliable Homotopy Continuation*. Research Report. LIX, Ecole polytechnique, 2015.
- [94] E. R. van Kampen. “On the Fundamental Group of an Algebraic Curve”. In: *American Journal of Mathematics* 55.1 (1933), pp. 255–260.
- [95] J. Verschelde. “Algorithm 795: PHCpack: A General-Purpose Solver for Polynomial Systems by Homotopy Continuation”. In: *ACM Trans. Math. Softw.* 25.2 (1999), pp. 251–276.
- [96] J. Verschelde and A. Haegemans. “Homotopies for Solving Polynomial Systems within a Bounded Domain”. In: *Theor. Comput. Sci.* 133.1 (1994), pp. 165–185.
- [97] J. Xu, M. Burr, and C. Yap. “An Approach for Certifying Homotopy Continuation Paths: Univariate Case”. In: *Proc. ISSAC 2018*. New York, NY, USA: Association for Computing Machinery, 2018, pp. 399–406.
- [98] O. Zariski. “On the Problem of Existence of Algebraic Functions of Two Variables Possessing a Given Branch Curve”. In: *American Journal of Mathematics* 51.2 (1929), pp. 305–328.